

---

## Chapter 8

# Design at the Register Transfer Level

---

### CHAPTER OBJECTIVES

---

1. Know how to use register transfer level (RTL) notation to describe register operations in a clocked sequential circuit.
2. Know how to declare edge-sensitive and level-sensitive (a) procedural blocks in Verilog, or (b) a process in VHDL.
3. Be able to write HDL code to avoid synthesizing accidental latches.
4. Be able to write controller-datapath models to avoid race conditions and mismatches between an HDL model and the circuit produced from it by a synthesis tool.
5. Understand HDL constructs for concurrent assignments.
6. Know which procedural assignment statements have immediate effect, and those which have deferred effect.
7. Know the distinction between blocking and nonblocking assignments in Verilog, or know the distinction between variable and signal assignments in VHDL.
8. Understand the operators in Verilog or VHDL, and know the difference between the logical shift and arithmetic shift operators in Verilog or VHDL.
9. Understand the case and loop constructs in Verilog or VHDL.
10. Be able to construct and use an algorithmic state machine (ASM) chart.
11. Know how to use a systematic, effective, and efficient methodology for datapath and controller design, based on an algorithmic state machine and datapath (ASMD) chart.
12. Know and use some of the fundamental features that distinguish SystemVerilog from Verilog-2005.

## 8.1 INTRODUCTION

---

The behavior of many digital systems depends on the history of their inputs, and the conditions that determine their future actions depend on the results of previous actions. Such systems are said to have “memory.” A digital system is a sequential logic system constructed with flip-flops and gates. Sequential circuits can be specified by means of state tables, as shown in Chapter 5. To specify a large digital system with a state table is very difficult, because the number of states can be enormous. To overcome this difficulty, digital systems are designed via a modular approach. The system is partitioned into subsystems, each of which performs some function. The modules are constructed from such digital devices as registers, decoders, multiplexers, arithmetic elements, and control logic. The various modules are interconnected with data paths and control signals to form a digital system. In this chapter, we will introduce a design methodology for describing and designing large, complex digital systems. The chapter concludes with a brief, selective introduction to SystemVerilog.

## 8.2 REGISTER TRANSFER LEVEL (RTL) NOTATION

---

The modules of a digital system are best defined by a set of registers and the operations that are performed on the binary information stored in them. Examples of register operations are *load*, *shift*, *clear*, and *increment*. Registers are assumed to be the basic components of the processor in a digital system. The information flow and processing performed on the data stored in the registers are referred to as *register transfer operations*. We'll see subsequently how a hardware description language (HDL) includes operators that correspond to the register transfer operations of a digital system. A digital system is represented at the *register transfer level* (RTL) when it is specified by the following three elements:

1. The set of registers in the system.
2. The operations that are performed on the data stored in the registers.
3. The control that supervises the sequence of operations in the system.

A hardware register is a connected group of flip-flops that stores binary information and has the capability of performing one or more elementary operations. A *register* can load new information or shift its contents to the right or the left. A *counter* is a register that increments a number by a fixed value (e.g., 1). A flip-flop is a one-bit register that can be set, cleared, or complemented. In fact, the flip-flops and associated gates of any sequential circuit can be called registers by this definition.

The operations executed on the information stored in registers are elementary operations that are performed in parallel on the bits of a data word during one clock cycle. The data produced by the operation may replace the binary information that was in the register before the operation executed. Alternatively, the result may be transferred/copied to another register (i.e., an operation on a register may leave its contents unchanged). The digital circuits introduced in Chapter 6 are registers that implement elementary

operations. A counter with a parallel load is able to perform the increment-by-one and load operations. A bidirectional shift register is able to perform the shift-right and shift-left operations by shifting its contents by one or more bits in a specified direction.

The operations in a digital system are controlled by signals that sequence the operations in a prescribed manner. Certain conditions that depend on results of previous operations may determine the sequence of future operations. The outputs of the control logic of a digital system are binary variables that initiate the various operations in the system's registers and move information along data paths.

Information transfer from one register to another is designated in symbolic form by means of a replacement operator. The statement

$$R2 \leftarrow R1$$

denotes a transfer of the contents of register  $R1$  into register  $R2$ —that is, a replacement of the contents of register  $R2$  by the contents of register  $R1$ . For example, an eight-bit register  $R2$  holding the value 01011010 could have its contents replaced by  $R1$  holding the value 10100101. By definition, the contents of the source register  $R1$  do not change after the transfer. They are merely copied to  $R2$ . The arrow symbolizes the transfer and its direction; it points from the register whose contents are being transferred and toward the register that will receive the contents. A control signal would determine when the operation actually executes.

The controller in a digital system is a finite state machine (see Chapter 5) whose outputs are the control signals governing the register operations. In synchronous machines, the operations are synchronized by the system clock. For example, register  $R2$  might be synchronized to have its contents replaced at the positive edge of the clock.

A statement that specifies a register transfer operation implies that a datapath (i.e., a set of circuit connections) is available from the outputs of the source register to the inputs of the destination register and that the destination register has a parallel load capability. Data can be transferred serially between registers, too, by repeatedly shifting their contents along a single wire, one bit at a time, taking multiple clock cycles. Normally, we want a register transfer operation to occur, not with every clock cycle, but only under a predetermined condition. A conditional statement governing a register transfer operation is symbolized with an if-then statement such as

$$\text{If } (TI = 1) \text{ then } (R2 \leftarrow R1)$$

where  $TI$  is a control signal generated in the control section. Note that the clock is not included explicitly as a variable in the register transfer statements. It is assumed that all transfers occur at a clock-edge transition (i.e., a transition from 0 to 1 or from 1 to 0). Although a control condition such as  $TI$  may become true before the clock transition, the actual transfer does not occur until the clock transition does. The transfers are initiated and synchronized by the action of the clock signal, but the actual transition of the outputs (in a physical system) does not result in instantaneous transitions at the outputs of the registers. Transfers are subject to propagation delays, which depend on the physical characteristics of the transistors implementing the flip-flops of the register and the wires connecting devices. There is always a delay, however small, between a cause and its effect in a physical system. Effects follow causes, and not vice versa.

A comma may be used in RTL notation to separate two or more operations that are executed at the same time (concurrently). Consider the statement

If ( $T3 = 1$ ) then ( $R2 \leftarrow R1, R1 \leftarrow R2$ )

This statement specifies an operation that exchanges the contents of two registers; moreover, the operation in both registers is triggered by the same clock edge, provided that  $T3 = 1$ . This simultaneous (concurrent) operation is possible with hardware registers that have edge-triggered flip-flops controlled by a common clock (synchronizing signal). Other examples of register transfers are as follows:

|                                |                                                      |
|--------------------------------|------------------------------------------------------|
| $R1 \leftarrow R1 + R2$        | Add contents of $R2$ to $R1$ ( $R1$ gets $R1 + R2$ ) |
| $R3 \leftarrow R3 + 1$         | Increment $R3$ by 1 (count upward)                   |
| $R4 \leftarrow \text{shr } R4$ | Shift right $R4$                                     |
| $R5 \leftarrow 0$              | Clear $R5$ to 0                                      |

In hardware, addition is done typically with a binary parallel adder, incrementing is done with a counter, and the shift operation is implemented with a shift register.

The type of operations most often encountered in digital systems can be classified into four categories:

1. Transfer operations, which transfer (i.e., copy) data from one register to another.
2. Arithmetic operations, which perform arithmetic (e.g., multiplication) on data in registers.
3. Logic operations, which perform bit manipulation (e.g., logical OR) of nonnumeric data in registers.
4. Shift operations, which shift data within a register.

The transfer operation does not change the information content of the data being moved from the source register to the destination register unless the source and destination are the same. The other three operations change the information content during the transfer. The register transfer notation and the symbols used to represent the various register transfer operations are not standardized. In this text, we employ two types of notation. The notation introduced in this section will be used informally to specify and explain digital systems at the register transfer level. The next section introduces the RTL symbols used in HDLs.

## 8.3 RTL DESCRIPTIONS

---

### VERILOG (Edge- and Level-Sensitive Behaviors)

Verilog descriptions of RTL operations use a combination of dataflow and behavioral constructs to specify the combinational logic functions and register operations implemented by hardware. Two distinctions are important: (1) Register transfers are specified

by means of procedural statements within an *edge-sensitive cyclic behavior*, and (2) Combinational circuit functions are specified at the RTL level by means of continuous assignment statements or by procedural assignment statements within a *level-sensitive cyclic behavior*. The language symbol used to designate a register transfer is either an equals sign (=) or an arrow (<=); the symbol used to specify a combinational circuit function is an equals sign.

Synchronization with the clock is represented by associating with an **always** statement an event control expression in which sensitivity to the clock event is qualified by a **posedge** or **negedge** keyword to indicate the active edge of the clock. The **always** keyword indicates that the associated block of statements will be executed repeatedly, for the life of the simulation. The @ operator and the event control expression preceding the block of statements synchronize the execution of the statements to the clock event.

The following examples show the various ways to specify register transfer operation in Verilog:

```
(a) assign S = A + B;           // Continuous assignment for addition operation
(b) always @ (A, B)           // Level-sensitive cyclic behavior
    S = A + B;                 // Combinational logic for addition operation
(c) always @ (negedge clock)  // Edge-sensitive cyclic behavior
    begin
        RA = RA + RB;          // Blocking procedural assignment for addition
        RD = RA;              // Register transfer operation
    end
(d) always @ (negedge clock)  // Edge-sensitive cyclic behavior
    begin                      // Concurrent signal assignments
        RA <= RA + RB;         // Nonblocking procedural assignment for addition
        RD <= RA;             // Register transfer operation
    end
```

Continuous assignments (e.g., **assign** S = A + B;) are used to represent and specify combinational logic. In simulation, a continuous assignment statement executes when a signal in the expression on the right-hand side changes. The effect of execution is immediate. (The simulation is suspended while the variable on the left-hand side is updated.) Similarly, a level-sensitive cyclic behavior (e.g., **always** @ (A, B)) executes during simulation when a change is detected by its sensitivity list (event control expression). The effect of assignments made by the = operator is immediate. The continuous assignment statement (**assign** S = A + B;) describes a binary adder with inputs A and B and output S. The target operand in a continuous assignment statement (S in this case) cannot be a register data type, but must be a type of net, for example, **wire**. The procedural assignment made in the level-sensitive cyclic behavior in the second example shows an alternative way of specifying a combinational circuit for addition. Within the cyclic behavior, the mechanism of the sensitivity list ensures that the output, S, will be updated whenever A, or B, or both change.

Two kinds of procedural assignments can be made in a Verilog procedural statement: *blocking* and *nonblocking*. They are distinguished by their symbols and by their operation. *Blocking* assignments use the equals symbol (=) as the assignment operator, and *nonblocking* assignments use the left arrow (<=) as the operator. Blocking assignment statements are executed *sequentially* in the order that they are listed in a sequential block; when they execute, they have an immediate effect on the contents of memory before the next statement can be executed.

Nonblocking assignments (<=) are made *concurrently*. A simulator implements this feature by evaluating the expression on the right-hand side of each nonblocking assignment in the list of such statements before making the assignment to their left-hand sides. Consequently, *there is no interaction between the result of any assignment and the evaluation of an expression affecting another assignment*. Also, the statements associated with an edge-sensitive cyclic behavior do not begin executing until the indicated edge-sensitive event occurs. Consider (c) in the example of a blocking assignment given above. In the list of blocking procedural assignments, the first statement transfers the sum ( $RA + RB$ ) to  $RA$ , and the second statement transfers the new value of  $RA$  into  $RD$ . The value in  $RA$  after the clock event is the sum of the values in  $RA$  and  $RB$  immediately before the clock event. At the completion of the operation, both  $RA$  and  $RD$  hold the same value. In the nonblocking procedural assignment ((d) above), the two assignments are performed concurrently, so that  $RD$  receives the original value of  $RA$ . The activity in both examples is launched by the clock undergoing a falling edge transition.

The registers in a system are clocked simultaneously (concurrently). The  $D$  input of each flip-flop determines the value that will be assigned to its output, independently of the input to any other flip-flop. To ensure synchronous operations in RTL design, and to ensure a match between an HDL model and the circuit synthesized from the model, it is necessary that nonblocking procedural assignments be used for all variables that are assigned a value within an edge-sensitive cyclic behavior. The nonblocking assignment that appears in an edge-sensitive cyclic behavior models the behavior of the hardware of a synchronous sequential circuit accurately. In general, the blocking assignment operator (=) is used in a procedural assignment statement only when it is necessary to specify a sequential ordering of multiple assignment statements, as in a testbench or in combinational logic.

---

### Practice Exercise 8.1—Verilog

If  $RA$ ,  $RB$ , and  $RD$  are four-bit registers, and  $RA = 0001$  and  $RB = 0010$  immediately before the active edge of the clock, what are the contents of  $RA$  and  $RD$  immediately after the clock if the following register operations are executed?

$RA <= RA + RB;$   
 $RB <= RA;$

**Answer:**  $RA = 0011$ ;  $RB = 0001$ ;

---

## VHDL (Edge- and Level-Sensitive Processes)

VHDL descriptions of RTL operations use a combination of dataflow and behavioral constructs to specify the combinational logic functions and register operations implemented by hardware. Two distinctions are important: (1) Combinational circuit functions are specified at the RTL-level by means of concurrent signal assignment statements or by sequential signal assignments, that is, signal assignment statements made within a *level-sensitive process*,<sup>1</sup> and (2) register transfers are specified by means of procedural statements within an edge-sensitive process.

Synchronization with the clock is represented by a sensitivity list that contains a clock signal, and by following the sensitivity list with an **if** statement whose primary clause decodes any asynchronous control signals, and whose secondary clause decodes the clock event to determine whether there was a rising or falling edge. The process executes repeatedly, subject to its sensitivity list, just as the registers of a digital system respond to the clock signal. The following examples show various ways to specify register transfer operations in VHDL:

```
(a) S <= A + B;                                // Concurrent signal assignment for addition operation
(b) process (A, B) begin                        // Level-sensitive process
    S <= A + B;
end process;
(c) process (clock) begin
    if clock'event and clock = '0' then begin
        VRA := VRA + VRB;                        // Variable assignment
        VRD := VRA;
        RA <= VRA + VRB;                          // Signal assignment
        RD <= VRA;
    end
end process;
(d) process (clock) begin
    RA <= RA + RB;
    RD <= RA;
end process;
```

Concurrent signal assignments generally represent and specify (implicitly) combinational logic. An exception is a conditional signal assignment having feedback. For example, the conditional signal assignment `q <= D when enable = '1' else q <= q;` implies the behavior of a transparent latch. In simulation, a concurrent signal assignment executes when the expression on the right-hand side changes. The effect of execution is immediate. (The signal on the left-hand side is updated immediately, at the current time step of the simulator.) Sequential signal assignments (in a process) are updated after the process executes the last statement. Similarly, a level-sensitive process, (e.g., **process** (A, B)) executes in simulation when a change is detected in a signal in its sensitivity list.

<sup>1</sup>Concurrent signal assignments are those which are made within the body of an architecture. The statements within a process are executed in the sequence in which they are listed, and assignments to signals are referred to as sequential signal assignments.

In simulation, a *variable assignment statement* assigns value to a variable immediately, before the next statement executes. In contrast, a *sequential signal assignment statement* schedules an assignment to the left-side signal, *but the assignment is not made until the process has evaluated its last statement*.

The actions of scheduling and assignment do not occur simultaneously. A simulator's event scheduling mechanism assures that all *variable* assignments caused by executing the process will be scheduled in the order they are generated, immediately (i.e., at the current time step of the simulator), but before any signal assignment statement assigns value. Consequently, the results of a signal assignment affect only subsequent executions of the process, but not the execution in which they are generated. If multiple statements assign value at the same time to the same signal in a process, the last such statement determines the result.

The concurrent signal assignments in (a) and (b) above update the value of  $S$  immediately, assigning the sum of  $A$  and  $B$  to  $S$ . In (c),  $VRA$  and  $VRB$  are assumed to be previously declared variables. The variable  $VRA$  is updated immediately; then  $VRD$  is updated with the new value of  $VRA$ . The process in (c) launches at the falling edge of *clock*. In (d), note that the assignments to  $RA$  and  $RD$  are made after the process executes the last statement.  $RA$  gets the sum  $RA + RB$  based on values of  $RA$  and  $RB$  at the clock event, and  $RD$  gets the value of  $RA$  at the clock event.

Two kinds of assignments may be made in a process: a variable assignment and a signal assignment. Variable assignments use the symbol  $:=$ , and signal assignments use the  $<=$  symbol. A list of signal assignments in a process is processed (evaluated) sequentially, but the assignments are not made until the process completes. In effect, both kinds of statements execute sequentially, but they differ in the assignment of their effects. Variable assignment statements have an immediate effect; signal assignment statements have a deferred effect. Consequently, there is no interaction between the result of a signal assignment and the evaluation of an expression affecting another assignment. These distinctions enable a simulator to mimic the concurrent activity of a hardware circuit.

The registers in a system are clocked simultaneously (concurrently) by a common (shared) clock.<sup>2</sup> The  $D$  input of each flip-flop determines the value that will be assigned to its output, independently of the input to any other flip-flop. To ensure synchronous operation in RTL design, and to ensure a match between a VHDL model and the circuit synthesized from the model, it is necessary that signal assignments be used for all signals that are assigned a value within an edge-sensitive process. The signal assignment mechanism that appears in a process models the behavior of the hardware of a synchronous sequential circuit accurately. In general, the variable assignment operator is used only when it is necessary to specify a sequential ordering of multiple variable assignment statements, such as in a testbench or in level-sensitive (combinational) logic. Otherwise, remember to use only signal assignments.

<sup>2</sup>Synchronizers are used when a system has multiple clock domains [5].



### Practice Exercise 8.2 – VHDL

Suppose *VRA*, *VRB*, and *VRD* are four-bit variables, and *RA*, *RB*, and *RD* are four-bit signals. If *VRA* = 0001 and *VRB* = 0010 immediately before the active edge of the clock, what are the contents of *VRA*, *VRD*, *RA*, and *RD* immediately after the clock if the following operations are executed?

```
VRA := VRA + VRB;
VRD := VRA;
RA <= VRA;
RD <= VRA;
```

**Answer:** *VRA* = 0011; *VRD* = 0011; *RA* = 0011; and *RD* = 0011.

## Operators

**Verilog** Verilog operators and their symbols are listed in Table 8.1. The arithmetic, logic, bitwise or reduction, and shift operators describe register transfer operations. The logical, relational, and equality operators specify control conditions and have Boolean expressions as their arguments. Operators in the same precedence group have the same precedence.

The operands of the arithmetic operators are numbers in a binary format. The +, −, \*, and / operators form the sum, difference, product, and quotient, respectively, of a pair of operands. The exponentiation operator (\*\*) was added to the language in 2001 and forms a double-precision floating-point value from a base and exponent having a real, integer, or signed value. Negative numbers are represented in 2's-complement form. The modulus operator produces the remainder from the division of two numbers. For example, 14 % 3 evaluates to 2.

There are two types of operators for binary words: bitwise and reduction. The bitwise operators perform a bit-by-bit operation on two vector operands to form a vector result. They take each bit in one operand and perform the operation with the corresponding bit in the other operand. The same symbol (e.g., &) is used for both operations, and the effect is determined by the context in which the symbol is used. Note that reduction NAND is not listed in the table. By combining reduction AND with negation the effect of reduction NAND can be obtained. Similarly for reduction NOR.

Negation (~) is a unary operator; it complements the bits of a single vector operand and to form a vector result. The reduction operators are also unary, acting on a single operand and producing a scalar (one-bit) result. They operate pairwise on the bits of a word, from right to left, and yield a one-bit result. For example, the reduction NOR (~|) results in 0 with operand 00101 and in 1 with operand 00000. The result of applying the NOR operation on the first two bits is used with the third bit, and so forth.

Negation is not used as a reduction operator—its operation on a vector produces a vector by complementing each bit of the operand. Truth tables for the bitwise operators acting on a pair of scalar operands are the same as those listed in Table 4.9 in Section 4.12 for the corresponding Verilog primitive (e.g., the **and** primitive and the **&** bitwise operator have

**Table 8.1**  
*Verilog 2001, 2005 HDL Operators*

| Operator Type        | Symbol    | Operation Performed    | Priority Group        |
|----------------------|-----------|------------------------|-----------------------|
| Arithmetic           | +         | addition               | 1 (unary), 4 (binary) |
|                      | −         | subtraction            | 1 (unary), 4 (binary) |
|                      | *         | multiplication         | 3                     |
|                      | /         | division               | 3                     |
|                      | **        | exponentiation         | 2                     |
|                      | %         | modulus                | 2                     |
| Bitwise or Reduction | ~         | negation (complement)  | 1                     |
|                      | &, ~&     | AND, NAND (reduction)  | 1                     |
|                      | , ~       | OR, NOR (reduction)    | 1                     |
|                      | ^, ~^     | XOR, XNOR (reduction)  | 1                     |
|                      | ^, ~^, ^~ | XOR, XNOR (binary)     | 9                     |
| Logical              | !         | negation               | 1                     |
|                      | &&        | AND (binary)           | 11                    |
|                      |           | OR (binary)            | 12                    |
|                      | &         | AND (binary)           | 8                     |
|                      |           | OR (binary)            | 10                    |
| Shift                | >>        | logical right shift    | 5                     |
|                      | <<        | logical left shift     | 5                     |
|                      | >>>       | arithmetic right shift | 5                     |
|                      | <<<       | arithmetic left shift  | 5                     |
| Relational           | >         | greater than           | 6                     |
|                      | <         | less than              | 6                     |
|                      | <=        | less than or equal     | 6                     |
|                      | >=        | greater than or equal  | 6                     |
| Equality             | ==        | equality               | 7                     |
|                      | !=        | inequality             | 7                     |
|                      | ===       | case equality          | 7                     |
|                      | !==       | case inequality        | 7                     |
| Conditional          | ? :       | ternary selection      | 13                    |
| Concatenation        | { } { }   | joins operands         | 14                    |

the same truth table). The output of an AND gate with two scalar inputs is the same as the result produced by operating on the two bits with the **&** operator.

### Practice Exercise 8.3 – Verilog

Write a continuous assignment statement to form the bitwise NOR of A and B;

**Answer:** `assign Y <= ~(A|B);`

The logical and relational operators are used to form Boolean expressions and can take variables or expressions as operands. (*Note:* A single variable is also an expression.) Used basically for determining true or false conditions, the logical and relational operators evaluate to 1 if the condition expressed is true and to 0 if the condition is false. If the condition is ambiguous, they evaluate to x. An operand evaluates to 0 if the value of the variable is equal to 0 and to 1 if the value is not equal to 0. For example, if  $A = 1010$  and  $B = 0000$ , then the expression  $A$  has the Boolean value 1 (the number is not equal to 0) and the expression  $B$  has the Boolean value 0. Results of other operations with these values of  $A$  and  $B$  are as follows:

|                     |                        |                               |
|---------------------|------------------------|-------------------------------|
| $A \&\& B = 0$      | // Logical AND:        | $(1010) \&\& (0000) = 0$      |
| $A \& B = 0000$     | // Bitwise AND:        | $(1010) \& (0000) = (0000)$   |
| $A \parallel B = 1$ | // Logical OR:         | $(1010) \parallel (0000) = 1$ |
| $A   B = 1010$      | // Bitwise OR:         | $(1010)   (0000) = (1010)$    |
| $!A = 0$            | // Logical negation    | $!(1010) = !(1) = 0$          |
| $\sim A = 0101$     | // Bitwise negation    | $\sim(1010) = (0101)$         |
| $!B = 1$            | // Logical negation    | $!(0000) = !(0) = 1$          |
| $\sim B = 1111$     | // Bitwise negation    | $\sim(0000) = 1111$           |
| $(A > B) = 1$       | // is greater than     |                               |
| $(A == B) = 0$      | // identity (equality) |                               |

The relational operators `===` and `!==` test for bitwise equality (identity) and inequality in Verilog's four-valued logic system. For example, if  $A = 0xx0$  and  $B = 0xx0$ , the test  $A===B$  would evaluate to true, but the test  $A==B$  would evaluate to x.

Verilog 2001 has logical and arithmetic shift operators. The logical shift operators shift a vector operand to the right or the left by a specified number of bits. The vacated bit positions are filled with zeros, regardless of the direction of the shift. For example, if  $R = 11010$ , then the statement

```
R = R >> 1;
```

shifts  $R$  to the right one position and produces the result 01101. In contrast, the arithmetic right-shift operator fills the vacated cell (the most significant bit (MSB)) with its original contents when the word is shifted to the right. The arithmetic left-shift operator fills the vacated cell with a 0 when the word is shifted to the left. The arithmetic right-shift operator is used when the sign extension of a number is important. If  $R = 11010$ , then the statement

```
R >>> 1;
```

produces the result  $R = 11101$ ; if  $R = 01101$ , it produces the result  $R = 00110$ . There is no distinction between the logical left-shift and the arithmetic left-shift operators.

Practice Exercise 8.4 – Verilog

If  $R = 1001$ , what is the value of  $R$  after the following statement executes?

```
R = R >>> 2;
```

**Answer:**  $R = 1110$

The Verilog concatenation operator provides a mechanism for appending multiple operands. It can be used to specify a shift, including the bits transferred into the vacant positions. This aspect of its operation was shown in HDL Example 6.1 for the shift register.

Practice Exercise 8.5 – Verilog

If  $A = 0101$  and  $B = 1010$ , find the result produced by  $R = \{B, 2'b11, A\}$ .

**Answer:**  $R = 1010110101$

Verilog specifies that expressions are evaluated from left to right, and their operators associate from left to right (with the exception of the conditional operator) according to the precedence shown in Table 8.1. For example, in the expression

**Table 8.2**  
*Verilog Operator Precedence*

|                                                        |                                                                                     |
|--------------------------------------------------------|-------------------------------------------------------------------------------------|
| $+ - ! \sim \& \sim \&   \sim   ^ \sim ^ \sim$ (unary) | Highest precedence                                                                  |
| $**$                                                   |  |
| $* / \%$                                               |                                                                                     |
| $+ -$ (binary)                                         |                                                                                     |
| $<< >> <<< >>>$                                        |                                                                                     |
| $<< = >> =$                                            |                                                                                     |
| $== != === !=$                                         |                                                                                     |
| $\&$ (binary) (Bitwise, Reduction AND)                 |                                                                                     |
| $^ \sim \sim ^$ (binary) (Bitwise, Reduction OR)       |                                                                                     |
| $ $ (binary)                                           |                                                                                     |
| $\&\&$ (logical AND)                                   |                                                                                     |
| $  $ (logical OR)                                      |                                                                                     |
| $?:$ (conditional operator)                            |                                                                                     |
| $\{ \} \{ \{ \} \}$ (concatenation)                    |                                                                                     |
|                                                        | Lowest precedence                                                                   |

$A + B - C$ , the value of  $B$  is added to  $A$ , and then  $C$  is subtracted from the result. In the expression  $A + B/C$ , the value of  $B$  is divided by  $C$ , and then the result is added to  $A$ , because the division operator ( $/$ ) has a higher precedence than the addition operator ( $+$ ). Use parentheses to establish precedence and clarify intent. For example, the result produced by the expression  $(A + B)/C$  is not the same as that gotten from the expression  $A + B/C$ .

**VHDL** Table 4.12 presented the predefined operators of VHDL. The binary logical operators are used to form Boolean expressions and can be Boolean, Boolean vector, and bit vector operands. The type of the result is the same as the type of the operands. Used for determining TRUE or FALSE condition, the logical operator evaluates to TRUE if the operand expression is true, and FALSE if the operand evaluates to false. If the expression is ambiguous, they evaluate to x. With operands  $A = '1010'$  and  $B = '0000'$ , here are some: examples of operations:

| Operation   | Result              | Operation                    |
|-------------|---------------------|------------------------------|
| $A$ and $B$ | 0000                | Bitwise AND                  |
| $A$ or $B$  | 1010                | Bitwise OR                   |
| not $A$     | -- Logical negation |                              |
| $A > B$     | -- Greater than     | TRUE Relational greater than |
| $A = B$     | -- Equals           | FALSE Relational identity    |

VHDL has logical and arithmetic shift operators. The logical shift operators shift a vector operand to the right or left by a specified number of bits. For both operators, the vacated bit positions are filled with 0s. For example, if  $R = 11010$ , then the statement  $R \text{ srl } 1$  shifts  $R$  to the right one position. The value that results from the shift-right-logical operation ( $11010 \text{ srl } 1$  is  $01101$ ). In contrast, the arithmetic right-shift operator (**sra**) fills the vacated cell (the MSB) with its original contents when the word is shifted to the right. The result of the **sra** operation is  $11101$ . If  $R = 01101$  the result of the **sra** operation is  $00110$ . The arithmetic left-shift operator fills the LSB of a word with a 0 when the word is shifted to the left.<sup>3</sup>

### Practice Exercise 8.6 – VHDL

If  $R = 1001$ , what is the value of  $R$  after the following statement executes?

$R = R \text{ sra } 2;$

**Answer:**  $R = 1110$

<sup>3</sup>The **sll** and **sla** operators produce identical results.

The VHDL concatenation operator provides a mechanism for appending multiple operands. It can also be used to specify a shift, including the bits transferred into the vacant positions. This aspect of the concatenation operation was shown in HDL Example 6.1.

---

### Practice Exercise 8.7 – VHDL

If  $A = 0101$  and  $B = 1010$ , Find the result produced by  $R = B \& A$ .

**Answer:**  $R = 10100101$

---

VHDL specifies that expressions are evaluated from left to right and their operators associate from left to right, according to the precedence shown in Table 4.12. For example, in the expression  $A + B - C$ , the value of  $B$  is added to  $A$ , and then  $C$  is subtracted from the result. In the expression  $A + B/C$ , the value of  $B$  is divided by  $C$ , then the result is added to  $A$ , because the division operator ( $/$ ) has a higher precedence than the addition operator ( $+$ ). Use parentheses to establish precedence and clarify intent. For example, the result produced by the expression  $(A + B)/C$  is not the same as the result produced by  $A + B/C$ .

## Loop Statements

Loop statements govern repeated execution of procedural statements in Verilog behaviors and in VHDL processes.

**Verilog** Verilog HDL has four types of loops that execute procedural statements repeatedly: *repeat*, *forever*, *while*, and *for*. Looping statements may appear only inside an **initial** or **always** block.

The **repeat** loop executes the associated statements, unconditionally, for only a specified number of times. The following clock generator is an example that was used previously:

```
initial
begin
    clock = 1'b0;
    repeat (16) #5 clock = ~ clock;
end
```

This code initializes *clock* and then toggles the clock 16 times to produce eight clock cycles with a cycle time of 10 time units.

**Practice Exercise 8.8 – Verilog**

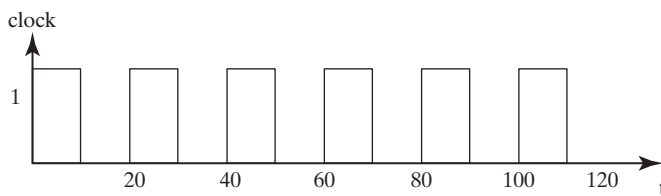
Draw the waveform of the signal produced by the following procedural statement:

```

initial
  begin
    clock = 1'b1;
    repeat (12) #10 clock = ~ clock;
  end

```

**Answer:** Figure PE 8.8



**FIGURE PE 8.8**

The **forever** loop causes unconditional, repetitive execution of a procedural statement or a block of procedural statements, without termination, for the life of the simulation. For example, the following loop produces a continuous, free-running clock that will run for the duration of simulation and have a cycle time of 20 time units:

```

initial
  begin
    clock = 1'b0;
    forever #10 clock = ~ clock;
  end

```

The **while** loop *conditionally* executes a statement or a block of statements repeatedly while an expression is true. If the expression is false to begin with, the statement is skipped and is never executed. The following example illustrates the use of the **while** loop:

```

integer count;
initial
  begin
    count = 0;
    while (count < 64)
      #5 count = count + 1;
  end

```

The value of *count* is incremented from 0 to 63. Each increment is delayed by five time units, and the loop exits at the count of 64.

In looping statements, the **integer** data type can index the loop. Integers are declared with the keyword **integer**, as in the previous example. Although it is possible to use a **reg** variable to index a loop, sometimes it is more convenient to declare an integer variable, rather than a **reg**, for counting purposes. Variables declared as data type **reg** are stored as unsigned numbers. Those declared as data type **integer** are stored as signed numbers in 2's-complement format. The default width of an integer is a minimum of 32 bits, which has implications for synthesis.

The **for** loop is a compact way to express the operations implied by a list of statements whose variables are indexed. The **for** loop contains three parts separated by two semicolons:

- An initial condition.
- An expression to check for the terminating condition.
- An assignment to change the control variable.

The following is an example of a **for** loop:

```
for (j = 0; j < 8; j = j + 1)
  begin
    // procedural statements go here
  end
```

The **for** loop statement repeats the execution of the procedural statements eight times. The control variable is *j*, the initial condition is  $j = 0$ , and the loop is repeated as long as *j* is less than 8. After each execution of the loop statement, the value of *j* is incremented by 1.

**VHDL** Loop statements govern the sequence in which VHDL statements within a process are executed. Loop statements may appear only in a **process**. VHDL has three types of loops that repeatedly execute sequential statements within a **process**. The simplest of these has the following syntax:

```
loop
  procedural statements
end loop;
```

The loop will execute without end unless a condition within the procedural statements causes termination. Termination can be caused by executing the **exit** statement, which terminates the loop unconditionally. The statements being executed in a loop will be aborted if the **next** statement is encountered. It causes the remaining statements to be skipped and begins executing the next iterant of the loop, but does not terminate the loop.

A **for** loop has the syntax below:

```
for identifier in range loop
  Procedural Statements
end loop;
```



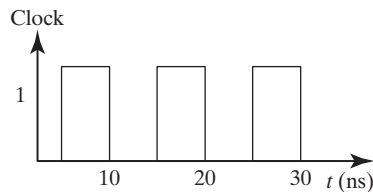
This form of loop executes conditionally, subject to the value of identifier being within a specified range.

### Practice Exercise 8.9 – VHDL

Draw the waveform of the signal produced by the following statements:

```
variable k: integer;
begin
  k := 0;
  clock <= 0;
  for k in range 0 to 3 loop
    clock = not clock after 5 ns;
    k := k + 1;
  end loop;
```

**Answer:** Figure PE 8.9



**FIGURE PE 8.9**

A **while** loop also executes conditionally, as governed by the syntax below:

```
while Boolean_Expression loop
  Procedural_Statements
end loop;
```

The **while** loop executes repeatedly, provided that a Boolean expression is TRUE. If the expression is FALSE when the statement is encountered, the statement will be skipped.

### HDL Example 8.1 (Decoder)

#### Verilog

A Verilog description of a two-to-four-line decoder using a **for** loop is shown below. Since output *Y* is evaluated in a procedural statement, it must be declared as type **reg**. The control variable for the loop is the **integer** *k*. When the loop is

expanded (unrolled), we get the following four conditions (*IN* and *Y* are in binary, and the index for *Y* is in decimal):

```
if IN = 00 then Y(0) = 1; else Y(0) = 0;
if IN = 01 then Y(1) = 1; else Y(1) = 0;
if IN = 10 then Y(2) = 1; else Y(2) = 0;
if IN = 11 then Y(3) = 1; else Y(3) = 0;
```

// Description of  $2 \times 4$  decoder using a **for** loop statement

```
module decoder (IN, Y);
  input      [1: 0] IN;           // Two binary inputs
  output     [3: 0] Y;           // Four binary outputs
  reg        [3: 0] Y;
  integer     k;                 // Control (index) variable for loop
  always @ (IN)
    for (k = 0; k <= 3; k = k + 1)
      if (IN == k) Y[k] = 1;
      else Y[k] = 0;
endmodule
```

## VHDL

// Description of  $2 \times 4$  decoder using a **for** loop statement

```
entity decoder is
  port (IN: in Std_Vector_Logic_(1 downto 0); Y: out Std_Vector_Logic (3 downto 0));
end decoder;

architecture Behavioral of decoder is
  integer k;
begin
  process (IN) begin
    for k in 0 to 3 loop
      if IN = k then Y(k) <= 1; else Y(k) <= 0;
      end if;
    end loop;
  end process;
end Behavioral
```

---

## Logic Synthesis with HDLs

Logic synthesis transforms an HDL model of a logic circuit into an optimized netlist of gates and registers that perform the operations specified by the source code. There are various target technologies that implement the synthesized design in hardware. The effective use of an HDL description requires that designers adopt a vendor-specific style

suitable for their particular synthesis tools. The type of ICs that implement the design may be an application-specific integrated circuit (ASIC), a programmable logic device (PLD), or a field-programmable gate array (FPGA). Logic synthesis is widely used in industry to design and implement large circuits efficiently, correctly, and rapidly. Logic synthesis tools interpret the source code of the HDL and translate it into an optimized gate structure, accomplishing (correctly) all of the work that would be done by manual methods using Karnaugh maps.

**Verilog** Designs written in Verilog or a comparable language for the purpose of logic synthesis tend to be at the register transfer level. This is because the Verilog constructs used in an RTL description can be converted into a gate-level description in a straightforward manner. The following examples discuss how a logic synthesizer can interpret a Verilog construct and convert it into a structure of gates.

A Verilog continuous assignment (**assign**) statement represents a Boolean equation for a combinational logic circuit. A continuous assignment with a Boolean expression for the right-hand side of the assignment statement is synthesized into the corresponding gate circuit implementing the expression. An expression with an addition operator (+) is interpreted as a binary adder using full-adder circuits. An expression with a subtraction operator (−) is converted into a gate-level subtractor consisting of full adders and exclusive-OR gates (Fig. 4.13). A statement with a conditional operator such as

```
assign Y = S ? In_1 : In_0;
```

translates into a two-to-one-line multiplexer with control input *S* and data inputs *In\_1* and *In\_0*. A statement with multiple conditional operators specifies a larger multiplexer.

A cyclic behavior (**always begin . . . end**) may imply a combinational or sequential circuit, depending on whether the event control expression is level sensitive or edge sensitive. A synthesis tool will interpret as combinational logic a level-sensitive cyclic behavior whose event control expression is sensitive to every variable that is referenced within the behavior (e.g., by the variable's appearing in the right-hand side of an assignment statement). The event control expression in a description of combinational logic may not be sensitive to an edge of any signal. For example,

```
always @ (In_1 or In_0 or S) // Alternative: @, In_0, S)
if (S) Y = In_1;
else Y = In_0;
```

translates into a two-to-one-line multiplexer. As an alternative, the **case** statement may be used to imply large multiplexers. The **casex** statement treats the logic values *x* and *z* as don't-cares when they appear in either the case expression or a case item.

An edge-sensitive cyclic behavior (e.g., **always @ (posedge clock) begin . . . end**) specifies a synchronous (clocked) sequential circuit. The implementation of the corresponding circuit consists of *D* flip-flops and the gates that implement the synchronous register transfer operations specified by the statements associated with the event control expression. Examples of such circuits are registers and counters. A sequential circuit description with a **case** statement translates into a control circuit with *D* flip-flops and

gates that form the inputs to the flip-flops. Thus, each statement in an RTL description is interpreted by the synthesizer and assigned to a corresponding gate and flip-flop circuit. For synthesizable sequential circuits, the event control expression must be sensitive to the positive or the negative edge of the clock (synchronizing signal), but not to both.

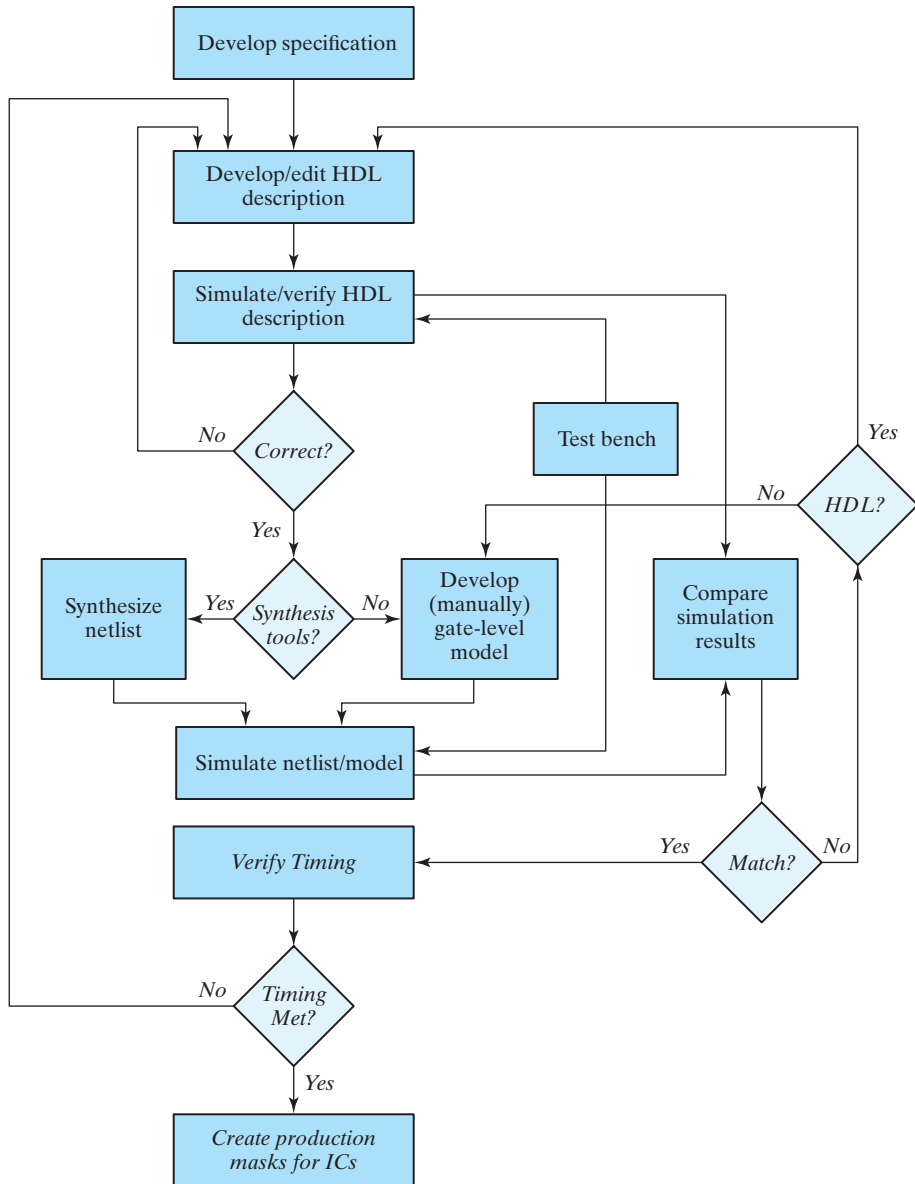
**VHDL** Logic synthesis tools associate VHDL concurrent signal assignment statements with combinational logic. The Boolean expressions on the right-hand side are translated into an optimized netlist, quickly accomplishing, without error, the work that could be done by manual methods using Karnaugh maps. The resulting designs tend to be at the register transfer level because VHDL operators and constructs have a direct correspondence to gate structures. For example, expressions with the  $+$  sign will be implemented with a binary adder. A conditional signal assignment, such as `Y <= In_0 when S = 1; else In_1;` will translate into a two-channel multiplexer controlled by *S*.

VHDL processes will translate into combinational or sequential circuits, depending on whether they imply combinational or sequential behavior. A level-sensitive process whose sensitivity list includes all of the variables that are read within the process implies combinational logic, provided that the signals that are assigned value within the process are assigned value by every path through the logic. Otherwise, the code may imply the need for a transparent latch. A feedback path within a process or a conditional signal assignment statement implies a latch. For example: `Y <= Data when En = 1; else Y;` will synthesis to a transparent latch. A common mistake is to fail to have a complete sensitivity list, which leads to synthesis of unwanted latches.

## Flowchart for Design

A simplified flowchart of the process used by industry to design digital systems is shown in Fig. 8.1. The RTL description of the HDL design is simulated and checked for proper operation. Its operational features must match those given in the specification for the behavior of the circuit. The testbench provides the stimulus signals to the simulator. If the result of the simulation is not satisfactory, the HDL description is corrected and checked again. After the simulation run shows a valid design, the RTL description is ready to be compiled by the logic synthesizer. All errors (syntax and functional) in the description must be eliminated before synthesis. The synthesis tool generates a netlist equivalent to a gate-level description of the design as it is represented by the model. If the model fails to express the functionality of the specification, the circuit will fail to do so also. Successful synthesis does not guarantee a correct design.

The design process requires that the gate-level circuit is simulated with the same set of stimuli used to check the RTL design. If any corrections are needed, the process is repeated until a satisfactory simulation is achieved. The results of the two simulations are compared to see if they match. If they do not, the designer must change the RTL description to correct any errors in the design. Then the description is compiled again by the logic synthesizer to generate a new gate-level description. Once the designer is satisfied with the results of all simulation tests, the design of the circuit is ready for physical implementation in a technology (e.g., a FPGA). In practice, additional testing



**FIGURE 8.1**  
A simplified flowchart for HDL-based modeling, verification, and synthesis

will be performed to verify that the timing specifications of the circuit can be met in the chosen hardware technology. That task is not within the scope of this text.

Logic synthesis provides several advantages to the designer. It takes less time to write an HDL description and synthesize a gate-level realization than it does to develop the

circuit by manual entry from K-maps, truth tables, or logic diagrams. The ease of changing an HDL model facilitates exploration of design alternatives. It is faster, easier, less expensive, and less risky to check the validity of the design by simulation than it is to produce a hardware prototype for evaluation. A schematic and the database for fabricating the integrated circuit can be generated automatically by synthesis tools. The HDL model can be compiled by different tools into different technologies (e.g., ASIC cells or FPGAs), providing multiple returns on the investment made to create and verify the model.

HDLs do not provide foolproof methodology to design logic circuits. The languages allow syntactically correct description of logic circuits that have no counterpart in reality. We will restrict our examples to those that demonstrate sound coding practices, and not consider purely academic examples of HDL code. It is important that students learn to write synthesizable HDL code.

## 8.4 ALGORITHMIC STATE MACHINES (ASMs)

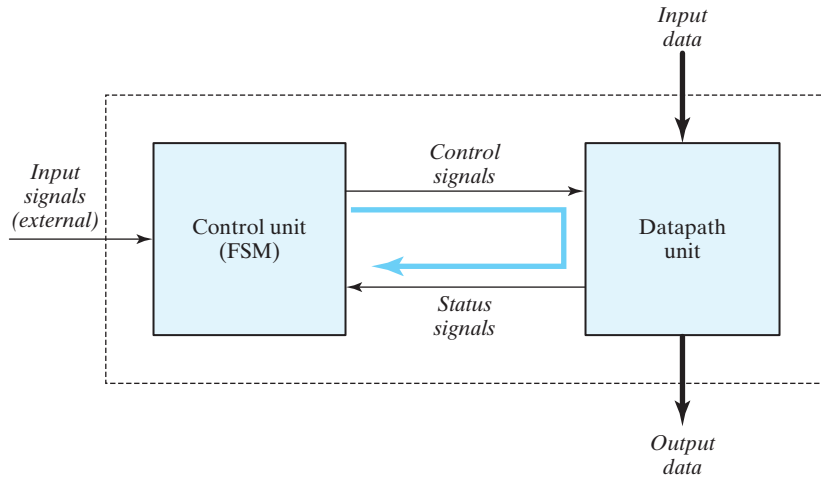
---

The binary information stored in a digital system can be classified as either data or control information. Data are discrete elements of information (binary words) that are manipulated by performing arithmetic, logic, shift, and other similar data-processing operations. These operations are implemented with digital hardware components such as adders, decoders, multiplexers, counters, and shift registers. Control information provides command signals that coordinate and execute the various operations in the data section of the machine in order to accomplish the desired data-processing tasks.

The design of the logic of a digital system can be divided into two distinct efforts. One is concerned with designing the digital circuits that perform the data-processing operations. The other is concerned with designing the control circuits that determine the sequence in which the various manipulations of data are performed.

The relationship between the control logic and the data-processing operations in a digital system is shown in Fig. 8.2. The data-processing path, commonly referred to as the *datapath unit*, manipulates data in registers according to the system's requirements. The *control unit* issues a sequence of commands to the datapath unit. Note that an internal feedback path from the datapath unit to the control unit provides status conditions that the control unit uses together with the external (primary) inputs to determine the sequence of control signals (outputs of the control unit) that direct the operation of the datapath unit. We'll see later how to correctly model this feedback relationship with an HDL.

The control logic that generates the signals for sequencing the operations in the datapath unit is a finite state machine (FSM), that is, a synchronous sequential circuit. The control commands for the system are produced by the FSM as functions of (1) the primary inputs, (2) the status signals, and (3) the state of the machine. In a given state, the outputs of the controller are the inputs to the datapath unit and determine the operations that it will execute. Depending on its present state, external inputs, and the status



**FIGURE 8.2**  
Control and datapath interaction

conditions of the datapath,<sup>4</sup> the FSM goes to its next state to initiate other operations. The digital circuits that act as the control logic provide a time sequence of signals for initiating the operations in the datapath and also determine the next state of the control subsystem itself.

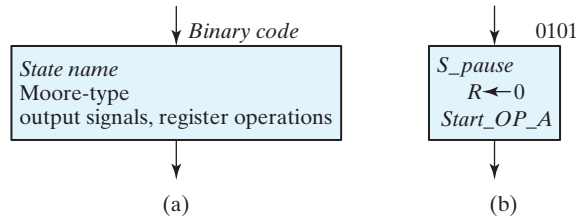
The control sequence and datapath tasks of a digital system are specified by means of a hardware algorithm. An algorithm consists of a finite number of procedural steps that specify how to obtain a solution to a problem. A hardware algorithm is a procedure for solving the problem with a given piece of equipment. The most challenging and creative part of digital design is the formulation of hardware algorithms for achieving required objectives. The goal is to implement the algorithms in silicon as an integrated circuit.

A flowchart is a convenient way to specify the sequence of procedural steps and decision paths for an algorithm. A flowchart for a hardware algorithm translates the verbal instructions to an information diagram that enumerates the sequence of operations together with the conditions necessary for their execution. An *algorithmic state machine* (ASM) chart is a special-purpose flowchart that has been developed to specifically define algorithms for execution on digital hardware. A *state machine* is another term for a sequential circuit, which is the basic structure of a digital system.

## ASM Chart

An ASM chart resembles a conventional flowchart, but it is interpreted somewhat differently. A conventional flowchart describes the procedural steps and decision paths of an algorithm in a sequential manner, without taking into consideration their time

<sup>4</sup> For example, a status signal could indicate that the contents of a register are valid and therefore ready to be read.



**FIGURE 8.3**  
ASM chart state box

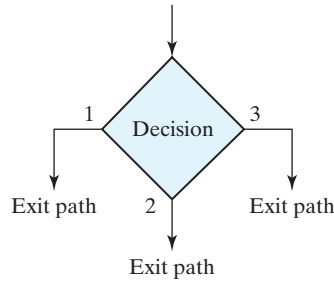
relationship. The ASM chart describes the sequence of events, that is, the ordering of events in time, as well as the timing relationship between the states of a sequential controller and the events that occur while going from one state to the next (i.e., the events that are synchronous with changes in the state). The chart is adapted to specify accurately the control sequence and datapath operations in a digital system, taking into consideration the constraints of digital hardware.

An ASM chart is composed of three basic elements: the state box, the decision box, and the conditional box. The boxes themselves are connected by directed edges indicating the sequential precedence and evolution of the states as the machine operates. There are various ways to attach information to an ASM chart. In one, a state in the control sequence is indicated by a state box, as shown in Fig. 8.3(a). The shape of the state box is a rectangle within which are written register operations that occur when the state transitions to its next state, or the names of output signals that the control generates while being in the indicated state. They govern the register operations that execute at the clock edge causing a state transition. The state is given a symbolic name, which is placed within the upper left corner of the box. The binary code assigned to the state is placed at the upper right corner. (The state symbol and code can be placed in other places as well.) Figure 8.3(b) gives an example of a state box. The state has the symbolic name *S\_pause*, and the binary code assigned to it is 0101. Inside the box is written the register operation  $R \leftarrow 0$ , which indicates that register *R* is to be cleared to 0 when the machine transitions to its next state. The name *Start\_OP\_A* inside the box indicates, for example, a Moore-type output signal that is asserted while the machine is in state *S\_pause*; the signal launches a certain operation in the datapath unit.

The style of state box shown in Fig. 8.3(b) is sometimes used in ASM charts, but *it can lead to confusion* about *when* the register operation  $R \leftarrow 0$  is to execute. Although the operation is written inside the state box, it actually occurs when the machine makes a transition from *S\_pause* to its next state. In fact, writing the register operation within the state box is a way (albeit possibly confusing) to indicate that the controller must assert a signal that will cause the register operation to occur when the machine changes state. *Later we'll introduce a chart and notation that are more suited to digital design and that will eliminate any ambiguity about the register operations controlled by a state machine.*

The decision box of an ASM chart describes the effect of an input (i.e., a primary, or external, input or a status, or internal, signal) on the control subsystem. The box is diamond shaped and has two or more exit paths, corresponding to possible state transitions,

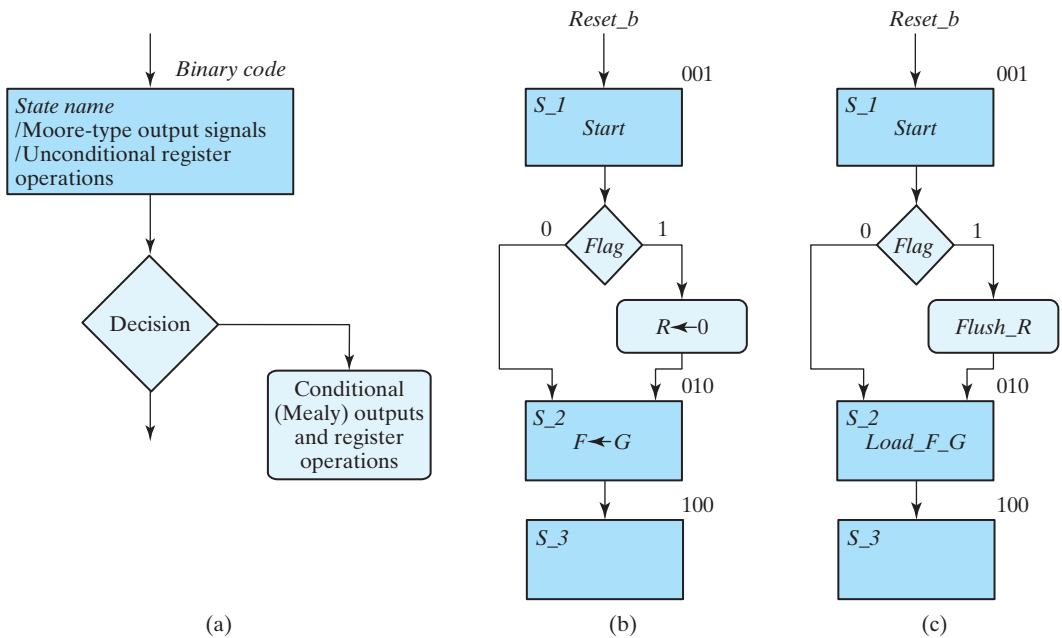




**FIGURE 8.4**  
ASM chart decision box

as shown in Fig. 8.4. The decision whose outcome is to be tested is written inside the box (e.g.,  $A < B$ ). One or the other exit path is taken, depending on the result of the test. In the binary case, one path is taken if the result is true and another when the result is false. When the result of an input decision is assigned a binary value, the two paths are indicated by the labels 1 (TRUE) and 0 (FALSE), respectively.

The state and decision boxes of an ASM chart are similar to those used in conventional flowcharts. The third element, the *conditional* box, is unique to the ASM chart. The shape of the conditional box is shown in Fig. 8.5(a). Its rounded corners differentiate



**FIGURE 8.5**  
ASM chart conditional box and examples

it from the state box. The input path to the conditional box must come from one of the exit paths of a decision box. The outputs listed inside the conditional box are generated as Mealy-type signals during a given state; the register operations listed in the conditional box are associated with a transition from the state. Figure 8.5(b) shows an example with a conditional box. The control unit generates the output signal *Start* while in state  $S_1$  and checks the value of input *Flag*. If  $Flag = 1$ , then  $R$  is cleared to 0; otherwise,  $R$  remains unchanged. In either case, the next state is  $S_2$ . A register operation is associated with  $S_2$ . We again note that *this style of chart can be a source of confusion*, because the state machine does not execute the indicated register operation  $R \leftarrow 0$  when it is in  $S_1$  or the operation  $F \leftarrow G$  when it is in  $S_2$ . The notation actually indicates that when the controller is in  $S_1$ , it must assert a Mealy-type signal that will cause the register operation  $R \leftarrow 0$  to execute in the datapath unit,<sup>5</sup> subject to the condition that  $Flag = 1$ , *at the next active edge of the clock*. Likewise, in state  $S_2$ , the controller must generate a Moore-type output signal that causes the register operation  $F \leftarrow G$  to execute in the datapath unit. The operations in the datapath unit are synchronized to the clock edge that causes the state to move from  $S_1$  to  $S_2$  and from  $S_2$  to  $S_3$ , respectively. Thus, *the control signal generated in a given state affects the operation of a register in the datapath when the next clock transition occurs*. The result of the operation is apparent in the next state.

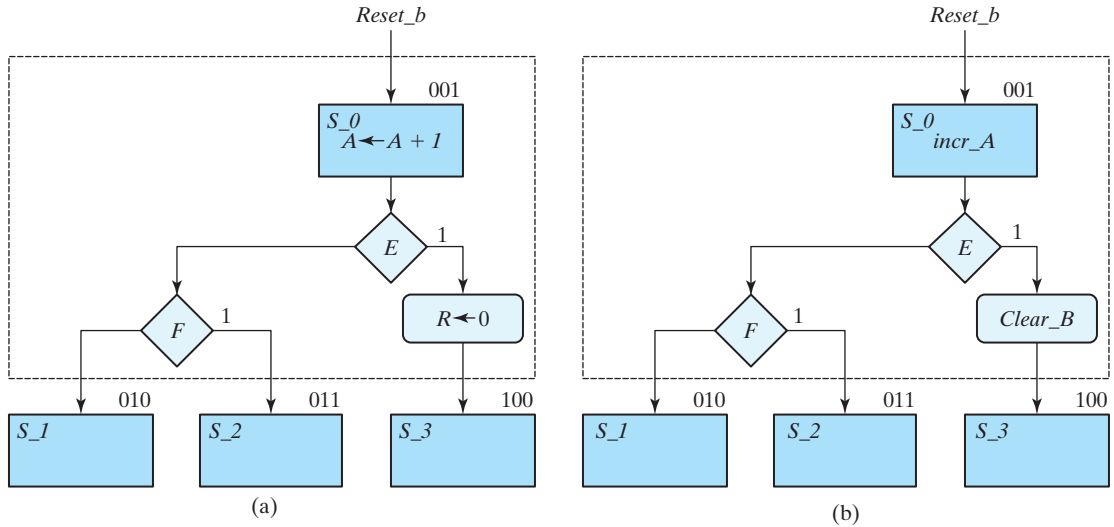
The ASM chart in Fig. 8.5(b) mixes descriptions of the datapath and the controller. An ASM chart for *only the controller* is shown in Fig. 8.5(c), in which the register operations are omitted. In their place are the control signals that must be generated by the control unit to launch the operations of the datapath unit. This chart is useful for describing the controller, but it does not contain adequate information about the datapath. (We'll address this issue later.)

## ASM Block

An ASM block is a structure consisting of one state box and all the decision and conditional boxes connected to its exit path. An ASM block has one entrance and any number of exit paths represented by the structure of the decision boxes. An ASM chart consists of one or more interconnected blocks. An example of an ASM block is given in Fig. 8.6(a). Associated with state  $S_0$  are two decision boxes and one conditional box. The diagram distinguishes the block with dashed lines around the entire structure, but this is not usually done, since the ASM chart uniquely defines each block from its structure. A state box without any decision or conditional boxes constitutes a simple block.

Each block in the ASM chart describes the state of the system during one clock-pulse interval (i.e., the interval between two successive active edges of the clock). The operations within the state and conditional boxes in Fig. 8.6(a) are initiated by a common clock pulse when the state of the controller transitions from  $S_0$  to its next state. The same clock pulse transfers the system controller to one of the next states,  $S_1$ ,  $S_2$ , or  $S_3$ , as dictated by the binary values of  $E$  and  $F$ . The ASM chart for the controller alone

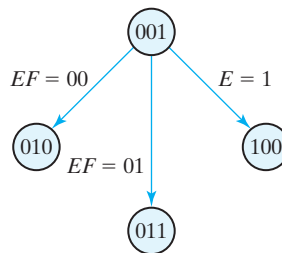
<sup>5</sup> If the path came from a state box the asserted signals would be Moore-type signals, dependent on only the state, and should be listed within the box.



**FIGURE 8.6**  
ASM blocks

is shown in Fig. 8.6(b). The Moore-type signal *incr\_A* is asserted unconditionally while the machine is in *S\_0*; the Mealy-type signal *Clear\_R* is generated conditionally when the state is *S\_0* and *E* is asserted. In general, the Moore-type outputs of the controller are generated unconditionally and are indicated within a state box; the Mealy-type outputs are generated conditionally and are indicated in the conditional boxes connected to the edges that leave a decision box.

The ASM chart is similar to a state transition diagram. Each state block is equivalent to a state in a sequential circuit. The decision box is equivalent to the binary information written along the directed lines that connect two states in a state diagram. As a consequence, it is sometimes convenient to convert the chart into a state diagram and then use sequential circuit procedures to design the control logic. As an illustration, the ASM chart of Fig. 8.6 is drawn as a state diagram (outputs are omitted) in Fig. 8.7. The states



**FIGURE 8.7**  
State diagram equivalent to the ASM chart of Fig. 8.6

are symbolized by circles, with their binary values written inside. The directed lines indicate the conditions that determine the next state. The unconditional and conditional operations that must be performed in the datapath unit are not indicated in the state diagram of the controller.

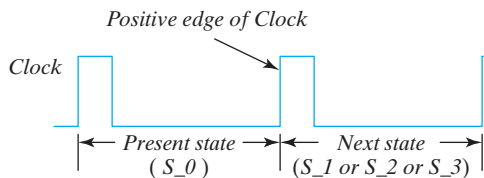
## Simplifications of an ASM Chart

A binary decision box of an ASM chart can be simplified by labeling only the edge corresponding to the asserted decision variable and leaving the other edge without a label. A further simplification is to omit the edges corresponding to the state transitions that occur when a reset condition is asserted. Output signals that are not asserted are not shown on the chart; the presence of the name of an output signal indicates that it is asserted.

## Timing Considerations

The timing for all registers and flip-flops in a digital system is controlled by a master-clock generator. The clock pulses are applied not only to the registers of the datapath but also to all the flip-flops in the state machine implementing the control unit. Inputs are also synchronized to the clock, because they are normally generated as outputs of another circuit that uses the same clock signals. If the input signal changes at an arbitrary time independently of the clock, we call it an asynchronous input. Asynchronous inputs may cause a variety of problems. To simplify the design, we will assume that all inputs are synchronized with the clock and change state in response to an edge transition.

*The major difference between a conventional flowchart and an ASM chart is in interpreting the time relationship among the various operations.* For example, if Fig. 8.6 were a conventional flowchart, then the operations listed would be considered to follow one after another in sequence: First register *A* is incremented, and only then is *E* evaluated. If  $E = 1$ , then register *R* is cleared and control goes to state  $S_3$ . Otherwise (if  $E = 0$ ), the next step is to evaluate *F* and go to state  $S_1$  or  $S_2$ . Activity is ordered, but there is no concept of timing or synchronization. In contrast, an ASM chart considers the entire block as one unit. All the register operations that are specified within the block must occur in synchronism at the edge transition of a common clock pulse while the system changes from  $S_0$  to the next state. This sequence of events is presented pictorially in Fig. 8.8. In this illustration, we assume positive-edge triggering of all flip-flops. An asserted asynchronous reset signal (*reset\_b*) transfers the control circuit into state  $S_0$ . While in state  $S_0$ , the control circuits check inputs *E* and *F* and generate



**FIGURE 8.8**  
Transition between states

appropriate signals accordingly. If *reset\_b* is not asserted, the following operations occur simultaneously at the next positive edge of the clock:

1. Register *A* is incremented.
2. If  $E = 1$ , register *R* is cleared.
3. Control transfers to the next state, as specified in Fig. 8.7.

Note that the two operations in the datapath and the change of state in the control logic occur *at the same time*. Note also that the ASM chart in Fig. 8.6(a) indicates the register operations that must occur in the datapath unit, but does not identify the control signal that is to be formed and provided by the control unit. Conversely, the chart in Fig. 8.6(b) indicates the control signals, but not the datapath operations. We will now present an ASMD chart to provide the clarity and complete information needed by designers of datapaths and their controllers, that is, digital processors.

### Practice Exercise 8.10

Draw an ASM chart for a synchronous state machine that is to monitor an input, *x\_in*, and assert *y\_out* after three consecutive 1s are observed, and remain asserted until a 0 is observed. It is implicit that the machine has a synchronous reset, but it is not shown on the chart.

**Answer:** Figure PE 8.10

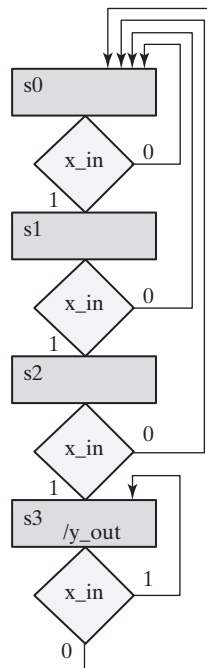


FIGURE PE 8.10

## ASMD Chart—The Rosetta Stone of Systematic Design

Algorithmic state machine and datapath (ASMD) charts clarify the information displayed by ASM charts and provide a *systematic, effective, and efficient* tool for designing a control unit for a given datapath unit. *An ASMD chart differs from an ASM chart in three important ways:* (1) An ASMD chart does not list register operations within a state box, (2) the edges of an ASMD chart are annotated with register operations that are concurrent with the state transition indicated by the edge, and (3) an ASMD chart includes conditional boxes identifying the signals which control the register operations that annotate the edges of the chart. Note that *an ASMD chart associates register operations with state transitions rather than with states; it also associates register operations with the signals that cause them*. Consequently, an ASMD chart represents a partition of a complex digital machine into its datapath and control units and clearly indicates the relationship between them. There is no room for confusion about the timing of register operations or about the signals that launch them.<sup>6</sup>

Designers form an ASMD chart in a three-step process that creates an annotated and completely specified ASM chart for the controller of a datapath unit.

Three steps form an ASMD chart:

1. Form an ASM chart showing only the states of the controller, decision boxes, and the names of input signals<sup>7</sup> that cause state transitions,
2. Convert the ASM chart into an ASMD chart by annotating the edges of the ASM chart to indicate the concurrent register operations of the datapath unit (i.e., register operations that are concurrent with a state transition), and
3. Modify the ASMD chart to identify the control signals that are generated by the controller and that cause the indicated operations in the datapath unit, as shown in (2).

The ASMD chart produced by this process clearly and completely specifies the finite state machine of the controller, identifies the registers operations of the datapath unit, identifies signals reporting the status of the datapath to the controller, and links register operations to the signals that control them. **The chart is language-neutral, and may be annotated with the symbols of whatever HDL is being used by a designer.**

One important use of a state machine is to control register operations on a datapath in a sequential machine that has been partitioned into a controller and a datapath. An ASMD chart links the ASM chart of the controller to the datapath it controls in a manner that serves as a universal model<sup>8</sup> representing all synchronous digital hardware design. ASMD charts help clarify the design of a sequential machine by separating the

<sup>6</sup> This distinction clarifies critical information about digital design and is so important that we take the liberty to refer to it as the Rosetta Stone of sequential machine design methodology.

<sup>7</sup> In general, the inputs to the control unit are external (primary) inputs and status signals that originate in the datapath unit.

<sup>8</sup> See Gajski, D. et al. "Essential Issues in Design." In: Staunstrup, J. Wolf W. Eds. *Hardware Software Co-Design: Principles and Practices*. Boston, MA: Kluwer, 1997

design of its datapath from the design of the controller, while maintaining a clear relationship between the two units. Register operations that occur concurrently with state transitions are annotated on a path of the chart, rather than in state boxes or in conditional boxes on the path, because these registers are not part of the controller. The outputs generated by the controller are the signals that control the registers of the datapath and cause the register operations annotated on the ASMD chart.

## 8.5 DESIGN EXAMPLE (ASMD CHART)

We will now present a simple example demonstrating the construction and explicit use of the ASMD chart and the register transfer representation. We start from the initial specifications of a system and proceed with the development of an appropriate ASMD chart from which the digital hardware is then designed.

The datapath unit is to consist of two *JK* flip-flops *E* and *F*, and one four-bit binary counter *A*. The individual flip-flops in *A* are denoted by  $A_3, A_2, A_1$ , and  $A_0$ , with  $A_3$  holding the most significant bit of the count. The initial state of the system is assumed to be a *reset state*, that is, the state reached, in this example, by application of an active-low reset signal, *reset\_b*. That state is *S\_idle*, so named because nothing happens until a signal, *Start*, initiates the system's operation by clearing the counter *A* and flip-flop *F*. At each subsequent clock pulse, the counter is incremented by 1 until the operations stop. Counter bits  $A_2$  and  $A_3$  determine the sequence of operations:

If  $A_2 = 0$ , *E* is cleared to 0 and the count continues.

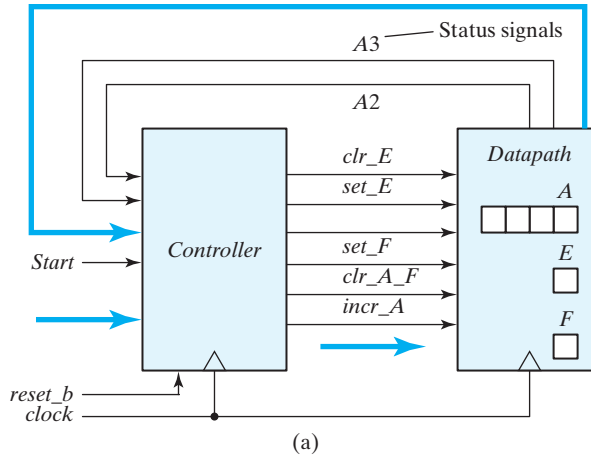
If  $A_2 = 1$ , *E* is set to 1; then, if  $A_3 = 0$ , the count continues, but if  $A_3 = 1$ , *F* is set to 1 on the next clock pulse and the system stops counting.

Then, if *Start* = 0, the system remains in the initial state, but if *Start* = 1, the operation cycle repeats.

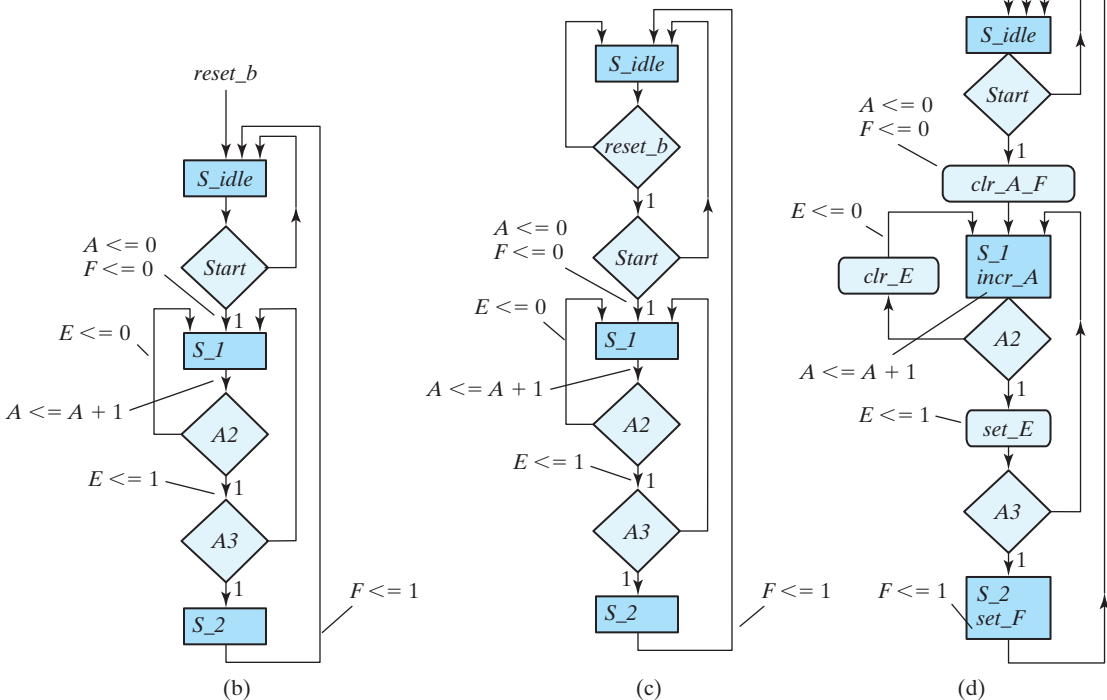
We begin with a block diagram of the system's architecture shown in Fig. 8.9(a), with (1) the registers of the datapath unit (*A*, *E*, *F*), (2) the external (primary) input signals (*Start*, *reset\_b*, *clock*), (3) the status signals fed back from the datapath unit to the control unit ( $A_2, A_3$ ), and (4) the control signals generated by the control unit and input to the datapath unit (*clr\_E*, *set\_E*, *set\_F*, *clr\_A\_F*, *incr\_A*). Note that the names of the control signals clearly indicate the operations that they cause to be executed in the datapath unit. For example, *clr\_A\_F* clears registers *A* and *F*. The name of the signal *reset\_b* (alternatively, *reset\_bar*) indicates that the reset action is active low. The internal details of each unit are not shown. We show the reset signal, *reset\_b*, connected to only the controller, assuming that it will assert control signals as needed to clear registers in the datapath unit.

### ASMD Chart

An annotated ASM chart for the system is shown in Fig. 8.9(b) for asynchronous reset action and in Fig. 8.9(c) for synchronous reset action. The chart shows the state transitions of the controller and the datapath operations associated with those transitions.



Note: A3 denotes A[3],  
A2 denotes A[2],  
 $\leq$  denotes nonblocking assignment  
reset\_b denotes active-low reset condition



**FIGURE 8.9**

(a) Block diagram for design example; (b) ASM chart for controller state transitions, annotated with datapath register operations, asynchronous reset; (c) ASM chart for controller state transitions, annotated with datapath register operations, synchronous reset; (d) ASMD chart for a completely specified controller, identifying datapath operations and associated control signals, and asynchronous reset



The chart is not in its final form, for it does not identify the control signals generated by the controller. The HDL assignment arrow ( $\leftarrow$ ) is shown to indicate register transfer operations because we will ultimately annotate the ASMD chart with a Verilog non-blocking assignment or a VHDL signal assignment.

When the reset action is synchronous, the transition to the reset state is synchronous with the clock. This transition is shown for  $S\_idle$  in the diagram, but *all other synchronous reset paths from other states are omitted to de-clutter the chart*. The system remains in the reset state,  $S\_idle$ , until  $Start$  is asserted. When that happens (i.e.,  $Start = 1$ ), the state moves to  $S\_1$ . At the next clock edge, depending on the values of  $A_2$  and  $A_3$  (decoded in a priority order), the state returns to  $S\_1$  or goes to  $S\_2$ . From  $S\_2$ , it moves unconditionally to  $S\_idle$ , where it awaits another assertion of  $Start$ .

The edges of the chart represent the state transitions that occur at the active (i.e., synchronizing) edge of the clock (e.g., the rising edge) and are annotated with the register operations that are to occur in the datapath. With  $Start$  asserted in  $S\_idle$ , the state will transition to  $S\_1$  and the registers  $A$  and  $F$  will be cleared. Note that, on the one hand, if a register operation is annotated on the *edge* leaving a state box, the operation occurs *unconditionally* and will be controlled by a *Moore*-type signal. For example, register  $A$  is incremented at every clock edge that occurs while the machine is in the state  $S\_1$ . On the other hand, the register operation setting register  $E$  annotates the edge leaving the *decision box* for  $A_2$ . The signal controlling the operation will be a *Mealy*-type signal asserted *conditionally* when the system is in state  $S\_1$  and  $A_2$  has the value 1. Likewise, the control signal clearing  $A$  and  $F$  is asserted *conditionally*: The system is in state  $S\_idle$  and  $Start$  is asserted.

In addition to showing that the counter is incremented in state  $S\_1$ , the annotated paths show that other operations occur conditionally with the same clock edge:

- Either  $E$  is cleared and control stays in state  $S\_1$  ( $A_2 = 0$ ) or
- $E$  is set and control stays in state  $S\_1$  ( $A_2A_3 = 10$ ) or
- $E$  is set and control goes to state  $S\_2$  ( $A_2A_3 = 11$ ).

When control is in state  $S\_2$ , a Moore-type control signal must be asserted to set flip-flop  $F$  to 1, and the state returns to  $S\_idle$  at the next active edge of the clock.

The third and final step in creating the ASMD chart is to insert conditional boxes for the signals generated by the controller or to insert Moore-type signals in the state boxes, as shown in Fig. 8.9(d). The signal  $clr\_A\_F$  is generated *conditionally* in state  $S\_idle$ , depending on  $Start$ ,  $incr\_A$  is generated *unconditionally* in  $S\_1$ ,  $clr\_E$  and  $set\_E$  are generated *conditionally* in  $S\_1$ , and  $set\_F$  is generated *unconditionally* in  $S\_2$ . The ASM chart has three states and three blocks. The block associated with  $S\_idle$  consists of the state box, one decision box, and one conditional box. The block associated with  $S\_2$  consists of only the state box. In addition to *clock* and *reset\_b*, the control logic has one external input,  $Start$ , and two status inputs,  $A_2$  and  $A_3$ .

In this example, we have shown how a verbal (text) description (specification) of a design is translated into an ASMD chart that completely describes the controller for the datapath, indicating the control signals and their associated register operations, and explicitly indicating the timing and synchronization of the complete machine. This design

example does not necessarily have a practical application, and in general, depending on the interpretation, the ASMD chart produced by the three-step design process for the controller may be simplified and formulated differently. However, once the ASMD chart is established, the procedure for designing the circuit is straightforward. *In practice the ASMD chart can be used to write HDL models of the controller and the datapath and then synthesize a circuit directly from the HDL description.* We will first design the system manually and then write the HDL description, keeping synthesis as an optional step for those who have access to synthesis tools.

## Timing Sequence

Every block in an ASMD chart specifies the signals which control the operations that are to be launched by one common clock pulse. The control signals specified within the state and conditional boxes in the block are asserted while the controller is in the indicated state, and the annotated register operations occur in the datapath unit when the state makes a transition along an edge that exits the state. The change from one state to the next is performed in the control logic. In order to appreciate the timing relationship involved, we list in Table 8.3 the step-by-step sequence of operations after each clock edge, beginning with an assertion of the signal *Start* until the system returns to the reset (initial) state, *S\_idle*.

Table 8.3 shows the binary values of the counter and the two flip-flops after every clock pulse. The table also shows separately the status of  $A_2$  and  $A_3$ , as well as the

**Table 8.3**  
*Sequence of Operations for Design Example*

| Counter |       |       | Flip-Flops |     | Conditions | State              |        |
|---------|-------|-------|------------|-----|------------|--------------------|--------|
| $A_3$   | $A_2$ | $A_1$ | $A_0$      | $E$ |            |                    | $F$    |
| 0       | 0     | 0     | 0          | 1   | 0          | $A_2 = 0, A_3 = 0$ | $S\_I$ |
| 0       | 0     | 0     | 1          | 0   | 0          |                    |        |
| 0       | 0     | 1     | 0          | 0   | 0          |                    |        |
| 0       | 0     | 1     | 1          | 0   | 0          |                    |        |
| 0       | 1     | 0     | 0          | 0   | 0          | $A_2 = 1, A_3 = 0$ |        |
| 0       | 1     | 0     | 1          | 1   | 0          |                    |        |
| 0       | 1     | 1     | 0          | 1   | 0          |                    |        |
| 0       | 1     | 1     | 1          | 1   | 0          |                    |        |
| 1       | 0     | 0     | 0          | 1   | 0          | $A_2 = 0, A_3 = 1$ |        |
| 1       | 0     | 0     | 1          | 0   | 0          |                    |        |
| 1       | 0     | 1     | 0          | 0   | 0          |                    |        |
| 1       | 0     | 1     | 1          | 0   | 0          |                    |        |
| 1       | 1     | 0     | 0          | 0   | 0          | $A_2 = 1, A_3 = 1$ |        |
| 1       | 1     | 0     | 1          | 1   | 0          |                    |        |
| 1       | 1     | 0     | 1          | 1   | 1          | $S\_idle$          |        |

present state of the controller. We begin with state  $S_1$  right after the input signal *Start* has caused the counter and flip-flop  $F$  to be cleared. The table shows the values of  $A_3, \dots, A_0, E$ , and  $F$  while the state is  $S_1$ . They were formed when the state made the transition from  $S_{idle}$  to  $S_1$  with  $Clr\_A\_F$  asserted. We will assume that the machine had been running before it entered  $S_{idle}$ , instead of entering it from a reset condition. Therefore, the value of  $E$  is assumed to be 1, because  $E$  is set to 1 when the machine enters  $S_2$ , before moving to  $S_{idle}$  (as shown at the bottom of the table), and because  $E$  does not change during the transition from  $S_{idle}$  to  $S_1$ . The system stays in state  $S_1$  during the next 13 clock pulses. Each pulse increments the counter and either clears or sets  $E$ . Note the relationship between the time at which  $A_2$  becomes a 1 and the time at which  $E$  is set to 1. When  $A = (A_3 A_2 A_1 A_0) 0011$ , the next (4th) clock pulse increments the counter to 0100, but that same clock edge sees the value of  $A_2$  as 0, so  $E$  remains cleared. The next (5th) pulse changes the counter from 0100 to 0101, and because  $A_2$  is equal to 1 *before* the clock pulse arrives,  $E$  is set to 1. Similarly,  $E$  is cleared to 0 not when the count goes from 0111 to 1000, but when it goes from 1000 to 1001, which is when  $A_2$  is 0 in the *present* value of the counter.

When the count reaches 1100, both  $A_2$  and  $A_3$  are equal to 1. The next clock edge increments  $A$  by 1, sets  $E$  to 1, and transfers control to state  $S_2$ . Control stays in  $S_2$  for only one clock period. The clock edge associated with the path leaving  $S_2$  sets flip-flop  $F$  to 1 and transfers control to state  $S_{idle}$ . The system stays in the initial state  $S_{idle}$  as long as *Start* is equal to 0.

From an observation of Table 8.3, it may seem that the operations performed on  $E$  are delayed by one clock pulse. This is the difference between an ASMD chart and a conventional flowchart. If Fig. 8.9(d) were a conventional flowchart, we would assume that  $A$  is first incremented and the incremented value would have been used to check the status of  $A_2$ . The operations that are performed in the digital hardware, as specified by a block in the ASMD chart, occur during the same clock cycle and not in a sequence of operations following each other in time, as is the usual interpretation in a conventional flowchart. Thus, the value of  $A_2$  to be considered in the decision box is taken from the value of the counter in the present state and before it is incremented. This is because the decision box affecting  $E$  belongs with the same block as state  $S_1$ . The digital circuits in the control unit generate the signals for all the operations specified in the present block *prior to the arrival of the next clock pulse*. The next clock edge executes all the operations in the registers and flip-flops, including the flip-flops in the controller that determine the next state, using the present values of the output signals of the controller. Thus, the signals that control the operations in the datapath unit are formed in the controller in the clock cycle (control state) *preceding* the clock edge at which the operations execute.

---

### Practice Exercise 8.11

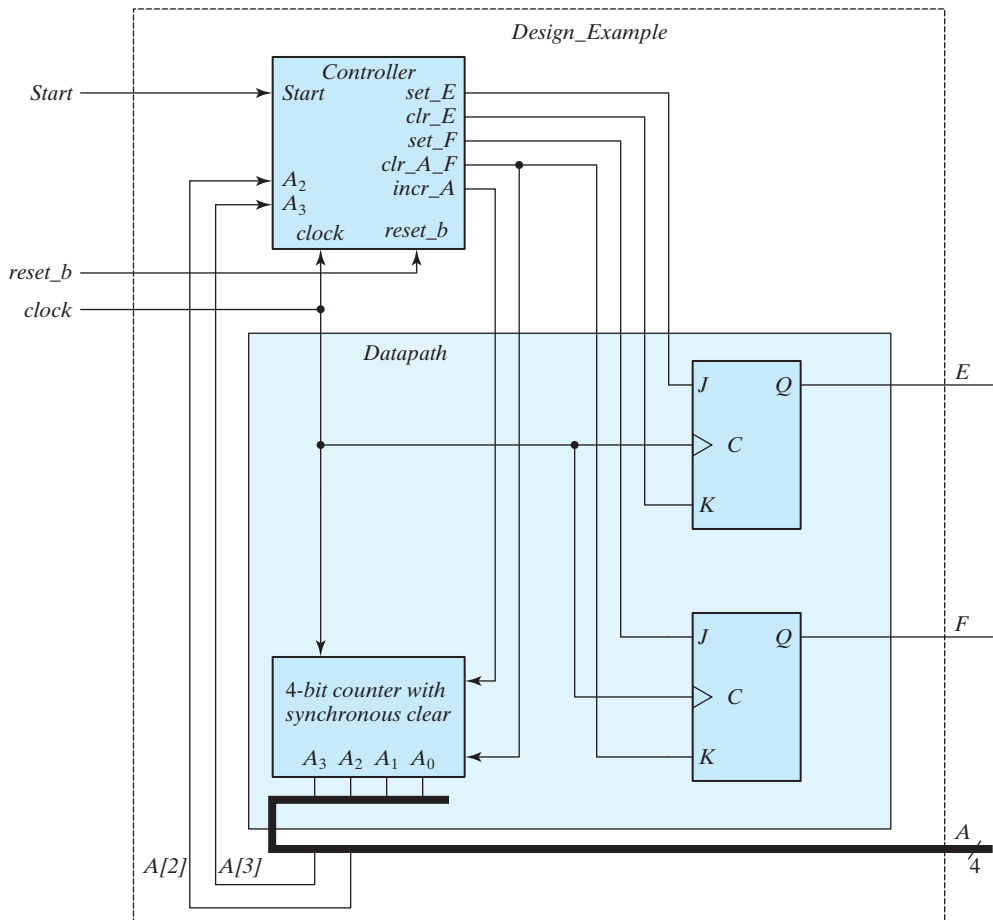
What information is annotated to the edges of an ASMD chart?

**Answer:** Register operations of the datapath are annotated to the edges of an ASMD chart.

---

## Smart and Effective Controller and Datapath Hardware Design

The ASMD chart provides all the information needed to design the digital system—the datapath and the controller. The actual boundary between the hardware of the controller and that of the datapath can be arbitrary, but we advocate, first, that the datapath unit contain only the hardware associated with its operations and the logic required, perhaps, to form status signals used by the controller, and, second, that the control unit contain all of the logic required to generate the signals that control the operations of the datapath unit. The requirements for the design of the datapath are indicated by the control signals inside the state and conditional boxes of the ASMD chart and are specified by the annotations of the edges indicating datapath operations. The control logic is determined from the decision boxes and the required state transitions. The hardware configuration of the datapath and controller in the preceding example is shown in Fig. 8.10.



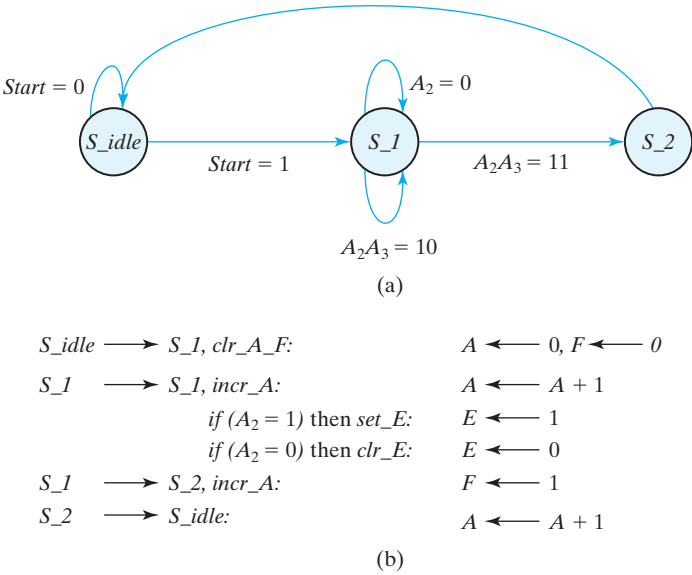
**FIGURE 8.10**  
Datapath and controller for design example

Note that the input signals of the control unit are the external (primary) inputs (*Start*, *reset\_b*, and *clock*) and the status signals from the datapath ( $A_2$  and  $A_3$ ). The status signals provide information about the present condition of the datapath. This information, together with the primary inputs and information about the present state of the machine, is used to form the output of the controller and the value of the next state. The outputs of the controller are inputs to the datapath and determine which operations will be executed when the clock undergoes a transition. Note, also, that the state of the control unit is not an output of the control unit.

The control subsystem is shown in Fig. 8.10 with only its inputs and outputs, with names matching those of the ASMD chart. The detailed design of the controller is considered subsequently. The datapath unit consists of a four-bit binary counter and two *JK* flip-flops. The counter is similar to the one shown in Fig. 6.12, except that additional internal gates are required for the synchronous clear operation. The counter is incremented with every clock pulse when the controller state is  $S_1$ . It is cleared only when control is at state  $S_{idle}$  and *Start* is equal to 1. The logic for the signal *clr\_A\_F* will be included in the controller and requires an AND gate to guarantee that both conditions are present. Similarly, we can anticipate that the controller will use AND gates to form signals *set\_E* and *clr\_E*. Depending on whether the controller is in state  $S_1$  and whether  $A_2$  is asserted, *set\_F* controls flip-flop *F* and is asserted *unconditionally* during state  $S_2$ . Note that all flip-flops and registers, including the flip-flops in the control unit, use a common clock.

## Register Transfer Representation

A digital system is represented at the register transfer level by specifying the registers in the system, the operations performed, and the control sequence. The register operations and control information can be specified with an ASMD chart. It is convenient to separate the control logic from the register operations of the datapath. The ASMD chart provides this separation and a clear sequence of steps to design a controller for a datapath. The control information and register transfer operations can also be represented separately, as shown in Fig. 8.11. The state diagram specifies the control sequence, and the register operations are represented by the register transfer notation introduced in Section 8.2. The state transition and the signal controlling the register operations are shown with the operation. This representation is an alternative to the representation of the system described in the ASMD chart of Fig. 8.9(d). Only the ASMD chart is really needed, but the state diagram for the controller is an alternative representation that is useful in design. The information for the state diagram is taken directly from the ASMD chart. The state names are specified in each state box. The conditions that cause a change of state are specified inside the diamond-shaped decision boxes of the ASMD chart and are used to annotate the state diagram. The directed lines between states, and the condition associated with each, follow the same path as in the ASMD chart. The register transfer operations for each of the three states are listed following the name of the state. They are taken from the state boxes or the annotated edges of the ASMD chart.



**FIGURE 8.11**  
Register transfer-level description of design example

State Table

The state diagram can be converted into a state table from which the sequential circuit of the controller can be designed. First, we must assign binary values to each state in the ASMD chart. For  $n$  flip-flops in the control sequential circuit, the ASMD chart can accommodate up to  $2^n$  states. A chart with 3 or 4 states requires a sequential circuit with two flip-flops. With 5 to 8 states, there is a need for three flip-flops. Each combination of flip-flop values represents a binary number for one of the states.

A *state table* for a controller is a list of present states and inputs and their corresponding next states and outputs. In most cases, there are many don't-care input conditions that must be included, so it is advisable to arrange the state table to take those conditions into consideration. We assign the following binary values to the three states:  $S\_idle = 00$ ,  $S\_1 = 01$ , and  $S\_2 = 11$ . Binary state 10 is not used and will be treated as a don't-care condition. The state table corresponding to the state diagram is shown in Table 8.4. Two flip-flops are needed, and they are labeled  $G_1$  and  $G_0$ . The controller has three inputs and five outputs. The inputs are taken from the conditions in the decision boxes, and consist of *Start* (a primary/external input), and the status signals ( $A_2$ ,  $A_3$ ) taken from the decision boxes. The outputs depend on the inputs and the present state of the control. Note that there is a row in the table for each possible transition between states. Initial state 00 goes to state

**Table 8.4**  
*State Table for the Controller of Fig. 8.10*

| Present-State<br>Symbol | Present<br>State |       | Inputs       |       |       | Next<br>State |       | Outputs      |              |              |                |               |
|-------------------------|------------------|-------|--------------|-------|-------|---------------|-------|--------------|--------------|--------------|----------------|---------------|
|                         | $G_1$            | $G_0$ | <i>Start</i> | $A_2$ | $A_3$ | $G_1$         | $G_0$ | <i>set_E</i> | <i>clr_E</i> | <i>set_F</i> | <i>clr_A_F</i> | <i>incr_A</i> |
| <i>S_idle</i>           | 0                | 0     | 0            | X     | X     | 0             | 0     | 0            | 0            | 0            | 0              | 0             |
| <i>S_idle</i>           | 0                | 0     | 1            | X     | X     | 0             | 1     | 0            | 0            | 0            | 1              | 0             |
| <i>S_1</i>              | 0                | 1     | X            | 0     | X     | 0             | 1     | 0            | 1            | 0            | 0              | 1             |
| <i>S_1</i>              | 0                | 1     | X            | 1     | 0     | 0             | 1     | 1            | 0            | 0            | 0              | 1             |
| <i>S_1</i>              | 0                | 1     | X            | 1     | 1     | 1             | 1     | 1            | 0            | 0            | 0              | 1             |
| <i>S_2</i>              | 1                | 1     | X            | X     | X     | 0             | 0     | 0            | 0            | 1            | 0              | 0             |

01 or stays in 00, depending on the value of input *Start*. The other two inputs are marked with don't-care X's, as they do not determine the next state in this case. While the system is in binary state 00 with *Start* = 1, the control unit provides an output labeled *clr\_A\_F* to initiate the required register operations. The transition from binary state 01 depends on inputs  $A_2$  and  $A_3$ . The system goes to binary state 11 only if  $A_2A_3 = 11$ ; otherwise, it remains in binary state 01. Finally, binary state 11 goes to 00 independently of the inputs.

## Control Logic

The procedure for designing a sequential circuit starting from a state table was presented in Chapter 5. If this procedure is applied to Table 8.4, we need to use five-variable maps to simplify the input equations. This is because there are five variables listed under the present-state and input columns of the table. Instead of using maps to simplify the input equations, we can obtain them directly from the state table by inspection. To design the sequential circuit of the controller with *D* flip-flops, it is necessary to go over the next-state columns in the state table and derive all the conditions that must set each flip-flop to 1. From Table 8.4, we note that the next-state column of  $G_1$  has a single 1 in the fifth row. The *D* input of flip-flop  $G_1$  must be equal to 1 during present state *S\_1* when both inputs  $A_2$  and  $A_3$  are equal to 1. This condition is expressed with the *D* flip-flop input equation

$$D_{G1} = S\_1A_2A_3$$

Similarly, the next-state column of  $G_0$  has four 1's, and the condition for setting this flip-flop is

$$D_{G0} = Start\ S\_idle + S\_1$$

To derive the five output functions, we can exploit the fact that binary state 10 is not used, which simplifies the equation for  $clr\_A\_F$  and enables us to obtain the following simplified set of output equations:

$$set\_E = S\_1A_2$$

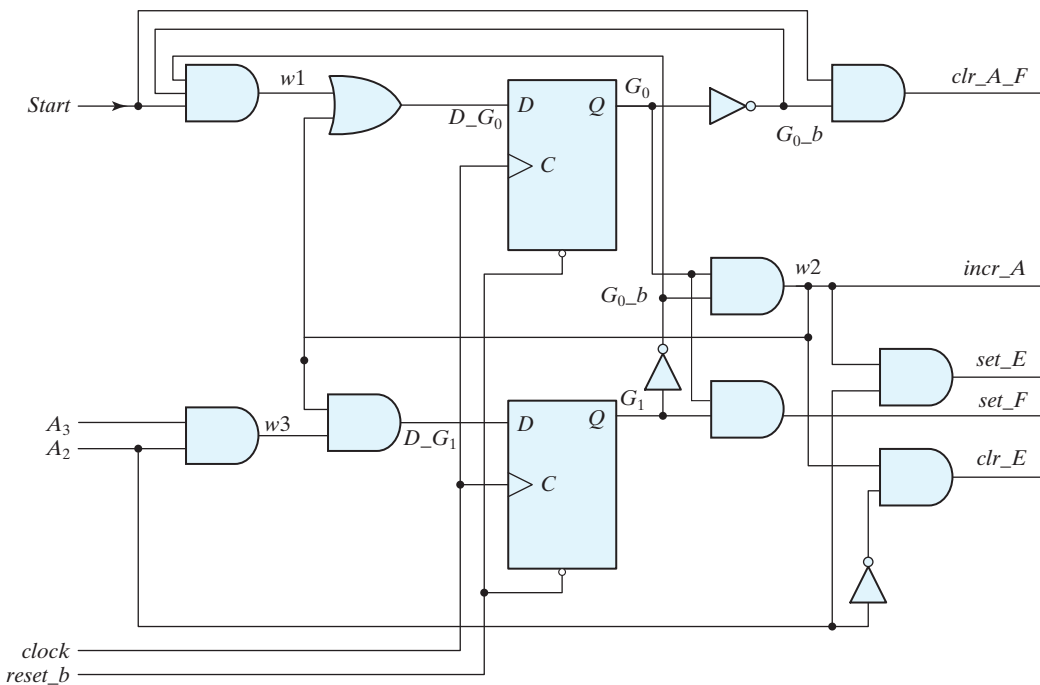
$$clr\_E = S\_1A_2'$$

$$set\_F = S\_2$$

$$clr\_A\_F = Start\ S\_idle$$

$$incr\_A = S\_1$$

The logic diagram showing the internal detail of the controller of Fig. 8.10 is drawn in Fig. 8.12. Note that although we derived the output equations from Table 8.4, they can also be obtained directly by inspection of Fig. 8.9(d). This simple example illustrates the manual design of a controller for a datapath, using an ASMD chart as a starting point. The fact that synthesis tools coupled with HDL models automatically execute these steps should be appreciated and exploited.



**FIGURE 8.12**  
Logic diagram of the control unit for Fig. 8.10



## 8.6 HDL DESCRIPTION OF DESIGN EXAMPLE

---

In previous chapters, we gave examples of HDL descriptions of combinational circuits, sequential circuits, and various standard components such as multiplexers, counters, and registers. We are now in a position to incorporate these components into the description of a specific design. As mentioned previously, a design can be described either at the structural or behavioral level. Behavioral descriptions may be classified as being either at the register transfer level or at an abstract algorithmic level. Consequently, we now consider three levels of design: structural description, RTL description, and algorithmic-based behavioral description.

The *structural* description is the lowest and most detailed level. The digital system is specified in terms of the physical components and their interconnection. The various components may include gates, flip-flops, and standard circuits such as multiplexers and counters. The design is hierarchically decomposed into functional units, and each unit is described by an HDL module. A top-level design unit combines the entire system by instantiating and connecting all the lower level design units. This style of description requires that the designer have sufficient experience not only to understand the functionality of the system but also to implement it by selecting and connecting other functional elements.

The *RTL* description specifies the digital system in terms of the registers, the operations performed, and the control that sequences the operations. This type of description simplifies the design process because it consists of procedural statements that determine the relationship between the various operations of the design without reference to any specific structure. The RTL description implies a certain hardware configuration among the registers, allowing the designer to create a design that can be synthesized automatically, rather than manually, into standard digital components.

The *algorithmic-based behavioral* description is the most abstract level, describing the function of the design in a procedural, algorithmic form similar to a programming language. It does not provide any detail on how the design is to be implemented with hardware. The algorithm-based behavioral description is most appropriate for simulating complex systems in order to verify design ideas and explore trade-offs. Descriptions at this level are accessible to nontechnical users who understand programming languages. Some algorithms, however, might not be synthesizable.

We will now illustrate the RTL and structural descriptions by using the design example of the previous section. The design example will serve as a model of coding style for future examples and will present alternative syntax options.

### RTL Description

---

#### HDL Example 8.2

The block diagram in Fig. 8.10 describes the design example. An HDL description of the design example can be written as a single RTL description or as a top-level design unit having instantiations of separate design units for the controller and the datapath.

The former option simply ignores the boundaries between the functional units; the design units in the latter option establish the boundaries shown in Fig. 8.9(a) and Fig. 8.10. We advocate the second option, because, in general, it distinguishes more clearly between the controller and the datapath, and supports the effort to verify the partitioned design. This choice also allows one to easily substitute alternative controllers for a given datapath (e.g., replace an RTL model by a structural model).

**Verilog** The description follows the ASMD chart of Fig. 8.9(d), which contains a complete description of the controller, the datapath, and the interface between them (i.e., the outputs of the controller and the status signals). Likewise, our description has three modules: *Design\_Example\_RTL*, *Controller\_RTL*, and *Datapath\_RTL*. The descriptions of the controller and the datapath units are taken directly from Fig. 8.9(d). The top-level design unit, *Design\_Example\_RTL*, declares the input and output ports of the machine and instantiates *Controller\_RTL* and *Datapath\_RTL*. At this stage of the description, it is important to remember to declare *A* as a vector. Failure to do so will produce *port mismatch* errors when the descriptions are compiled together, because the datapath declares *A* as a vector. Note that the status signals *A2* and *A3*, but not *A0* and *A1*, are passed to the controller. The primary (external) inputs to the controller are *Start*, *clock* (to synchronize the system), and *reset\_b*. The active-low input signal *reset\_b* is needed to initialize the state of the controller to *S\_idle*. Otherwise, the controller could not be placed in a known initial state.

The controller is described by three cyclic (**always**) behaviors. An edge-sensitive behavior updates the state at the positive edge of the clock, depending on whether a reset condition is asserted. Two level-sensitive behaviors describe the combinational logic for the next state and the outputs of the controller, as specified by the ASMD chart. Notice that the description includes default assignments to all of the outputs (e.g., *set \_E = 0*). This coding practice allows the code of the **case** logic to be simplified by expressing only explicit assertions of the variables (i.e., values are *assigned by exception*). The practice also ensures that every path through the assignment logic assigns a value to every variable. Thus, a synthesis tool will interpret the logic to be combinational; failure to assign a value to every variable on every path of logic implies the need for a transparent latch (memory) to implement the logic. Synthesis tools will provide the latch, wasting silicon area.

The three states of the controller are given symbolic names and are encoded into binary values. Only three of the possible two-bit patterns are used, so the case statement for the next-state logic includes a **default** assignment to handle the possibility that one of the three assigned codes is not detected. The alternative is to allow the hardware synthesis tool to make an arbitrary assignment to the next state (*next\_state = 2'bx*). Also, the first statement of the next-state logic assigns *next\_state = S\_idle* to guarantee that the next state is assigned in every thread of the logic. This is a precaution against accidentally forgetting to make an assignment to the next state in every thread of the logic, with the result that the description implies the need for memory, which a synthesis tool will implement with a transparent latch.

The description of *Datapath\_RTL* is written by testing for an assertion of each control signal from *Controller\_RTL*. The register transfer operations are displayed in the ASMD chart (Fig. 8.9(d)). Note that the signal assignments must ensure that the

register operations and state transitions are concurrent, a feature that is especially crucial during control state  $S_1$ . In this state,  $A$  is incremented by 1 and the value of  $A2$  ( $A/2$ ) is checked to determine the operation to execute at register  $E$  at the next clock. To accomplish a valid synchronous design, it is necessary to ensure that  $A/2$  is checked before  $A$  is incremented. If blocking assignments were used, the statement that checks  $E$  would have to be first, and the  $A$  statement that increments last. However, by using nonblocking assignments, we accomplish the required synchronization without being concerned about the order in which the statements are listed. The counter  $A$  in *Data-path\_RTL* is cleared synchronously because *clr\_A\_F* is synchronized to the clock.

The behaviors of the controller and the datapath interact in a chain reaction: At the active edge of the clock, the state and datapath registers are updated. A change in the state, a primary input, or a status input causes the level-sensitive behaviors of the controller to update the value of the next state and the outputs. The updated values are used at the *next* active edge of the clock to determine the state transition and the updates of the datapath.

Note that the manual method of design developed (1) a block diagram (Fig. 8.9(a)) showing the interface between the datapath and the controller, (2) an ASMD chart for the system (Fig. 8.9(d)), (3) the logic equations for the inputs to the flip-flops of the controller, and (4) a circuit that implements the controller (Fig. 8.12). In contrast, an RTL model describes the state transitions of the controller and the operations of the datapath as a step toward automatically synthesizing the circuit that implements them. The descriptions of the datapath and controller are derived directly from the ASMD chart in both cases. In summary: the ASMD chart provides a systematic, efficient, and effective methodology for designing a datapath and its controller.

```
// RTL description of design example
module Design_Example_RTL (A, E, F, Start, clock, reset_b);
// Specify ports of the top-level module of the design
// See block diagram, Fig. 8.10
output [3: 0] A;
output E, F;
input Start, clock, reset_b;

// Instantiate controller and datapath units
Controller_RTL M0 (set_E, clr_E, set_F, clr_A_F, incr_A, A[2], A[3], Start, clock, reset_b);
Datapath_RTL M1 (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
endmodule

module Controller_RTL (set_E, clr_E, set_F, clr_A_F, incr_A, A2, A3, Start, clock,
reset_b);
output reg set_E, clr_E, set_F, clr_A_F, incr_A;
input Start, A2, A3, clock, reset_b;
reg [1: 0] state, next_state;
parameter S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11; // State codes
```

```

always @ (posedge clock, negedge reset_b) // State transitions (edge sensitive)
if (reset_b == 0) state <= S_idle;
else state <= next_state;
// Code next-state logic directly from ASMD chart (Fig. 8.9d)
always @ (state, Start, A2, A3) begin // Next-state logic (level sensitive)
    next_state = S_idle;
    case (state)
        S_idle: if (Start) next_state = S_1; else next_state = S_idle;
        S_1: if (A2 & A3) next_state = S_2; else next_state = S_1;
        S_2: next_state = S_idle;
        default: next_state = S_idle;
    endcase
end
// Code output logic directly from ASMD chart (Fig. 8.9d)
always @ (state, Start, A2) begin
    set_E = 0; // Default assignments; assign by exception
    clr_E = 0;
    set_F = 0;
    clr_A_F = 0;
    incr_A = 0;
    case (state)
        S_idle: if (Start) clr_A_F = 1;
        S_1: begin incr_A = 1; if (A2) set_E = 1; else clr_E = 1; end
        S_2: set_F = 1;
    endcase
end
endmodule

module Datapath_RTL (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A, clock);
    output reg [3: 0] A; // Register for counter
    output reg E, F; // Flags
    input set_E, clr_E, set_F, clr_A_F, incr_A, clock;
    // Code register transfer operations directly from ASMD chart (Fig. 8.9(d))
    always @ (posedge clock) begin if (set_E) E <= 1;
        if (clr_E) E <= 0;
        if (set_F) F <= 1;
        if (clr_A_F) begin A <= 0; F <= 0; end
        if (incr_A) A <= A + 1;
    end
endmodule

```

---

**VHDL** The VHDL description of the design example follows the ASMD chart of Fig. 8.9(d), which contains a complete description of the controller, the datapath, and the interface between them (i.e., the status signals and the outputs of the controller).

Likewise, the VHDL description has three entities: *Design\_Example\_RTL\_vhdl*, *Controller\_RTL\_vhdl*, and *Datapath\_RTL\_vhdl*. The descriptions of the controller and datapath units are taken directly from Fig. 8.9(d). The top-level entity, *Design\_Example\_RTL\_vhdl* declares the input and output ports of the machine, and its companion architecture, *Behavioral*, declares and instantiates the controller and datapath entities as components in the design, and establishes the internal signal wiring via the declared port mappings.

Note that port *A* at the top level of entity is declared as a vector, which matches the corresponding port of the datapath unit. Failure to have that match will produce an error when the descriptions are compiled together. Also note that the status signals *A*(2) and *A*(3) are passed to the controller from the datapath unit, but not *A*(0) and *A*(1), which are not needed. The so-called primary (external) inputs to the controller are *Start*, *clock* (to synchronize the system), and *reset\_b* (to initialize the state of the controller). Without *reset\_b*, the controller could not be placed in a known initial state.

The behavioral description of the control unit is described by three concurrent processes. The first process detects rising-edge transitions of *clock*, and governs the state transition of the machine, subject to a synchronous reset signal. Note that the order in which *reset\_b* and *clock* are tested in this process dictates that *reset\_b* has priority. While it is asserted, active-low transitions of *clock* are ignored, and the state remains in *S\_idle*.

The second process provides state transition *combinational* logic determining the next state of the controller, as specified by the ASMD chart. The process is launched whenever *A*2, *A*3, *Start*, *clock*, or the state change. The process then decodes the present state and determines the next state. Notice that the **case** statement is preceded by a default assignment to *next\_state* (*next\_state* <= *S\_idle*). This assignment ensures that every path through the logic assigns a value to *next\_state*. Failure to assign a value to every signal through every path will result in the synthesis of a hardware latch (memory) That is an undesirable result because it wastes silicon area—the latch is not needed to implement combinational logic.

The third process describes the output logic of the controller. It is sensitive to *Start*, *clock*, and *state*, and determines the output signals as Mealy or Moore signals at each state. Notice that statements making default assignments de-asserting the outputs precede the case statement that decodes *state*. This coding practice simplifies the case logic by expressing only explicit assertions of the variables (i.e., values are *assigned by exception*). This approach also assures that every path through the assignment logic assigns a value to every signal. Thus, a synthesis tool will interpret the logic to be combinational (i.e., not requiring memory); failure to assign a value to every signal on every path of the logic implies the need for a transparent latch (memory) to implement the logic. Synthesis tools will oblige you, provide the latches, and waste silicon area.

The three states of the controller are given symbolic names in the specification of the data type *state\_type*. The actual binary encoding is not apparent.<sup>9</sup> We know that two flip-flops will be needed to encode three states, leaving one code unused. Therefore, the **case** statement has a specification for **others** to handle the possibility that one of the three assigned codes is not detected. The alternative is to allow the synthesis tool

<sup>9</sup> We will discuss explicit encoding later.

to make an arbitrary assignment to the next state (i.e., *next\_state* <= 'xx'). Also, the first statement of the next state logic assigns *next\_state* <= *S\_idle* to guarantee that the next state is assigned in every thread of the logic. This is a recommended precaution against accidentally forgetting to make an assignment to the next state in every thread of the logic, which would imply the need for memory and cause a synthesis tool to infer a transparent latch—creating another opportunity to waste silicon area.

The description of *Datapath\_RTL\_vhdl* tests for an assertion of each control signal from *Controller\_RTL\_vhdl* and assigns the register operations that are annotated on the ASMD chart in Fig. 8.9(d). The states are declared to be signals, and signal assignment operators are used. This ensures that the register operations and the state transitions are concurrent, a feature that is especially crucial during control state *S\_1*, where *A* is incremented by 1 and the value of *A2* (*A*(2)) is checked to determine the operation to execute at register *E* at the next active edge of *clock*. To accomplish a valid synchronous design, it is necessary to ensure that *A*(2) is checked *before* *A* is incremented. If variables were used, one would have to place the two statements that check *E* first, with the statement that increments *A* placed last. However, by using concurrent signal assignments, we accomplish the required synchronization without being concerned about the order in which the statements are listed. The counter, *A*, in *Datapath\_RTL\_vhdl* is cleared synchronously because *clr\_A\_F* is synchronized to *clock*.

The processes describing the controller and the datapath interact in a chain reaction: At the active edge of the clock, the state and datapath registers are updated. A change in the state, a primary input, or a status input causes the level-sensitive processes of the controller to update the value of the next state and the outputs. The updated values are used at the *next* active edge of the clock to determine the state transition and the updates of the datapath.

Note that the manual method of design developed (1) a block diagram (Fig. 8.9(a)) showing the interface between the datapath and the controller, (2) an ASMD chart for the system (Fig. 8.9(d)), (3) the logic equations for the inputs to the flip-flops of the controller, and (4) a circuit that implements the controller (Fig. 8.12). In contrast, an RTL model describes the state transitions of the controller and the operations of the datapath as a step toward automatically synthesizing the circuit that implements them. The descriptions of the datapath and controller are derived directly from the ASMD chart in both cases. In summary, the ASMD chart promotes a systematic, efficient, and effective methodology for designing a datapath and its controller.

```
library ieee;
use ieee.std_logic_1164.all;

entity Design_Example_RTL_vhdl is
  port (A: out std_logic_vector (3 downto 0);
        E, F: out std_logic; ;
        Start, clock, reset_b: in std_logic);
end Design_Example_RTL_vhdl;

architecture Behavioral of Design_Example_RTL_vhdl is
  component Controller_RTL_vhdl
    port (set_E, clr_E, set_F, clr_A_F : in Std_Logic,
```

```

    A2, A3 in Std_Logic; Start, clock, reset_b : in Std_Logic); end component;
component Datapath_RTL_vhdl
    port (A, E, F, set_E, clr_E, set_F, clr_A_F, incr_A : in Std_Logic;
        clock : in Std_Logic); end component;
begin
    M0: Controller_RTL_vhdl (port map
        set_E => set_E, clr_E => clr_E, set_F => set_F, clr_A_F => clr_A_F,
        A2 => A(2), A3 => A(3), Start => Start, clock => clock, reset_b => reset_b);

    M1: Datapath_RTL_vhdl (port map
        A => A, E => E, F => F, set_E => set_E, clr_E => clr_E, set_F => set_F,
        clr_A_F => clr_A_F, incr_A => incr_A, clock => clock);
end Behavioral;

entity Controller_RTL_vhdl is
    port (set_E, clr_E, set_F, clr_A_F: out std_logic;
        A2, A3, Start, clock, reset_b: in std_logic);
end Controller_RTL;

architecture Behavioral of Controller_RTL_vhdl is
    type state_type is (S_idle, S_1, S_2);
    signal state, next_state: state_type;
begin
    process (clock, reset_b) begin -- Synchronous State Transitions
        if reset_b = '0' then state <= S_idle;
        else if (clock'event) and clock = '1' then state <= next_state;
        end if
    end process;
    process (A2, A3, Start, clock, state) -- State Transition Logic
    begin
        next_state <= S_idle;
        case state is
            when S_idle => if Start = '1' then next_state <= S_1;
                                else next_state <= S_idle; end if;
            when S_1 => if A2 and A3 = TRUE then next_state <= S_2;
                                else next_state <= S_1; end if;
            when S_2 => next_state <= S_idle;
            when others => next_state <= S_idle;
        end case;
    end process;

    process (A2, A3, Start, state) -- Output Logic
    begin set_E <= 0;
        clr_E <= 0;
        set_F <= 0;

```

```

    clr_A_F  <= 0;
    incr_A   <= 0;
    case state is
    when S_idle => if Start = '1' then clr_A_F <= '1';
                    else next_state <= S_idle; end if;
    when S_1   => begin incr_A <= '1'; if S_2 = '1' then set_E <= '1';
                                else clr_E <= '1'; end if; end;

    when S_2   => set_F <= '1';
    end case;
end process;
end Behavioral;

entity Datapath_RTL_vhdl is
    port (A: out std_logic_vector (3 downto 0), E, F: out std_logic;
          set_E, clr_E, set_F, clr_A_F, incr_A, clock: in std_logic);
end Datapath_RTL_vhdl;

architecture Behavioral of Datapath_RTL_vhdl is
begin
    process (clock) -- Code register transfer operations – see ASMD Chart (Fig. 8.9(d)).
    begin
        if set_E = '1' then E <= '1'; end if;
        if clr_E = '1' then E <= '0'; end if;
        if set_F = '1' then F <= '1'; end if;
        if clr_A_F = '1' then begin A >= '0'; F <= '0'; end if; end;
        if incr_A = '1' then A <= A + 1; end if;
    end process;
end Behavioral;

```

## Testing the HDL Description

The sequence of operations for the design example was investigated in the previous section. Table 8.3 shows the values of *E* and *F* while register *A* is incremented. It is instructive to devise a test that checks the circuit to verify the validity of the HDL description. The testbench in HDL Example 8.3 is to accomplish that task. (The procedure for writing testbenches is explained in Section 4.12.) The testbench generates signals for *Start*, *clock*, and *reset\_b*, and checks the results obtained from registers *A*, *E*, and *F*. Initially, the *reset\_b* signal is set to 0 to initialize the controller, and *Start* and *clock* are set to 0. At time  $t = 5$ , the *reset\_b* signal is de-asserted by setting it to 1, the *Start* input is asserted by setting it to 1, and the clock is then repeated for 16 cycles. The values of *A*, *E*, and *F* will be examined every 10 ns. The output of the simulation is listed in the example under the simulation log. Initially, at time  $t = 0$ , the values of the registers are unknown, so they are marked with the symbol **x**. The first positive clock transition, at time  $t = 10$ , clears *A* and *F*, but does not affect *E*, so *E* is unknown at this time. The rest of the table is identical



to Table 8.3. Note that since *Start* is still equal to 1 at time = 160, the last entry in the table shows that *A* and *F* are cleared to 0, and *E* does not change and remains at 1. This occurs during the second transition, from *S\_idle* to *S\_1*.

### HDL Example 8.3

---

#### Verilog

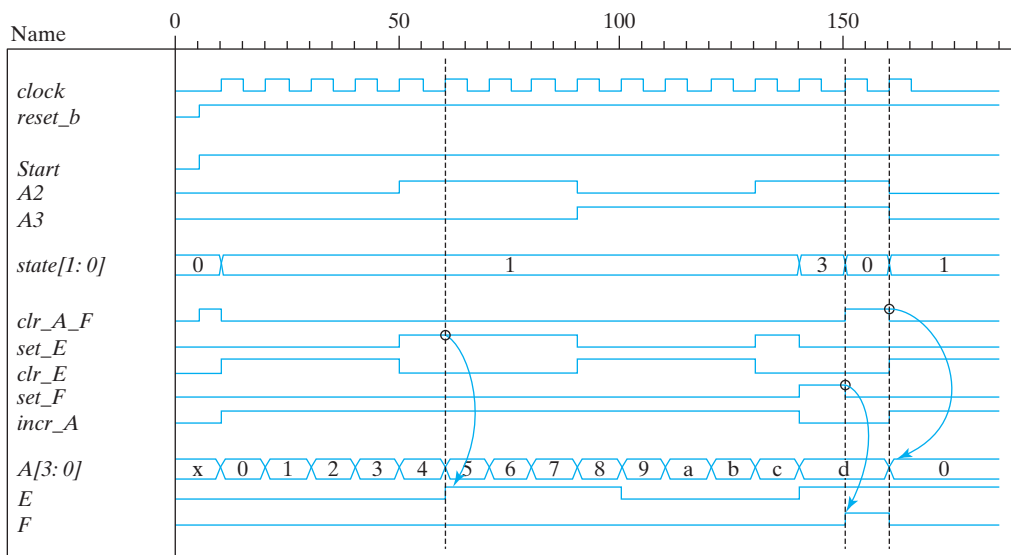
```
// Testbench for design example
`timescale 1 ns / 1 ps
module t_Design_Example_RTL;
reg    Start, clock, reset_b;
wire  [3: 0] A;
wire   E, F;
// Instantiate design example
Design_Example_RTL M0 (A, E, F, Start, clock, reset_b);
// Describe stimulus waveforms
initial #500 $finish; // Stopwatch
initial
begin
    reset_b = 0;
    Start = 0;
    clock = 0;
    #5 reset_b = 1; Start = 1;
repeat (32)
begin
    #5 clock = ~ clock; // Clock generator
end
end
initial
// $monitor displays A, E, and F every 10 ns
    $monitor ("A = %b E = %b F = %b time = %0d", A, E, F, $time);
endmodule
```

Simulation log:

```
A = xxxx  E = x  F = x  time = 0
A = 0000  E = x  F = 0  time = 10
A = 0001  E = 0  F = 0  time = 20
A = 0010  E = 0  F = 0  time = 30
A = 0011  E = 0  F = 0  time = 40
A = 0100  E = 0  F = 0  time = 50
A = 0101  E = 1  F = 0  time = 60
A = 0110  E = 1  F = 0  time = 70
A = 0111  E = 1  F = 0  time = 80
A = 1000  E = 1  F = 0  time = 90
A = 1001  E = 0  F = 0  time = 100
```

|          |       |       |            |
|----------|-------|-------|------------|
| A = 1010 | E = 0 | F = 0 | time = 110 |
| A = 1011 | E = 0 | F = 0 | time = 120 |
| A = 1100 | E = 0 | F = 0 | time = 130 |
| A = 1101 | E = 1 | F = 0 | time = 140 |
| A = 1101 | E = 1 | F = 1 | time = 150 |
| A = 0000 | E = 1 | F = 0 | time = 160 |

Waveforms produced by a simulation of *Design\_Example\_RTL* with the testbench are shown in Fig. 8.13. Numerical values are shown in hexadecimal format. The results are annotated to call attention to the relationship between a control signal and the operation that it causes to execute. For example, the controller asserts *set\_E* for one clock cycle *before* the clock edge at which *E* is set to 1. Likewise, *set\_F* asserts during the clock cycle before the edge at which *F* is set to 1. Also, *clr\_A\_F* is formed in the cycle before *A* and *F* are cleared. A more thorough verification of *Design\_Example\_RTL* would confirm that the machine recovers from a reset on the fly (i.e., a reset that is asserted randomly after the machine is operating). Note that the signals in the output of the simulation have been listed in groups showing (1) *clock* and *reset\_b*, (2) *Start* and the status inputs, (3) the state, (4) the control signals, and (5) the datapath registers. *It is strongly recommended that the state always be displayed*, because this information is essential for verifying that the machine is operating correctly and for debugging its description when it is not. For the chosen binary state code,  $S\_idle = 00_2 = 0_H$ ,  $S\_1 = 01_2 = 1_H$ , and  $S\_2 = 11_2 = 3_H$ .



**FIGURE 8.13**  
Simulation results for *Design\_Example\_RTL*

**VHDL** A VHDL testbench for `Design_Example_RTL_vhdl` generates stimulus signals and applies them to the UUT. The results of the VHDL simulation match those of the Verilog simulation shown in Fig. 8.13 and are not repeated here. We do, however, note the relationship between the formal names of the signals in the UUT and the actual names of the signals generated by the testbench and connected to them in the port map. For example, the testbench generates `t_clock`, which is connected to the signal `clock` at the interface to `Design_Example_RTL_vhdl`. Simulation results in VHDL can also be printed to a monitor or to a file.<sup>10</sup>

```

library ieee;
use ieee.std_logic_1164.all;

-- Testbench for Design_Example_RTL_vhdl in VHDL Example 8.2

entity t_Design_Example_RTL_vhdl is
    port ();
end t_Design_Example_RTL_vhdl;

architecture TestBench of t_Design_Example_RTL_vhdl is
    component Design_Example_RTL
        port (A: out std_logic_vector (3 downto 0); E, F: out std_logic); end component.
    signal t_A: std_logic_vector (3 downto 0);
    signal t_E, t_F: std_logic; ;
    signal t_Start, t_clock, t_reset_b: std_logic
    integer count: range 0 to 31: 0;    -- Counter, initialized to 0
    sig
begin
    M0: Design_Example_RTL_vhdl
    port map (A => t_A; E => t_E, F => t_F, Start=> t_Start, clock => t_clk, reset_b =>
        t_reset_b);

    process () begin;    -- Stimulus signals
    begin
        t_reset_b    <= '0';
        t_Start      <= '0'
        t_reset_b    <= '1' after 5 ns;
        t_Start      <= '1';
    end process;

    process (clock) begin
        while count <= 31 loop

```

<sup>10</sup> For a brief tutorial, see [www.gmvhdl.com/textio.yhtm](http://www.gmvhdl.com/textio.yhtm).

```

        clock <= not clock after 5 ns;
    end loop;
end process;
end TestBench;

```

---

## Structural Description

The RTL description of a design consists of procedural statements that determine the functional behavior of the digital circuit. This type of description can be compiled by HDL synthesis tools, from which it is possible to obtain the equivalent gate-level circuit of the design. It is also possible to describe the design by its structure rather than its function. A structural description of a design consists of instantiations of components that define the circuit elements and their interconnections. In this regard, a structural description is equivalent to a schematic diagram or a block diagram of the circuit. Contemporary design practice relies heavily on RTL descriptions, but we will present a structural description here to contrast the two approaches.

For convenience, the circuit is again decomposed into two parts: the controller and the datapath. The block diagram of Fig. 8.10 shows the high-level partition between these units, and Fig. 8.12 provides additional underlying structural details of the controller. The structure of the datapath is evident in Fig. 8.10 and consists of the flip-flops and the four-bit counter with synchronous clear. The top level of the description replaces *Design\_Example\_RTL*, *Controller\_RTL*, and *Datapath\_RTL* by *Design\_Example\_STR*, *Controller\_STR*, and *Datapath\_STR*, respectively. The descriptions of *Controller\_STR* and *Datapath\_STR* will be structural.

**Verilog** HDL Example 8.4 presents the structural description of the design example. It consists of a nested hierarchy of modules and gates describing (1) the top-level module, *Design\_Example\_STR*, (2) the modules describing the controller and the datapath, (3) the modules describing the flip-flops and counters, and (4) gates implementing the logic of the controller. For simplicity, the counter and flip-flops are described by RTL models.

The top-level module (see Fig. 8.10) encapsulates the entire design by (1) instantiating the controller and the datapath modules, (2) declaring the primary (external) input signals, (3) declaring the output signals, (4) declaring the control signals generated by the controller and connected to the datapath unit, and (5) declaring the status signals generated by the datapath unit and connected to the controller. The port list is identical to the list used in the RTL description. The outputs are declared as **wire** type here because they serve merely to connect the outputs of the datapath module to the outputs of the top-level module, with their logic value being determined within the datapath module.

The control module describes the circuit of Fig. 8.12. The outputs of the two flip-flops, *G1* and *G0*, and inputs, *D\_G1* and *D\_G0*, are declared as **wire** data type. The name of a variable is local to the module or procedural block in which it is declared. Nets may not be declared within a procedural block (e.g., **begin . . . end**). The rule to remember is that a variable must be a declared register type (e.g., **reg**) if and only if its value is assigned by a

procedural statement (i.e., a blocking or nonblocking assignment statement within a procedural block in cyclic or single-pass behavior or in the output of a sequential UDP). The instantiated gates specify the combinational part of the circuit. There are two flip-flop input equations and three output equations. The outputs of the flip-flops, *G1* and *G0*, and the input equations for *D\_G1* and *D\_G0* replace output *Q* and input *D* in the instantiated flip-flops. The *D* flip-flop is then described in the next module. The structure of the datapath unit has direct inputs to the *JK* flip-flops. Note the correspondence between the modules of the HDL description and the structures in Figs. 8.9, 8.10, and 8.12.

### HDL Example 8.4

---

#### Verilog

```
//Structural description of design example (see Figs. 8.9(a), 8.12)
module Design_Example_STR
  (output [3: 0] A,                                // V 2001 port syntax
   output E, F,
   input   Start, clock, reset_b
  );
  Controller_STR M0 (clr_A_F, set_E, clr_E, set_F, incr_A, Start, A[2], A[3], clock, reset_b );
  Datapath_STR M1 (A, E, F, clr_A_F, set_E, clr_E, set_F, incr_A, clock);
endmodule

// Control Unit
module Controller_STR
(output clr_A_F, set_E, clr_E, set_F, incr_A,
 input   Start, A2, A3, clock, reset_b
);
  wire G0, G1, D_G0, D_G1;
  parameter S_idle = 2'b00, S_1 = 2'b01, S_2 = 2'b11;
  wire w1, w2, w3;
  not (G0_b, G0);
  not (G1_b, G1);
  buf (incr_A, w2);
  and (set_F, G1, G0);
  not (A2_b, A2);
  or (D_G0, w1, w2);
  and (w1, Start, G0_b, G1_b);
  and (clr_A_F, G0_b, Start);
  and (w2, G0, G1_b);
  and (set_E, w2, A2);
  and (clr_E, w2, A2_b);
  and (D_G1, w3, w2);
  and (w3, A2, A3);
  D_flip_flop_AR M0 (G0, D_G0, clock, reset_b);
```

```

D_flip_flop_AR M1 (G1, D_G1, clock, reset_b);
endmodule

// datapath unit
module Datapath_STR
(output [3: 0] A,
 output E, F,
 input clr_A_F, set_E, clr_E, set_F, incr_A, clock
);
JK_flip_flop_2 M0 (E, E_b, set_E, clr_E, clock);
JK_flip_flop_2 M1 (F, F_b, set_F, clr_A_F, clock);
Counter_4 M2 (A, incr_A, clr_A_F, clock);
endmodule

// Counter with synchronous clear
module Counter_4 (output reg [3: 0] A, input incr, clear, clock);
always @ (posedge clock)
    if (clear) A <= 0; else if (incr) A <= A + 1;
endmodule
module D_flip_flop_AR (Q, D, CLK, RST); // Asynchronous reset
output Q;
input D, CLK, RST;
reg Q;
always @ (posedge CLK, negedge RST)
    if (RST == 0) Q <= 1'b0;
    else Q <= D;
endmodule

// Description of JK flip-flop
module JK_flip_flop_2 (Q, Q_not, J, K, CLK);
output Q, Q_not;
input J, K, CLK;
reg Q;
assign Q_not = ~Q;
always @ (posedge CLK)
    case ({J, K})
        2'b00: Q <= Q; // No change
        2'b01: Q <= 1'b0; // Clear
        2'b10: Q <= 1'b1; // Set
        2'b11: Q <= !Q; // Toggle
    endcase
endmodule

// Tests Bench
module t_Design_Example_STR;
reg Start, clock, reset_b;
wire [3: 0] A;

```

```

wire E, F;
// Instantiate design example
Design_Example_STR M0 (A, E, F, Start, clock, reset_b);
// Describe stimulus waveforms
initial #500 $finish; // Stopwatch
initial
begin
  reset_b = 0;
  Start = 0;
  clock = 0;
  #5 reset_b = 1; Start = 1;
  repeat (32)
    begin
      #5 clock = ~clock; // Clock generator
    end
  end
initial
  $monitor ("A = %b E = %b F = %b time = %0d", A, E, F, $time);
endmodule

```

---

The structural description was tested with the testbench that verified the RTL description to produce the results shown in Fig. 8.13. The only change necessary is the replacement of the instantiation of the example from *Design\_Example\_RTL* by *Design\_Example\_STR*. The simulation results for *Design\_Example\_STR* matched those for *Design\_Example\_RTL*. However, a comparison of the two descriptions indicates that the RTL style is easier to write and will lead to results faster if synthesis tools are available to automatically synthesize the registers, the combinational logic, and their interconnections.

**VHDL** -- Structural description of design example (see Figs. 8.9 (a) and 8.12)

```

library ieee;
use ieee.std_logic_1164.all;

entity Design_Example_STR_vhdl is
  port (A: out std_logic_vector (3 downto 0);
        E, F: out std_logic;
        Start, clock, reset_b: in std_logic);
end Design_Example_STR_vhdl;

architecture Structural_vhdl of Design_Example_STR_vhdl is
  component Controller_STR_vhdl
    port (clr_A_F, set_E, clr_E, set_F: in Std_Logic;
          incr_A, Start, A2, A3 : in Std_Logic; clock, reset_b : in Std_Logic);
  end component;

```

```

component Datapath_STR_vhdl
  port (A, E, F: out Std_Logic; clr_A_F, set_E, clr_E, set_F, incr_A: in Std_Logic;
        clock: in Std_Logic); end component.

begin
  M0: Controller_STR_vhdl
  port map(clr_A_F => clr_A_F, set_E => set_E, clr_E => clr_E, set_F => set_F, incr_A =>
    incr_A: out std_logic; Start, A2, A3, clock, reset_b: in std_logic);
  M1 Datapath_STR_vhdl
  port map (A => A, E => E, F => F, clr_A_F => clr_a_F, set_E => set_E, clr_E => clr_E,
    set_F => set_F, incr_A => incr_A, clock => clock);
end Structural_vhdl;
-- Control unit
entity Controller_STR_vhdl is
  port (clr_A_F, set_E, clr_E, set_F, incr_A: out std_logic;
        Start, A2, A3, clock, reset_b: in std_logic);
end Controller_STR_vhdl;
architecture Structural of Controller_STR_vhdl is
  component not_gate port (sig_out : out Std_Logic; sig_in : in Std_Logic);
  end component;
  component buf_gate port (sig_out : out Std_Logic; sig_in : in Std_Logic);
  end component;
  component or2_gate port (sig_out : out Std_Logic; sig1, sig2 : in Std_Logic);
  end component
  component and2_gate port (sig_out : out Std_Logic; sig1, sig2 : in Std_Logic);
  end component;
  component D_flop port (Q : out Std_Logic; D in Std_Logic; clk, reset : in Std_Logic);
  end component;

  begin
end component;

  C1: not_gate_vhdl port map (sig_out => G0_b, sig_in => G0);
  C2: not_gate_vhdl port map (sig_out => G1_b, sig_in => G1);
  C3: buf_gate_vhdl port map (sig_out => incr_A, sig_in => w2);
  C4: and2_gate_vhdl port map (sig_out => set_F, sig_in => G1, G0);
  C5: not_gate_vhdl port map (sig_out => A2_b, sig_in => A2);
  C6: or2_gate_vhdl port map (sig_out => D_G0, sig1 => w1, sig2 => w2);
  C7: and3_gate_vhdl port map (sig_out => w1, sig1 => Start, sig2 => Go_b, G1_b);
  C8: and2_gate_vhdl port map (sig_out => clr_A_F, sig1 => G0, sig2 => Start);
  C9: and2_gate_vhdl port map (sig_out => w2, sig1 => G0, sig2 => G1_b);
  C10: and2_gate_vhdl port map (sig_out => Set_E, sig1 => w2, sig2 => A2);
  C11: and2_gate_vhdl port map (sig_out => clr_E, sig1 => w2, sig2 => A2_b);
  C12: and2_gate_vhdl port map (sig_out => D_G1, sig1 => w3, sig2 => w2);
  C13: and2_gate_vhdl port map (sig_out => w3, sig1 => A2, sig2 => A3);
  C14: D_flip_flop_AR_vhdl port map (Q => G0, D => D_G0, clk => clock, reset => reset_b);
  C15: D_flip_flop_AR_vhdl port map (Q => G1, D => D_G1, clk => clock, reset => reset_b);
end Controller_STR_vhdl;

```



```

-- Datapath unit
entity Datapath_STR_vhdl is
    port (A: out std_logic_vector (3 downto 0); E, F: out std_logic;
          clr_A_F, set_E, clr_E, set_F, incr_A, clock: in std_logic);
end Datapath_STR_vhdl;

architecture Structural of Datapath_STR_vhdl is
    component JK_flip_flop_2 port (E, E_b: out std_logic; set_E, clr_E, clk: in std_logic);
end component;
    component JK_flip_flop_2 port (F, F_b: out std_logic; set_F, clr_A_F, clk: in std_logic);
end component;
    component counter_4 port (A: out std_logic_vector (3 downto 0); incr_A, clr_A_F, clock:
in std_logic);
end component;
begin
    M0: JK_flip_flop_2 port map (E => E, E_b => E_b, set_E => set_E, clr_E => clr_E,
    clk <= clock);
    M1: JK_flip_flop_2 port map (F => F, F_b => F_b, incr_A => incr_A, clr_A => clr_A,
    clk <= clock);
    M2 counter_4 port map (A => A; incr_A => incr_A, clr_A_F => clr_A_F, clock => clock);
end Structural;

entity Counter_4_vhdl is
    port (A: out std_logic_vector (3 downto 0); incr, clear, clock: in std_logic);
end Counter_4vhdl;

architecture Behavioral of Counter_4_vhdl is
begin
    process (clock)
    begin
        if clear = '1' then A <= 0;
            elsif clock'event and clock = '1' and incr = '1' then A <= A + '1';
        end if;
    end process;

entity D_flip_flop is
    port (Q: out std_logic; D, CLK, RST: in std_logic);
end D_flip_flop;

architecture Behavioral of D_flip_flop is
begin
    process (CLK, RST)
    begin
        if RST = '0' then Q <= '0' elsif CLK'event and CLK = '1' then Q <= D; end if;
    end process;
end Behavioral;

entity JK_flip_flop_2_vhdl is

```

```

    port (Q, Q_not: out std_logic; J, K, CLK: in std_logic);
end JK_flip_flop_2_vhdl;

architecture Behavioral of JK_flip_flop_2_vhdl is
begin
    Q_not <= not Q;
process (CLK)
begin
    if CLK'event and CLK = '1'
        case J & K is
            when '00' => Q <= Q; // No change
            when '01' => Q <= '0'; // Clear
            when '10' => Q <= '1'; // Set
            when '11' => Q <= not Q; // Toggle
        end case;
    end process;
end Behavioral;

entity t_Design_Example_STR_vhdl is
    port ();
end t_Design_Example_STR_vhdl;

architecture Behavioral of t_Design_Example_STR_vhdl is
    signal t_A: std_logic_vector (3 downto 0);
    signal t_E, t_F: std_logic;
    signal t_clock, t_reset_b;
    integer count range 0 to 31: 0; -- Counter, initialized to 0
-- Identify component
    component Design_Example_STR_vhdl
        port (A, E, F : out Std_Logic; Start, clock, reset_b in Std_Logic);
    begin
-- Instantiate component
M0: Design_Example_STR_vhdl
    port map (A => t_A, E => t_E, F => t_F,
              Start=> t_Start, clock => t_clock, reset_b => t_reset_b);
-- Describe stimulus waveforms

process () begin
    t_reset_b <= '0';
    t_Start <= '0';
    t_reset_b <= '1' after 5 ns;
    t_Start <= '1' after 5 ns;
end process;

process () begin
    t_clock <= 0;

```

```

while count <= 31 loop
    t_clock <= not t_clock after 5 ns;
end loop;
end process;
end Behavioral;

```

## 8.7 SEQUENTIAL BINARY MULTIPLIER

This section introduces a second design example. It presents a hardware algorithm for binary multiplication, proposes the register configuration for its implementation, and then shows how to use an ASMD chart to design its datapath and its controller.

The machine multiplies two unsigned binary numbers. The multiplier developed earlier, in Section 4.7, resulted in a combinational circuit with many adders and AND gates, and requires a large area of silicon as an integrated circuit. In contrast, this section develops a more efficient hardware algorithm resulting in a sequential multiplier that uses only one adder and a shift register. The savings in hardware and silicon area come about from a trade-off in the space (hardware)—time domain. A parallel adder uses more hardware, but forms its result in one cycle of the clock; a sequential adder uses less hardware, but takes multiple clock cycles to form its result.

The multiplication of two binary numbers is done with paper and pencil by successive (i.e., sequential) additions and shifting. The process is best illustrated with a numerical example. Let us multiply the unsigned two binary numbers 10111 and 10011:

$$\begin{array}{r}
 \begin{array}{r} 23 \\ 19 \\ \hline \end{array}
 \begin{array}{r}
 10111 \text{ multiplicand} \\
 10011 \text{ multiplier} \\
 \hline
 10111 \\
 10111 \\
 00000 \\
 00000 \\
 \hline
 10111 \\
 \hline
 437 \quad 110110101 \text{ product}
 \end{array}
 \end{array}$$

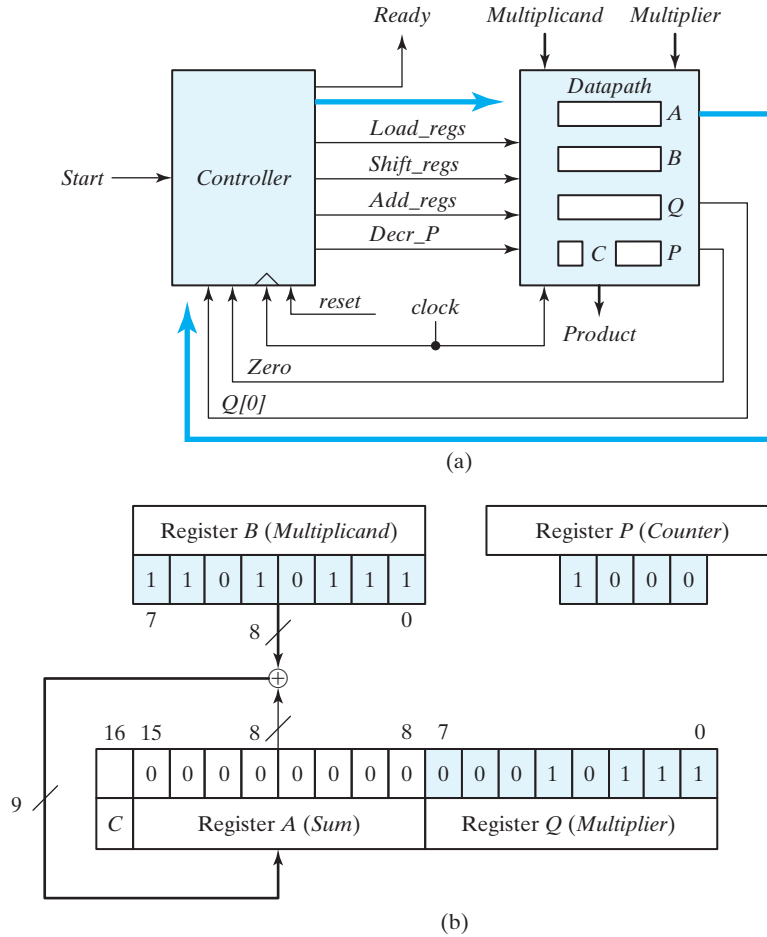
The process consists of successively adding and shifting copies of the multiplicand, depending on the bits of the multiplier. Successive bits of the multiplier are examined, least significant bit first. If the multiplier bit is 1, the multiplicand is copied down; otherwise, 0's are copied down. The numbers copied in successive lines are aligned with the associated bit of the multiplier by shifting the copy one position to the left from the previous number. Finally, the numbers are added and their sum forms the product. The product obtained from the multiplication of two unsigned binary numbers of  $n$  bits each can have up to  $2n$  bits. It is apparent that the operations of addition and shifting are executed by the algorithm.

When the multiplication process is implemented with digital hardware, it is convenient to change the process slightly. First, we note that, in the context of synthesizing a

sequential machine, the add-and-shift algorithm for binary multiplication can be executed in a single clock cycle or over multiple clock cycles. An algorithm that forms the product in the time span of a single clock cycle will synthesize the circuit of a parallel multiplier like the one discussed in Section 4.7. On the other hand, an RTL model of the algorithm adds shifted copies of the multiplicand to an accumulated partial product. The values of the multiplier, multiplicand, and partial product are stored in registers, and the operations of shifting and adding their contents are executed under the control of a state machine. Among the many possibilities for distributing the effort of multiplication over multiple clock cycles, we will consider that in which only one partial product is formed and accumulated in a single cycle of the clock. (One alternative would be to use additional hardware to form and accumulate two partial products in a clock cycle, but this would require more logic gates and either faster circuits or a slower clock.) First, instead of providing digital circuits to store and add simultaneously as many binary numbers as there are 1's in the multiplier, it is less expensive to provide only the hardware needed to sum two binary numbers and accumulate the partial products in a register. Second, instead of shifting the multiplicand to the left, the partial product being formed is shifted to the right. This leaves the partial product and the multiplicand in the required relative positions. Third, when the corresponding bit of the multiplier is 0, there is no need to add all 0's to the partial product, since doing so will not alter its resulting value.

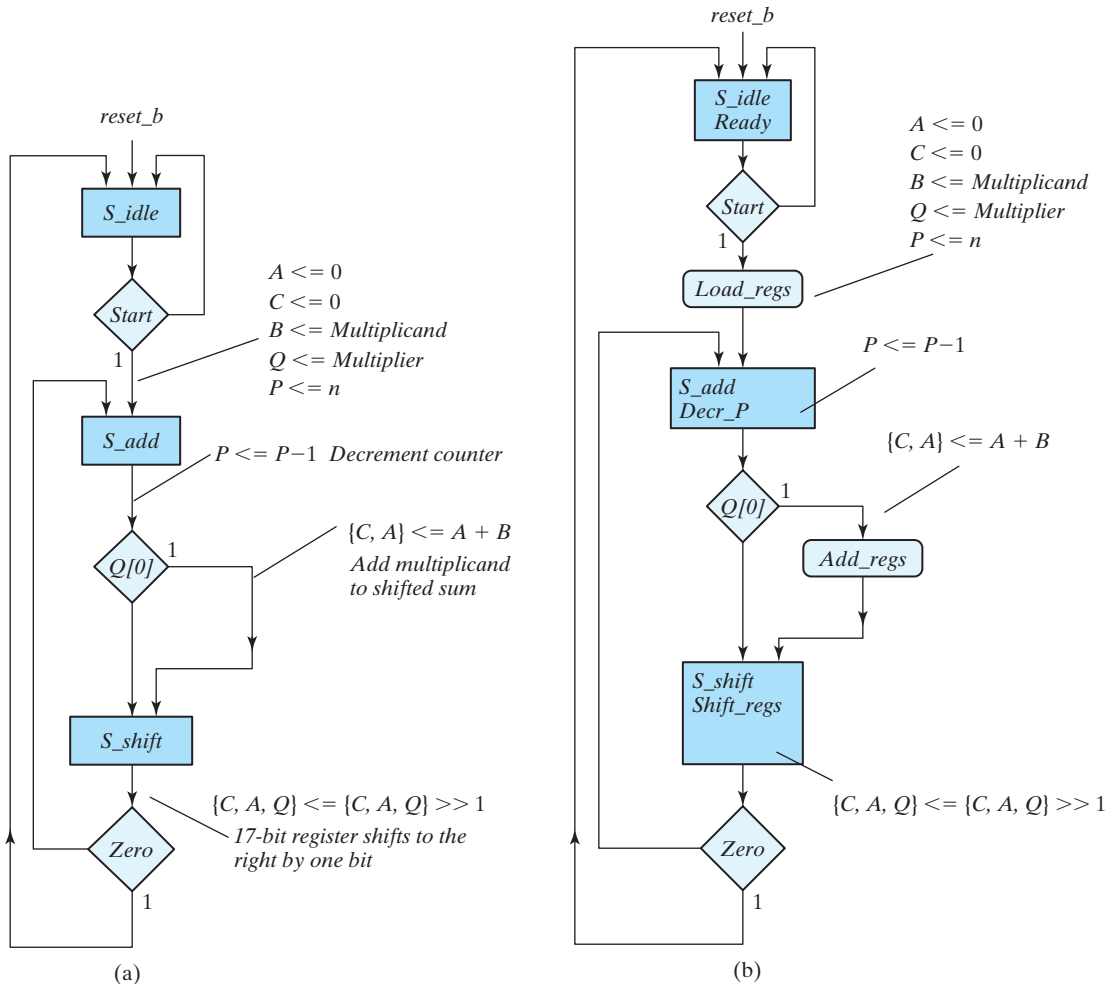
## Register Configuration

A block diagram for the sequential binary multiplier is shown in Fig. 8.14(a), and the register configuration of the datapath is shown in Fig. 8.14(b). The multiplicand is stored in register  $B$ , the multiplier is stored in register  $Q$ , and the partial product is formed in register  $A$  and stored in  $A$  and  $Q$ . A parallel adder adds the contents of register  $B$  to register  $A$ . Flip-flop  $C$  stores the carry after the addition. The counter  $P$  is initially set to hold a binary number equal to the number of bits in the multiplier. This counter is decremented after the formation of each partial product. When the content of the counter reaches zero, the product is formed in the double register  $A$  and  $Q$ , and the process stops. The control logic stays in an initial state until *Start* becomes 1. The system then performs the multiplication. The sum of  $A$  and  $B$  forms the  $n$  most significant bits of the partial product, which is transferred to  $A$ . The output carry from the addition, whether 0 or 1, is transferred to  $C$ . Both the partial product in  $A$  and the multiplier in  $Q$  are shifted to the right. The least significant bit of  $A$  is shifted into the most significant position of  $Q$ , the carry from  $C$  is shifted into the most significant position of  $A$ , and 0 is shifted into  $C$ . After the shift-right operation, one bit of the partial product is transferred into  $Q$  while the multiplier bits in  $Q$  are shifted one position to the right. In this manner, the least significant bit of register  $Q$ , designated by  $Q[0]$ , holds the bit of the multiplier that must be inspected next. The control logic determines whether to add or not on the basis of this input bit. The control logic also receives a signal, *Zero*, from a circuit that checks counter  $P$  for zero.  $Q[0]$  and *Zero* are status inputs for the control unit. The input signal *Start* is an external control input. The outputs of the control logic launch the required operations in the registers of the datapath unit.



**FIGURE 8.14**  
(a) Block diagram and (b) datapath of a binary multiplier

The interface between the controller and the datapath consists of the *status* signals, which are inputs to the controller, and the *output* signals of the controller. The control signals govern the synchronous register operations of the datapath. Signal *Load\_regs* loads the internal registers of the datapath, *Shift\_regs* causes the shift register to shift, *Add\_regs* forms the sum of the multiplicand and register A, and *Decr\_P* decrements the counter. The controller also forms *Ready*, an output that signals to the host environment that the machine is ready to multiply. The contents of the register holding the product vary during execution, so it is useful to have a signal indicating that its contents are valid. Note, again, that *the state of the control is not an interface signal between the control unit and the datapath*. Only the signals needed to control the datapath are included in the interface. Putting the state in the interface would require a decoder in the datapath, and would require a wider and more active bus than the control signals alone.



**FIGURE 8.15**  
ASMD chart for binary multiplier

## ASMD Chart

The ASMD chart for the binary multiplier is shown in Fig. 8.15. The intermediate form in Fig. 8.15(a) annotates the ASM chart of the controller with the register operations, and the completed chart in Fig. 8.15(b) identifies the Moore and Mealy outputs of the controller. As long as the circuit is in *S\_idle*, the initial state, and *Start* = 0, no action occurs and the system remains in state *S\_idle* with *Ready* asserted. The multiplication process is launched when an external agent asserts *Start* = 1. Then, (1) control goes to state *S\_add*, (2) register *A* and carry flip-flop *C* are cleared to 0,

(3) registers  $B$  and  $Q$  are loaded with the multiplicand and the multiplier, respectively, and (4) the sequence counter  $P$  is set to a binary number  $n$ , equal to the number of bits in the multiplier. In state  $S\_add$ , the multiplier bit held in  $Q[0]$  is checked, and, if it is equal to 1, the multiplicand in  $B$  is added to the partial product in  $A$ . The carry from the addition is transferred to  $C$ . The partial product in  $A$  and  $C$  is left unchanged if  $Q[0] = 0$ . The counter  $P$  is decremented by 1 regardless of the value of  $Q[0]$ , so  $Decr\_P$  is formed in state  $S\_add$  as a Moore output of the controller. In both cases, the next state is  $S\_shift$ . Registers  $C$ ,  $A$ , and  $Q$  are combined into one composite register,  $CAQ$ , formed by the concatenation  $\{C, A, Q\}$ , and its contents are shifted once to the right to obtain a new partial product. This shift operation is symbolized in the flowchart with a logical right-shift operator,  $\gg$ . It is equivalent to the following statement in register transfer notation:

Shift right  $CAQ$ ,  $C \leftarrow 0$

In terms of individual register symbols, the shift operation can be described by the following register operations:

$$\begin{aligned} A &\leftarrow \text{shr } A, A_{n-1} \leftarrow C \\ Q &\leftarrow \text{shr } Q, Q_{n-1} \leftarrow A_0 \\ C &\leftarrow 0 \end{aligned}$$

Both registers  $A$  and  $Q$  are shifted right. The leftmost bit of  $A$ , designated by  $A_{n-1}$ , receives the carry from  $C$ . The leftmost bit of  $Q$ ,  $Q_{n-1}$ , receives the bit from the rightmost position of  $A$  in  $A_0$ , and  $C$  is reset to 0. In essence, this is a long shift of the composite register  $CAQ$  with 0 inserted into the serial input, which is at  $C$ .

The value in counter  $P$  is checked after the formation of each partial product. If the contents of  $P$  are different from zero, status bit *Zero* is set equal to 0 and the process is repeated to form a new partial product. The process stops when the counter reaches 0 and the controller's status input *Zero* is equal to 1. Note that the partial product formed in  $A$  is shifted into  $Q$  one bit at a time and eventually replaces the multiplier. The final product is available in  $A$  and  $Q$ , with  $A$  holding the most significant bits and  $Q$  the least significant bits of the product.

Table 8.5 repeats the previous numerical data to clarify the multiplication process. The procedure follows the steps outlined in the ASMD chart. The data shown in the table can be compared with simulation results.

The type of registers needed for the data processor subsystem can be derived from the register operations listed in the ASMD chart. Register  $A$  is a shift register with parallel load to accept the sum from the adder and must have a synchronous clear capability to reset the register to 0. Register  $Q$  is a shift register. The counter  $P$  is a binary down counter with a facility to parallel load a binary constant. The  $C$  flip-flop must be designed to accept the input carry and have a synchronous clear. Registers  $B$  and  $Q$  need a parallel load capability in order to receive the multiplicand and multiplier prior to the start of the multiplication process.

**Table 8.5**  
*Numerical Example For Binary Multiplier*

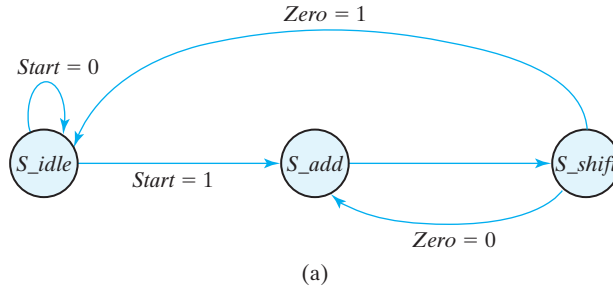
| <b>Multiplicand <math>B = 10111_2 = 17_H = 23_{10}</math> Multiplier <math>Q = 10011_2 = 13_H = 19_{10}</math></b> |          |              |          |          |
|--------------------------------------------------------------------------------------------------------------------|----------|--------------|----------|----------|
|                                                                                                                    | <b>C</b> | <b>A</b>     | <b>Q</b> | <b>P</b> |
| Multiplicand in $Q$                                                                                                | 0        | 00000        | 10011    | 101      |
| $Q_0 = 1$ ; add $B$                                                                                                |          | <u>10111</u> |          |          |
| First partial product                                                                                              | 0        | 10111        |          | 100      |
| Shift right $CAQ$                                                                                                  | 0        | 01011        | 11001    |          |
| $Q_0 = 1$ ; add $B$                                                                                                |          | <u>10111</u> |          |          |
| Second partial product                                                                                             | 1        | 00010        |          | 011      |
| Shift right $CAQ$                                                                                                  | 0        | 10001        | 01100    |          |
| $Q_0 = 0$ ; shift right $CAQ$                                                                                      | 0        | 01000        | 10110    | 010      |
| $Q_0 = 0$ ; shift right $CAQ$                                                                                      | 0        | 00100        | 01011    | 001      |
| $Q_0 = 1$ ; add $B$                                                                                                |          | <u>10111</u> |          |          |
| Fifth partial product                                                                                              | 0        | 11011        |          |          |
| Shift right $CAQ$                                                                                                  | 0        | 01101        | 10101    | 000      |
| Final product in $AQ = 0110110101_2 = 1b5_H$                                                                       |          |              |          |          |

## 8.8 CONTROL LOGIC

The design of a digital system can be divided into two parts: the design of the register transfers in the datapath unit and the design of the logic of the control unit. The control logic is a finite state machine; its Mealy- and Moore-type outputs control the operations of the datapath by determining when operations execute. The inputs to the control unit are the primary (external) inputs and the internal status signals fed back from the datapath to the controller. The design of the system can be synthesized from an RTL description derived from the ASMD chart. Alternatively, a manual design must derive the logic governing the inputs to the flip-flops holding the state of the controller. The information needed to form the state diagram of the controller is already contained in the ASMD chart, since the rectangular blocks that designate state boxes are the states of the sequential circuit. The diamond-shaped blocks that designate decision boxes determine the logical conditions for the next state transition in the state diagram and assertions of the conditional outputs.

As an example, the control state diagram for the binary multiplier developed in the previous section is shown in Fig. 8.16(a). The information for the diagram is taken directly from the ASMD chart of Fig. 8.15. The three states  $S\_idle$  through  $S\_shift$  are taken from the rectangular state boxes. The inputs  $Start$  and  $Zero$  are taken from the diamond-shaped decision boxes. The register transfer operations for each of the three states are listed in Fig. 8.16(b) and are taken from the corresponding state and conditional boxes in the ASMD chart. Establishing the state transitions is the initial focus, so the outputs of the controller are not shown.





| State Transition |                | Register Operations                                                              |
|------------------|----------------|----------------------------------------------------------------------------------|
| From             | To             |                                                                                  |
| <i>S_idle</i>    |                | <i>Initial state</i>                                                             |
| <i>S_idle</i>    | <i>S_add</i>   | $A \leq 0, C \leq 0, P \leq dp\_width$<br>$A \leq Multiplier, Q \leq Multiplier$ |
| <i>S_add</i>     | <i>S_shift</i> | $P \leq P - 1$<br>if ( $Q[0]$ ) then ( $A \leq A + B, C \leq C_{out}$ )          |
| <i>S_shift</i>   |                | shift right {CAQ}, $C \leq 0$                                                    |

(b)

**FIGURE 8.16**  
Control specifications for binary multiplier

We must execute two steps when implementing the control logic: (1) establish the required sequence of states, and (2) provide signals to control the register operations. The sequence of states is specified in the ASMD chart or the state diagram. The signals for controlling the operations in the registers are specified in the register transfer statements annotated on the ASMD chart or listed in tabular format. For the multiplier, these signals are *Load\_regs* (for parallel loading the registers in the datapath unit), *Decr\_P* (for decrementing the counter), *Add\_regs* (for adding the multiplicand and the partial product), and *Shift\_regs* (for shifting register CAQ). The block diagram of the control unit is shown in Fig. 8.14(a). The inputs to the controller are *Start*,  $Q[0]$ , and *Zero*, and the outputs are *Ready*, *Load\_regs*, *Decr\_P*, *Add\_regs*, and *Shift\_regs*, as specified in the ASMD chart. We note that  $Q[0]$  affects only the output of the controller, not its state transitions. The machine transitions from *S\_add* to *S\_shift* unconditionally.

An important step in the design is the assignment of coded binary values to the states. The simplest assignment is the sequence of binary numbers, as shown in Table 8.6. Another assignment is the so-named Gray code, according to which only one bit changes when going from one number to the next. A state assignment often used in control design is the *one-hot* assignment. This assignment uses as many bits and flip-flops as there are states in the circuit. At any given time, only one bit is equal to 1 (the one that is hot)

**Table 8.6**  
*State Assignment for Control*

| State          | Binary | Gray Code | One-Hot |
|----------------|--------|-----------|---------|
| <i>S_idle</i>  | 00     | 00        | 001     |
| <i>S_add</i>   | 01     | 01        | 010     |
| <i>S_shift</i> | 10     | 11        | 100     |

while all others are kept at 0 (all cold). This type of assignment uses a flip-flop for each state. Indeed, one-hot encoding uses more flip-flops than other types of coding, but it usually leads to simpler decoding logic for the next state and the output of the machine. Because the decoding logic does not become more complex as states are added to the machine, the speed at which the machine can operate is not limited by the time required to decode the state.

Since the controller is a sequential circuit, it can be designed manually by the sequential logic procedure outlined in Chapter 5. In most cases this method is difficult to carry out manually because a typical control circuit may have a large number of states and inputs. As a consequence, it is necessary to use specialized methods for control logic design that may be considered as variations of the classical sequential logic method. We will now present two such design procedures. One uses a sequence register and decoder, and the other uses one flip-flop per state. The method will be presented for a small circuit, but it applies to larger circuits as well. Of course, the need for these methods is eliminated if one has software that automatically synthesizes the circuit from an HDL description.

## Sequence Register and Decoder

The sequence-register-and-decoder (manual) method, as the name implies, uses a register for the control states and a decoder to provide an output corresponding to each of the states. (The decoder is not needed if a one-hot code is used.) A register with  $n$  flip-flops can have up to  $2^n$  states, and an  $n$ -to- $2^n$ -line decoder has up to  $2^n$  outputs. An  $n$ -bit sequence register is essentially a circuit with  $n$  flip-flops, together with the associated gates that effect their state transitions.

The ASMD chart and the state diagram for the controller of the binary multiplier have three states and two inputs. (There is no need to consider  $Q[0]$ .) To implement the design with a sequence register and decoder, we need two flip-flops for the register and a two-to-four-line decoder. The outputs of the decoder will form the Moore-type outputs of the controller directly. The Mealy-type outputs will be formed from the Moore outputs and the inputs.

The state table for the finite state machine of the controller is shown in Table 8.7. It is derived directly from the ASMD chart of Fig. 8.15(b) or the state diagram of Fig. 8.16(a). We designate the two flip-flops as  $G_1$  and  $G_0$  and assign the binary codes 00, 01, and 10 to states *S\_idle*, *S\_add*, and *S\_shift*, respectively. Note that the input columns have don't-care entries whenever the input variable is not used to determine the next state or

**Table 8.7**  
*State Table for Control Circuit*

|                | Present State |       | Inputs       |             |             | Next State |       | Outputs      |                  |               |                 |                   |
|----------------|---------------|-------|--------------|-------------|-------------|------------|-------|--------------|------------------|---------------|-----------------|-------------------|
|                | $G_1$         | $G_0$ | <i>Start</i> | <i>Q[0]</i> | <i>Zero</i> | $G_1$      | $G_0$ | <i>Ready</i> | <i>Load_regs</i> | <i>Decr_P</i> | <i>Add_regs</i> | <i>Shift_regs</i> |
| <i>S_idle</i>  | 0             | 0     | 0            | X           | X           | 0          | 0     | 1            | 0                | 0             | 0               | 0                 |
| <i>S_idle</i>  | 0             | 0     | 1            | X           | X           | 0          | 1     | 1            | 1                | 0             | 0               | 0                 |
| <i>S_add</i>   | 0             | 1     | X            | 0           | X           | 1          | 0     | 0            | 0                | 1             | 0               | 0                 |
| <i>S_add</i>   | 0             | 1     | X            | 1           | X           | 1          | 0     | 0            | 0                | 1             | 1               | 0                 |
| <i>S_shift</i> | 1             | 0     | X            | X           | 0           | 0          | 1     | 0            | 0                | 0             | 0               | 1                 |
| <i>S_shift</i> | 1             | 0     | X            | X           | 1           | 0          | 0     | 0            | 0                | 0             | 0               | 1                 |

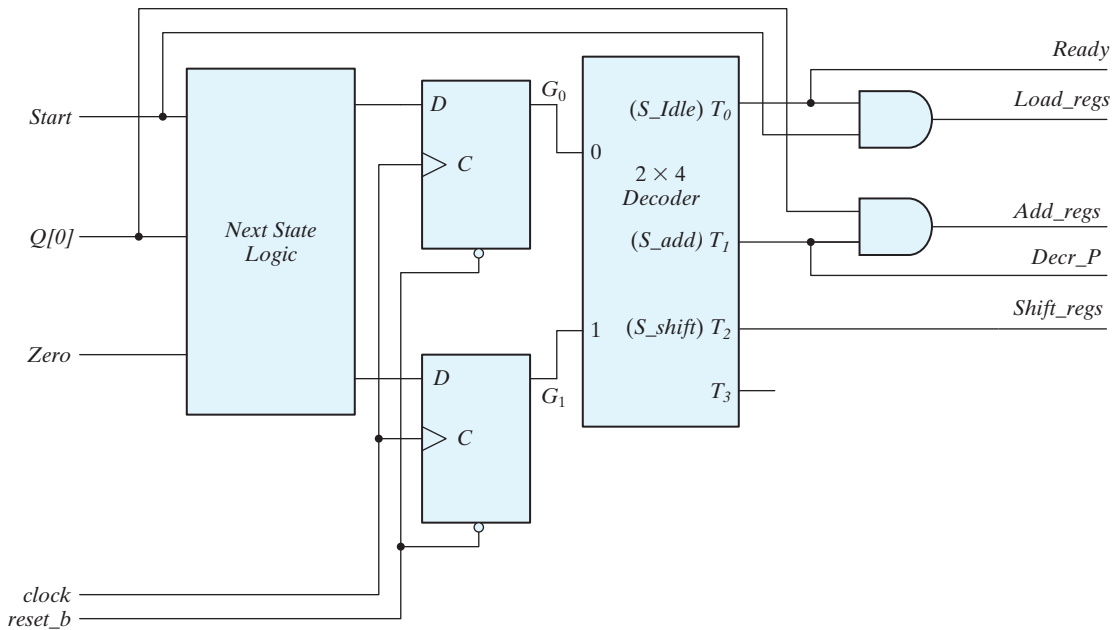
the outputs. The outputs of the control circuit are designated by the names given in the ASMD chart. The particular Moore-type output variable that is equal to 1 at any given time is determined from the equivalent binary value of the present state. Those output variables are shaded in Table 8.7. Thus, when the present state is  $G_1G_0 = 00$ , output *Ready* must be equal to 1, while the other outputs remain at 0. Since the Moore-type outputs are a function of only the present state, they can be generated with a decoder circuit having the two inputs  $G_1$  and  $G_0$  and using three of the decoder outputs  $T_0$  through  $T_2$ , as shown in Fig. 8.17(a), which does not include the wiring for the state feedback.

The state machine of the controller can be designed from the state table by means of the classical procedure presented in Chapter 5. This example has a small number of states and inputs, so we could use maps to simplify the Boolean functions. In most control logic applications, the number of states and inputs is much larger. In general, the application of the classical method requires an excessive amount of work to obtain the simplified input equations for the flip-flops and is prone to error. The design can be simplified if we take into consideration the fact that the decoder outputs are available for use in the design. Instead of using flip-flop outputs as the present-state conditions, *we use the outputs of the decoder to indicate the present-state condition of the sequential circuit.* Moreover, instead of using maps to simplify the flip-flop equations, we can obtain them directly by inspection of the state table. For example, from the next-state conditions in the state table, we find that the next state of  $G_1$  is equal to 1 when the present state is *S\_add* and is equal to 0 when the present state is *S\_idle* or *S\_shift*. These conditions can be specified by the equation

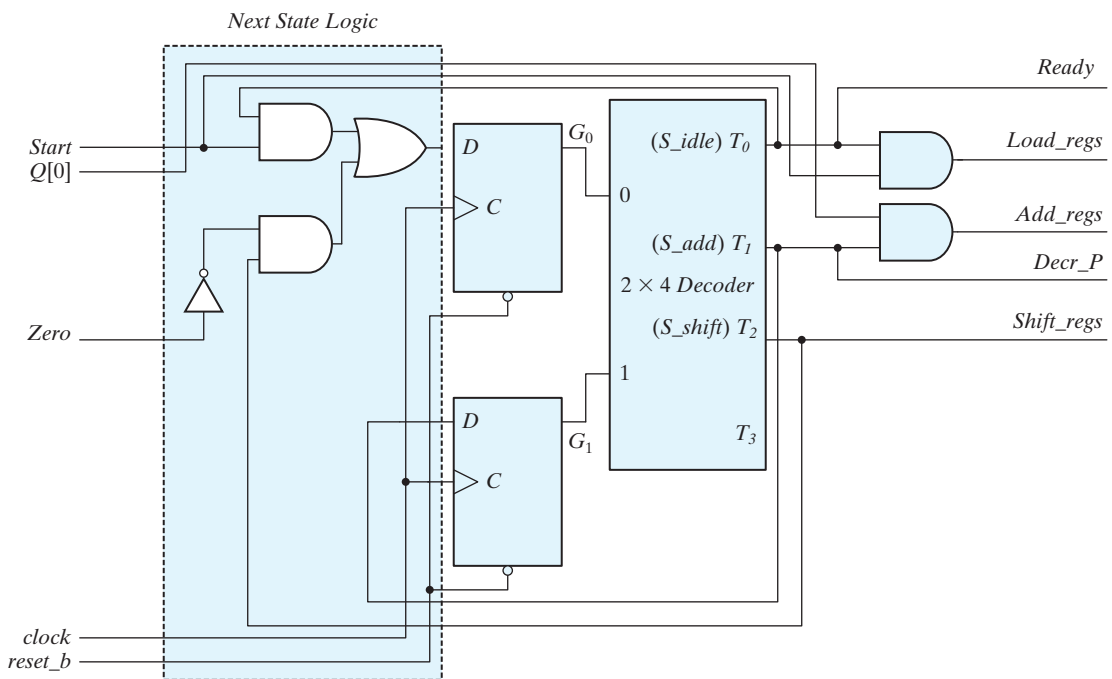
$$D_{G1} = T_1$$

where  $D_{G1}$  is the *D* input of flip-flop  $G_1$ . Similarly, the *D* input of  $G_0$  is

$$D_{G0} = T_0 \textit{Start} + T_2 \textit{Zero}'$$



(a)



(b)

**FIGURE 8.17**

Logic diagram derived from Table 8.7 for controlling a binary multiplier using a sequence register and decoder

When deriving input equations by inspection from the state table, we cannot be sure that the Boolean functions have been simplified in the best possible way. (Synthesis tools take care of this detail automatically.) In general, it is advisable to analyze the circuit to ensure that the equations derived do indeed produce the required state transitions.

The logic diagram of the control circuit is drawn in Fig. 8.17(b). It consists of a register with two flip-flops  $G_1$  and  $G_0$  and a  $2 \times 4$  decoder. The outputs of the decoder are used to generate the inputs to the next-state logic as well as the control outputs. The outputs of the controller should be connected to the datapath to activate the required register operations.

### One-Hot Design (One Flip-Flop per State)

Another method of control logic design is the one-hot assignment, which results in a sequential circuit with one flip-flop per state. Only one of the flip-flops contains a 1 at any time; all others are reset to 0. The single 1 propagates from one flip-flop to another under the control of decision logic. In such a configuration, each flip-flop represents a state that is present only when the control bit is transferred to it.

This method uses the maximum number of flip-flops for the sequential circuit. For example, a sequential circuit with 12 states requires a minimum of four flip-flops. By contrast, with the method of one flip-flop per state, the circuit requires 12 flip-flops, one for each state. At first glance, it may seem that this method would increase system cost, since more flip-flops are used. But the method offers some advantages that may not be apparent. One advantage is the simplicity with which the logic can be designed by inspection of the ASMD chart or the state diagram. *No state or excitation tables are needed if D-type flip-flops are employed.* The one-hot method offers a savings in design effort, an increase in operational simplicity, and a possible decrease in the total number of gates, since a decoder is not needed.

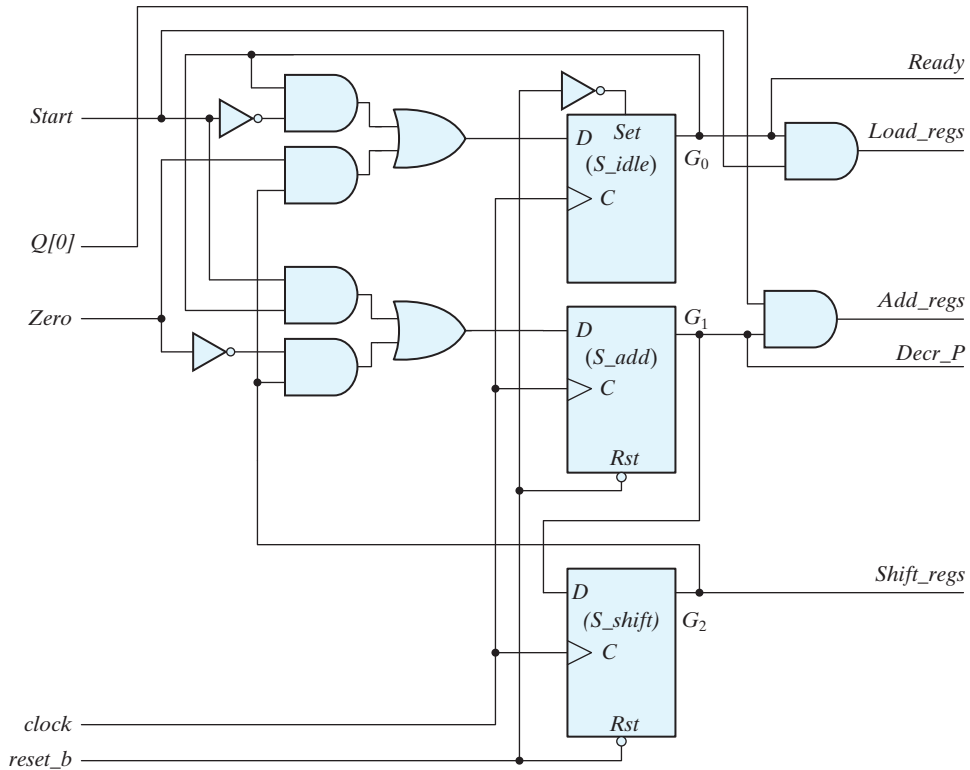
The design procedure for a one-hot state assignment will be demonstrated by obtaining the control circuit specified by the state diagram of Fig. 8.16(a). Since there are three states in the state diagram, we choose three  $D$  flip-flops and label their outputs  $G_0$ ,  $G_1$ , and  $G_2$ , corresponding to  $S\_idle$ ,  $S\_add$ , and  $S\_shift$ , respectively. The input equations for setting each flip-flop to 1 are determined from the present state and the input conditions along the corresponding directed lines going into the state. For example,  $D_{G_0}$ , the input to flip-flop  $G_0$ , is set to 1 if the machine is in state  $G_0$  and  $Start$  is not asserted, or if the machine is in state  $G_2$  and  $Zero$  is asserted. These conditions are specified by the input equation:

$$D_{G_0} = G_0 Start' + G_2 Zero$$

In fact, the condition for setting a flip-flop to 1 is obtained directly from the state diagram, that is, from the condition specified in the directed lines going into the corresponding flip-flop state ANDed with the previous flip-flop state. If there is more than one directed line going into a state, all conditions must be ORed. Using this procedure for the other three flip-flops, we obtain the remaining input equations:

$$D_{G_1} = G_0 Start + G_2 Zero'$$

$$D_{G_2} = G_1$$



**FIGURE 8.18**  
Logic diagram for one-hot state controller

The logic diagram of the one-hot controller is shown in Fig. 8.18. The circuit consists of three *D* flip-flops labeled  $G_0$  through  $G_2$ , together with the associated gates specified by the input equations. Initially, flip-flop  $G_0$  must be set to 1 and all other flip-flops must be reset to 0, so that the flip-flop representing the initial state is enabled. This can be done by using an asynchronous preset on flip-flop  $G_0$  and an asynchronous clear for the other flip-flops. Once started, the controller with one flip-flop per state will propagate from one state to the other in the proper manner. Only one flip-flop will be set to 1 with each clock edge; all others are reset to 0, because their *D* inputs are equal to 0.

## 8.9 HDL DESCRIPTION OF BINARY MULTIPLIER

HDL Example 8.5 presents RTL description of the binary multiplier designed in Section 8.7. For simplicity, the entire description is “flattened” and encapsulated in one design unit. Comments will identify the controller and the datapath.

**HDL Example 8.5 (Sequential Binary Multiplier)****Verilog**

The first part of the description declares all of the inputs and outputs as specified in the block diagram of Fig. 8.14(a). The machine is parameterized for a five-bit datapath to enable a comparison between its simulation data and the result of the multiplication with the numerical example listed in Table 8.5. The same model can be used for a datapath having a different size merely by changing the value of the parameters. The second part of the description declares all registers in the controller and the datapath, as well as the one-hot encoding of the states. The third part specifies implicit combinational logic (continuous assignment statements) for the concatenated register *CAQ*, the *Zero* status signal, and the *Ready* output signal. The continuous assignments for *Zero* and *Ready* are accomplished by assigning a Boolean expression to their **wire** declarations. The next section describes the control unit, using a single edge-sensitive cyclic behavior to describe the state transitions, and a level-sensitive cyclic behavior to describe the combinational logic for the next state and the outputs. Again, note that default assignments are made to *next\_state*, *Load\_regs*, *Decr\_P*, *Add\_regs*, and *Shift\_regs*. The subsequent logic of the case statement assigns their value by exception. The state transitions and the output logic are written directly from the ASMD chart of Fig. 8.15(b).

The datapath unit describes the register operations within a separate edge-sensitive cyclic behavior.<sup>11</sup> (For clarity, separate cyclic behaviors are used; *we do not mix the description of the datapath with the description of the controller.*) Each control input is decoded and is used to specify the associated operations. The addition and subtraction operations will be implemented in hardware by combinational logic. Signal *Load\_regs* causes the counter and the other registers to be loaded with their initial values, etc. Because the controller and datapath have been partitioned into separate units, the control signals completely specify the behavior of the datapath; explicit information about the state of the controller is not needed and is not made available to the datapath unit.

The next-state logic of the controller includes a default case item to direct a synthesis tool to map any of the unused codes to *S\_idle*. The default case item and the default assignments preceding the **case** statement ensure that the machine will recover if it somehow enters an unused state. They also prevent unintentional synthesis of latches. (Remember, a synthesis tool will synthesize latches when what was intended to be combinational logic in fact fails to completely specify the input–output function of the logic.)

```
module Sequential_Binary_Multiplier (Product, Ready, Multiplicand, Multiplier,
Start, clock, reset_b);
// Default configuration: five-bit datapath
parameter dp_width = 5; // Set to width of datapath
```

<sup>11</sup>The width of the datapath here is *dp\_width*.

```

output          [2*dp_width-1: 0] Product;
output          Ready;
input           [dp_width-1: 0] Multiplicand, Multiplier;
input           Start, clock, reset_b;
parameter      BC_size = 3; // Size of bit counter
parameter      S_idle = 3'b001, // one-hot code
                S_add = 3'b010,
                S_shift = 3'b100;

reg            [2: 0] state, next_state;
reg            [dp_width-1: 0] A, B, Q; // Sized for datapath
reg            C;
reg            [BC_size-1: 0] P;
reg            Load_regs, Decr_P, Add_regs, Shift_regs;
// Miscellaneous combinational logic
assign         Product = {A, Q};
wire          Zero = (P == 0);           // counter is zero
                // Zero = ~|P;           // alternative
wire          Ready = (state == S_idle); // controller status
// control unit
always @ (posedge clock, negedge reset_b)
  if (!reset_b) state <= S_idle; else state <= next_state;
always @ (state, Start, Q[0], Zero) begin
  next_state = S_idle;
  Load_regs = 0;
  Decr_P = 0;
  Add_regs = 0;
  Shift_regs = 0;
case (state)
  S_idle: if (Start) begin next_state = S_add; Load_regs = 1; end
  S_add:  begin next_state = S_shift; Decr_P = 1; if (Q[0]) Add_regs = 1; end
  S_shift: begin Shift_regs = 1; if (Zero) next_state = S_idle;
else next_state = S_add; end
  default: next_state = S_idle;
endcase
end
// datapath unit
always @ (posedge clock) begin
  if (Load_regs) begin
    P <= dp_width;
    A <= 0;
    C <= 0;
    B <= Multiplicand;
    Q <= Multiplier;
  end

```



```

if (Add_regs) {C, A} <= A + B;
if (Shift_regs) {C, A, Q} <= {C, A, Q} >> 1;
if (Decr_P) P <= P - 1;
end
endmodule

```

---

## VHDL

VHDL models can be extended by using generic constants to describe register sizes, bus sizes, and other features that vary from application to application. Carefully-named constants can make code more readable, flexible, and reusable, thereby promoting efficient revisions to a design. Here, we wish to design a multiplier to support a word size of five bits. By using constants, the same model can be reused with differently-sized words. A *generic constant* is declared within an entity and is available for use in any architecture that is paired with the entity. Ordinary constants are local to the architecture or process in which they are declared.

The syntax template for a generic constant is given below:

```

entity entity_name is
    generic (constant_names : constant_type;
            constant_names : constant_type;
            . . .
            constant_names : constant_type);
    port ( . . . );
end entity_name;

```

Note that the template declares generic constants before the port of the entity is declared, so the constants can be used to provide flexibility in the definition of the port.

The sequential binary multiplier is described by processes.

```

entity Sequential_Binary_Multiplier_vhdl is
    generic (dp_width : integer := 5); -- Width of the datapath
    port (Product: out Std_Logic_Vector (2*dp_width - 1 downto 0); Ready:
    out: bit; Multiplicand, Multiplier: in std_logic_vector (dp_width-1 downto 0);
    Start, clock, reset_b: in Std_Logic);
end Sequential_Binary_Multiplier_vhdl;

architecture Behavioral of Sequential_Binary_Multiplier_vhdl is
    constant BC_size integer := 3
    constant S_idle: Std_Logic_Vector (2 downto 0) := '001';
    constant S_add: Std_Logic_Vector (2 downto 0) := '010';
    constant S_shift: Std_Logic_Vector (2 downto 0) := '100';

```

```

signal    state, next_state: Std_Logic_Vector (2 downto 0)
signal    A, B, Q: Std_Logic_Vector (dp_width-1 downto 0);
signal    C: Std_Logic;
signal    P: Std_Logic_Vector (BC_size-1 downto 0);
begin      -- Concurrent signal assignments
    Product <= A & Q;          -- Concatenation
    Zero <= P = '0';          -- Counter is 0
    Ready <= state = S_idle;  -- Controller status

-- Control Unit
process (clock, reset_b) -- State transitions, asynch, active-low reset
begin
    if reset_b = '0' then state <= S_idle;
    elsif clock'event and clock = '1' then state <= next_state; end if;
end process;

process (state, Start, Q(0), Zero) -- Next-state logic
begin
    -- Defaults for assign by exception
    next_state <= S_idle;
    Load_regs <= '0';
    Decr_P <= '0';
    Add_regs <= '0';
    Shift_regs <= '0';

    -- State decoding logic
    case state is
        when S_idle => if Start = '1'
            then begin next_state <= S_add; Load_regs
                <= '1'; end if;
        when S_add => begin next_state = S_shift; Decr_P <= '1';
            if Q(0) = '1' then Add_regs <= '1'; end if;
            end;
        when S_shift => begin Shift_regs <= '1';
            if Zero = '1' then next_state <= S_idle;
            else next_state <= S_add; end if;
        when others => next_state <= S_idle;
    end case;
end process;

process (clock)
    -- Datapath Unit (Register ops)
begin
    if clock'event and clock = '1' then begin

```

```

if Load_regs = '1' then begin
    P <= dp_width;
    A <= 0;
    C <= 0;
    B <= Multiplicand;
    Q <= Multiplier;
end if;
if Add_regs = '1' then C & A <= A + B; end if;
if Shift_regs = '1' then C & A & Q <= srl (C & A & Q) 1; end if;
if Decr_P = '1' then P <= P - 1; end if;
end if;
end process;
end Behavioral

```

## Testing the Multiplier

*Irrational exuberance can doom a design.* It is naive to conclude that an HDL description of a system is correct on the basis of the output it generates under the application of a few input signals. A more strategic approach to testing and verification exploits the partition of the design into its datapath and control unit. This partition supports separate verification of the controller and the datapath. A separate testbench can be developed to verify that the datapath executes each operation and generates status signals correctly. After the datapath unit is verified, the next step is to verify that each control signal is formed correctly by the control unit. A separate testbench can verify that the control unit exhibits the complete functionality specified by the ASMD chart (i.e., it makes the correct state transitions and asserts its outputs in response to the external inputs and the status signals).

A verified control unit and a verified datapath unit together do not guarantee that the system will operate correctly. The final step in the design process is to *integrate the verified models* within a parent design unit and verify the functionality of the overall machine with the control unit controlling the datapath unit. The interface between the controller and the datapath must be examined in order to verify that the ports are connected correctly. For example, a mismatch in the listed order of signals may not be detected by the compiler. After the datapath unit and the control unit have been verified, a third testbench should verify the specified functionality of the complete system. In practice, this requires writing a comprehensive test plan identifying that functionality. For example, the test plan would identify the need to verify that the sequential multiplier asserts the signal *Ready* in state *S\_idle*. The exercise to write a test plan is not academic: *The quality and scope of the test plan determine the worth of the verification effort.* The test plan guides the development of the testbench and increases the likelihood that the final design will match its specification.

**HDL Example 8.6 (Testbench)****Verilog**

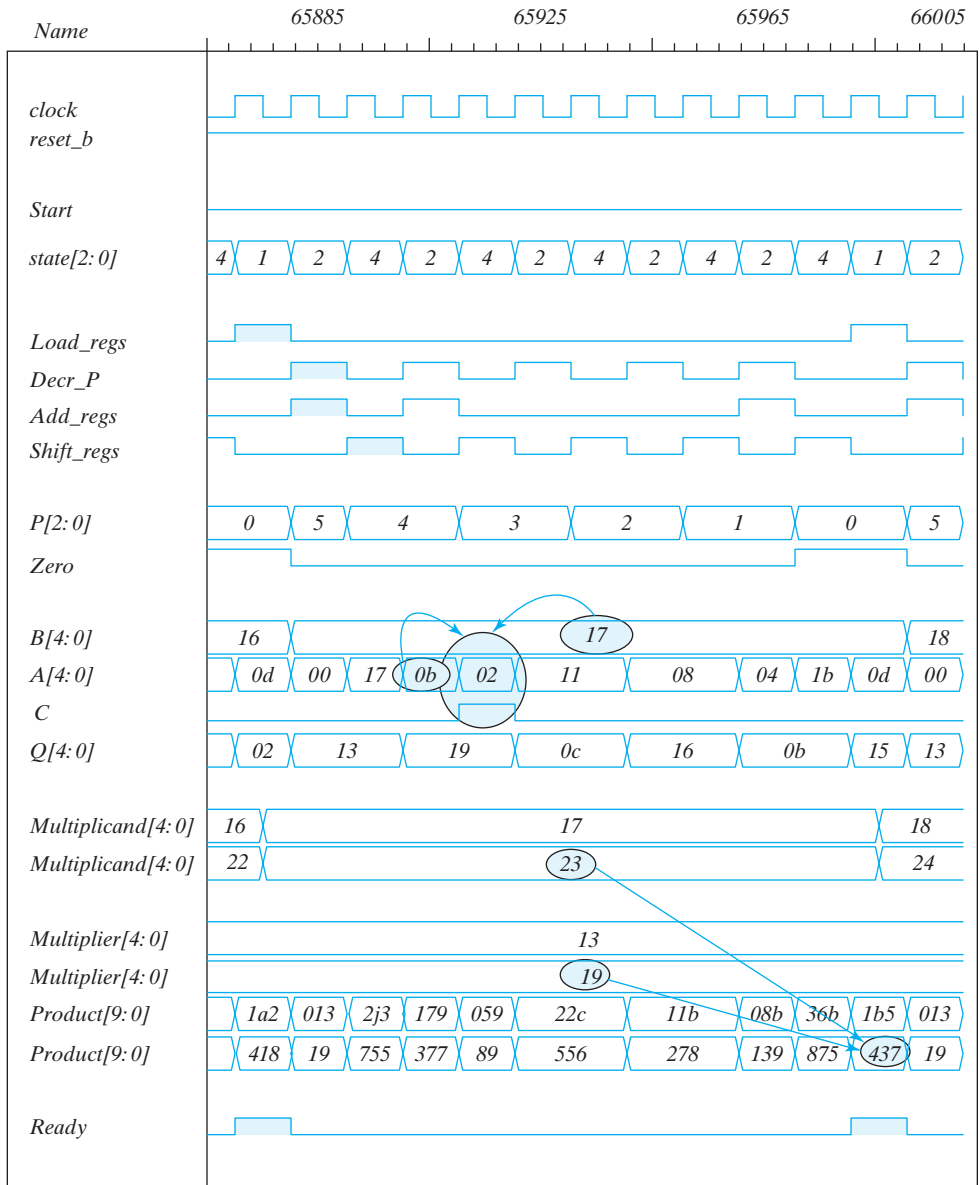
Testing and verifying an HDL model usually requires access to more information than the inputs and outputs of the machine. Knowledge of the state of the control unit, the control signals, the status signals, and the internal registers of the datapath might all be necessary for debugging. Fortunately, Verilog provides a mechanism to hierarchically de-reference identifiers so that any variable at any level of the design hierarchy can be viewed by the testbench. Procedural statements can display the information required to support efforts to debug the machine. This mechanism references the variable by its hierarchical path name. For example, the register *P* within the datapath unit is not an output port of the multiplier, but it can be referenced as *M0.P*. The hierarchical path name consists of the sequence of module identifiers or block names, separated by periods and specifying the location of the variable in the design hierarchy. We also note that simulators commonly have a graphical user interface that displays all levels of the hierarchy of a design.

The first testbench below uses the system task **\$strobe** to display the result of the computations. This task is similar to the **\$display** and **\$monitor** tasks explained in Section 4.12. The **\$strobe** system task provides a synchronization mechanism to ensure that data are displayed only after all assignments in a given time step are executed. This is very useful in synchronous sequential circuits, where the time step begins at a clock edge and multiple assignments may occur at the same time step of simulation. When the system is synchronized to the positive edge of the clock, using **\$strobe** after the **always @ (posedge clock)** statement ensures that the display shows values of the signal after the clock pulse.

The testbench module *t\_Sequential\_Binary\_Multiplier* instantiates the module *Sequential\_Binary\_Multiplier* of HDL Example 8.5. Both modules must be included as source files when simulating the multiplier with a Verilog HDL simulator. The result of this simulation displays a simulation log with numbers identical to the ones in Table 8.5. The code includes a second testbench to exhaustively multiply five-bit values of the multiplicand and the multiplier. Waveforms for a sample of simulation results are shown in Fig. 8.19. The numerical values of *Multiplicand*, *Multiplier*, and *Product* are displayed in decimal and hexadecimal formats. Insight can be gained by studying the displayed waveforms of the control state, the control signals, the status signals, and the register operations. Enhancements to the multiplier and its testbench are considered in the problems at the end of this chapter. In this example,  $19_{10} \times 23_{10} = 437_{10}$ , and  $17_H + 0b_H = 02_H$  with  $C = 1$ . Note the need for the carry bit.

// Testbench for the binary multiplier

```
module t_Sequential_Binary_Multiplier;
  parameter dp_width = 5; // Set to width of datapath
  wire      [2*dp_width-1:0] Product; // Output from multiplier
  wire      Ready;
  reg       [dp_width-1:0] Multiplicand, Multiplier; // Inputs to multiplier
```



**FIGURE 8.19**  
Simulation waveforms for one-hot state controller

```

reg      Start, clock, reset_b;
// Instantiate multiplier
Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start,
clock, reset_b);
// Generate stimulus waveforms
initial #200 $finish;
initial
  begin
    Start = 0;
    reset_b = 0;
    #2 Start = 1; reset_b = 1;
    Multiplicand = 5'b10111;    Multiplier = 5'b10011;
    #10 Start = 0;
  end
initial
begin
  clock = 0;
  repeat (26) #5 clock = 'clock;
end
// Display results and compare with Table 8.5
always @ (posedge clock)
$strobe ("C=%b A=%b Q=%b P=%b time=%0d",M0.C,M0.A,M0.Q,M0.P, $time);
endmodule

```

Simulation log:

```

C=0 A=00000  Q=10011  P=101 time=5
C=0 A=10111  Q=10011  P=100 time=15
C=0 A=01011  Q=11001  P=100 time=25
C=1 A=00010  Q=11001  P=011 time=35
C=0 A=10001  Q=01100  P=011 time=45
C=0 A=10001  Q=01100  P=010 time=55
C=0 A=01000  Q=10110  P=010 time=65
C=0 A=01000  Q=10110  P=001 time=75
C=0 A=00100  Q=01011  P=001 time=85
C=0 A=11011  Q=01011  P=000 time=95
C=0 A=01101  Q=10101  P=000 time=105
C=0 A=01101  Q=10101  P=000 time=115
C=0 A=01101  Q=10101  P=000 time=125

```

/\* Testbench for exhaustive simulation

```

module t_Sequential_Binary_Multiplier;
parameter dp_width = 5; // Width of datapath
wire      [2 * dp_width - 1: 0] Product;
wire      Ready;
reg       [dp_width - 1: 0] Multiplicand, Multiplier;

```

```

reg      Start, clock, reset_b;
Sequential_Binary_Multiplier M0 (Product, Ready, Multiplicand, Multiplier, Start,
clock, reset_b);
initial #1030000 $finish;
initial begin clock = 0; #5 forever #5 clock = ~clock; end
initial fork  reset_b = 1;
            #2 reset_b = 0;
            #3 reset_b = 1;
join
initial begin #5 Start = 1; end
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (32) #10 begin Multiplier = Multiplier + 1;
        repeat (32) @ (posedge M0.Ready) #5 Multiplicand = Multiplicand + 1;
    end
end
endmodule
*/

```

---

## VHDL

```

--
entity t_Sequential_Binary_Multiplier_vhdl is
    generic dp_width integer := 5;
    port ();
end t_Sequential_Binary_Multiplier_vhdl;

architecture Behavioral of t_Sequential_Binary_Multiplier_vhdl is
    signal t_Product: Std_Logic_Vector (2*dp_width-1 downto 0);
    signal t_Ready: Std_Logic;
    signal Multiplicand, Multiplier: Std_Logic_Vector (dp_width-1 downto 0);
    signal Start, clock, reset_b: Std_Logic;
    integer count range 0 to 25: 0;      -- Clock cycle counter, initialized to 0
    component Sequential_Binary_Multiplier_vhdl
        port (Product: out Std_Logic_Vector (2*dp_width-1 downto 0); Ready: Std_Logic;
            Multiplicand, Multiplier: in Std_Logic_Vector (dp_width-1 downto 0);
            Start, clock, reset_b: in Std_Logic);
    begin

    -- Instantiate UUT

    M0: Sequential_Binary_Multiplier_vhdl port map (Product => t_Product, Ready
=> t_Ready, Multiplicand => t_multiplicand, Multiplier => t_multiplier, Start =>
t_Start, clock => t_clock, reset_b => t_reset_b);

```

```

-- Generate stimulus waveforms
process begin
    t_Start <= '0';
    t_reset_b <= '0';
    t_Start <= '1' after 2 ns;
    t_reset_b <= '1' after 2 ns;
    t_multiplicand <= '10111';
    t_multiplier <= '10011';
    t_start <= '0' after 10 ns;
end process

process begin
    t_reset_b <= '1';
    t_reset_b <= '0' after 2 ns;
    t_reset_b <= '1' after 3 ns;
end process

process () begin                                -- 26 clock cycles
    t_clock <= 0;
    while count <= 25 loop
        t_clock <= not t_clock after 5 ns;
    end loop;

end process;

```

The alternative architecture below can be used for exhaustive simulation of the data words for the multiplier and multiplicand.

```

architecture Behavioral of t_Sequential_Binary_Multiplier_vhdl is
    signal t_Product: Std_Logic_Vector (2*dp_width-1 downto 0);
    signal t_Ready: Std_Logic;
    signal Multiplicand, Multiplier: Std_Logic_Vector (dp_width-1 downto 0);
    signal Start, clock, reset_b: Std_Logic;
    integer count range 0 to 25: 0;    -- Clock cycle counter, initialized to 0
    component Sequential_Binary_Multiplier_vhdl
        port (Product: out Std_Logic_Vector (2*dp_width-1 downto 0); Ready:
            Std_Logic;
            Multiplicand, Multiplier: in Std_Logic_Vector (dp_width-1 downto 0);
            Start, clock, reset_b: in Std_Logic);
    begin

    -- Instantiate UUT

    M0: Sequential_Binary_Multiplier_vhdl port map (Product => t_Product, Ready
        => t_Ready, Multiplicand => t_multiplicand, Multiplier => t_multiplier,
        Start => t_Start, clock => t_clock, reset_b => t_reset_b);

    -- Generate stimulus waveforms
    process begin

```



```

t_Start <= '0';
t_reset_b <= '0';
t_Start <= '1' after 5 ns;
t_reset_b <= '1' after 2 ns;
t_start <= '0' after 10 ns;
end process

process begin
t_reset_b <= '1';
t_reset_b <= '0' after 2 ns;
t_reset_b <= '1' after 3 ns;
end process

process begin
t_multiplicand <= '0' after 5 ns;
t_multiplier <= '0';

while outer_count <= 31 loop
begin multiplier <= multiplier + 1;
while inner_count <= 31 loop wait until Ready'event and Ready = '1';
Multiplicand <= Multiplicand + 1;
end loop;
end loop;
end process;

```

---

## Behavioral Description of a Parallel Multiplier

Structural modeling implicitly specifies the functionality of a digital machine by prescribing an interconnection of gate-level hardware units. In this form of modeling, a synthesis tool performs Boolean optimization and translates the HDL description of a circuit into a netlist of gates in a particular technology, for example, CMOS standard cells or an FPGA. Hardware design at this level often requires cleverness and accrued experience. It is the most tedious and detailed form of modeling. In contrast, behavioral RTL modeling specifies functionality abstractly, in terms of HDL operators and other language constructs. The RTL model does not specify a gate-level implementation of the registers or the logic to control the operations that manipulate their contents—those tasks are accomplished by a synthesis tool. RTL modeling implicitly schedules operations by explicitly assigning them to clock cycles. The most abstract form of behavioral modeling describes only an algorithm, without any reference to a physical implementation, a set of resources, or a schedule for their use. Thus, algorithmic modeling allows a designer to explore trade-offs in the space (hardware) and time domains, trading processing speed for hardware complexity. That work is beyond our purpose here.

HDL Example 8.7 presents an RTL model and an algorithmic model of a binary multiplier. Both use a level-sensitive cyclic behavior. The RTL model expresses the functionality of a multiplier in a single statement. A synthesis tool will associate with the multiplication operator a gate-level circuit equivalent to that shown in Section 4.7 In

simulation, when either the multiplier or the multiplicand changes, the product will be updated. The time required to form the product will depend on the propagation delays of the gates available in the technology used by the synthesis tool. The second model is an algorithmic description of the multiplier. A synthesis tool will unroll the loop of the algorithm and infer the need for a gate-level circuit equivalent to that shown in Section 4.7.

Be aware that a synthesis tool may not be able to synthesize a given algorithmic description, even though the associated HDL model will simulate and produce correct results. One difficulty is that the sequence of operations implied by an algorithm might not be physically realizable in a single clock cycle. It then becomes necessary to distribute the operations over multiple clock cycles. A tool for synthesizing RTL logic will not be able to automatically accomplish the required distribution of effort, but a tool that is designed to synthesize algorithms should be successful. In effect, a behavioral synthesis tool would have to allocate the registers and adders to implement multiplication. If only a single adder is to be shared by all of the operations that form a partial sum, the activity must be distributed over multiple clock cycles and in the correct sequence, ultimately leading to the sequential binary multiplier for which we have explicitly designed the controller for its datapath. Behavioral synthesis tools require a different and more sophisticated style of modeling and are not within the scope of this text.

### HDL Example 8.7

---

#### Verilog

```
// Behavioral (RTL) description of a parallel multiplier (n = 8)
module Mult (Product, Multiplicand, Multiplier);
    input [7: 0]    Multiplicand, Multiplier;
    output reg [15: 0] Product;
    always @ (Multiplicand, Multiplier)
        Product = Multiplicand * Multiplier;
endmodule

module Algorithmic_Binary_Multiplier #(parameter dp_width = 5) (
    output [2*dp_width-1: 0] Product, input [dp_width-1: 0] Multiplicand, Multiplier);
    reg [dp_width-1: 0]  A, B, Q; // Sized for datapath
    reg C;
    integer k;
    assign Product = {C, A, Q};
    always @ (Multiplier, Multiplicand) begin
        Q = Multiplier;
        B = Multiplicand;
        C = 0;
        A = 0;
        for (k = 0; k <= dp_width-1; k = k + 1) begin
            if (Q[0]) {C, A} = A + B;
            {C, A, Q} = {C, A, Q} >> 1;
        end
    end
```

```

end
endmodule

module t_Algorithmic_Binary_Multiplier; // Self-checking testbench
parameter dp_width = 5; // Width of datapath
wire [2* dp_width -1: 0] Product;
reg [dp_width -1: 0] Multiplicand, Multiplier;
integer Exp_Value;
reg Error;
Algorithmic_Binary_Multiplier M0 (Product, Multiplicand, Multiplier);
// Error detection
initial # 1030000 finish;
always @ (Product) begin
    Exp_Value = Multiplier * Multiplicand;
    // Exp_Value = Multiplier * Multiplicand +1; // Inject error to confirm detection
    Error = Exp_Value ^ Product;
end
// Generate multiplier and multiplicand exhaustively for 5 bit operands
initial begin
    #5 Multiplicand = 0;
    Multiplier = 0;
    repeat (32) #10 begin Multiplier = Multiplier + 1;
    repeat (32) #5 Multiplicand = Multiplicand + 1;
    end
end
endmodule

```

---

## VHDL

-- Behavioral (RTL) description of a parallel multiplier (word size = five bits)

**entity** MULT **is**

**generic** dp\_width: **integer** := 5;

**port** (Multiplicand, Multiplier: **in** Std\_Logic\_Array (dp\_width-1 **downto** 0);

        Product: **out** Std\_Logic\_Array (2\*dp\_width-1 **downto** 0));

**end** MULT;

**architecture** RTL **of** MULT **is**

**signal** A, B, Q: Std\_Logic\_Array (dp\_width-1 **downto** 0); -- Sized for datapath width

**integer** k;

**begin**

        Product <= C & A & Q;

**process** (Multiplier, Multiplicand) **begin**

        Q <= Multiplier;

        B <= Multiplicand;

        C <= '0';

```

A <= '0';

for k in 0 to 31 loop
  if Q(0) then
    C & A <= A + B;
    C & A & Q <= C & Q & A srl 1;
  end if;
end loop;
end process;
end RTL;

entity t_MULT is
  generic dp_width: integer := 5;
  port ();
end MULT;

architecture Testbench of MULT is
  component MULT port (Multiplicand, Multiplier, Product);
  signal t_Multiplicand, t_Multiplier: Std_Logic_Vector (dp_width - 1 downto 0);
  signal t_product: Std_Logic_Vector (2*dp_width - 1 downto 0);
  integer Exp_value;
  signal Error: bit;
  integer k_outer, k_inner;
begin
  -- Instantiate UUT
  M0: MULT port map (Multiplicand => t_Multiplicand, Multiplier, => t_Multiplier,
    Product => t_Product);

  -- Error Detection
  process (Product) begin
    Exp_Value <= Multiplier * Multiplicand; -- Replace with next line for test
    -- Exp_Value <= Multiplier * Multiplicand + 1; -- Inject error to confirm detection
    Error <= Exp_Value xor Product;
  end process;
end Testbench;

-- Generate multiplier and multiplicand exhaustively for 5-bit operands
process
  Multiplicand <= 0 after 5 ns;
  Multiplicand <= 0;
  for k_outer in 0 to 31 loop
    Multiplier <= Multiplier + 1;
    for k_inner in 0 to 31 loop
      Multiplicand <= Multiplicand + 1;
    end loop;
  end loop;
end process;

```

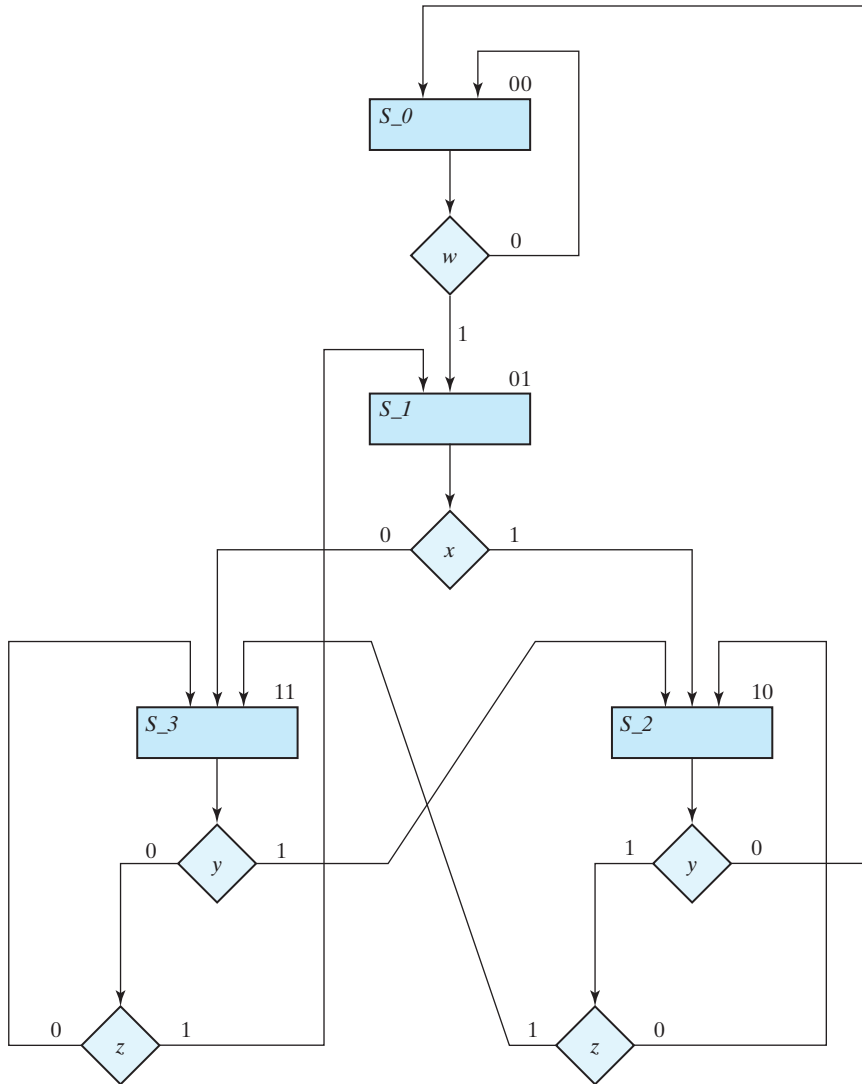
---

## 8.10 DESIGN WITH MULTIPLEXERS

The register-and-decoder scheme for the design of a controller has three parts: the flip-flops that hold the binary state value, the decoder that generates the control outputs, and the combinational logic (gates) that determine the next-state and output signals. In Section 4.11, it was shown that a combinational circuit can be implemented with multiplexers instead of individual gates. Replacing the gates with multiplexers results in a regular pattern of three levels of components. The first level consists of multiplexers that determine the next state of the register. The second level contains a register that holds the present binary state. The third level has a decoder that asserts a unique output line for each control state. These three components are predefined standard cells in many integrated circuits.

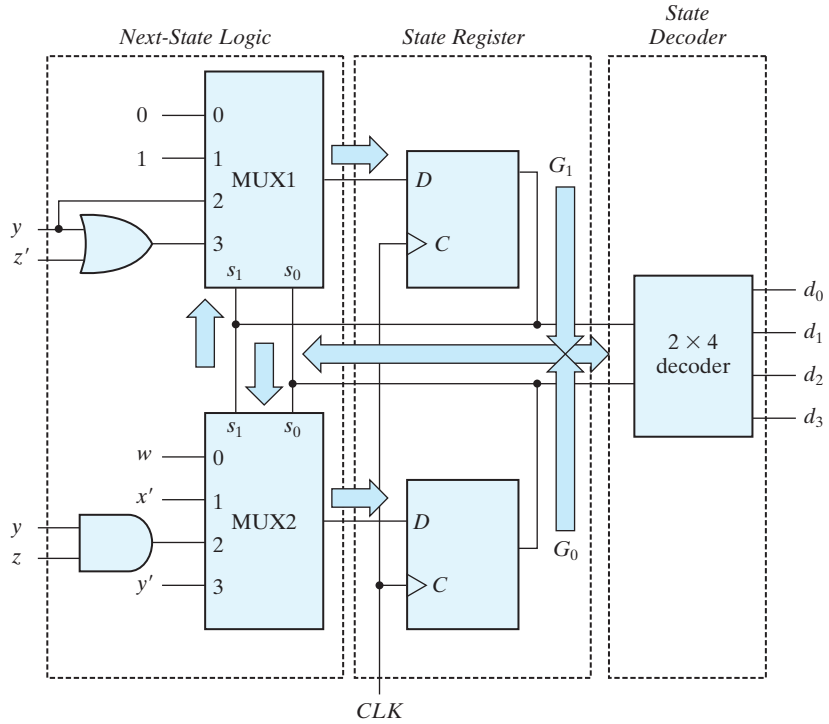
Consider, for example, the ASM chart of Fig. 8.20, consisting of four states and four control inputs. We are interested in only the control signals governing the state sequence. These signals are independent of the register operations of the datapath, so the edges of the graph are not annotated with datapath register operations, and the graph does not identify the output signals of the controller. The binary assignment for each state is indicated at the upper right corner of the state boxes. The decision boxes specify the state transitions as a function of the four control inputs:  $w$ ,  $x$ ,  $y$ , and  $z$ . The three-level control implementation, shown in Fig. 8.21, consists of two multiplexers, MUX1 and MUX2; a register with two flip-flops,  $G_1$  and  $G_0$ ; and a decoder with four outputs— $d_0$ ,  $d_1$ ,  $d_2$ , and  $d_3$ , corresponding to  $S_0$ ,  $S_1$ ,  $S_2$ , and  $S_3$ , respectively. The outputs of the state-register flip-flops are applied to the decoder inputs and also to the select inputs of the multiplexers. In this way, the present state of the register is used to select one of the inputs from each multiplexer. The outputs of the multiplexers are then applied to the  $D$  inputs of  $G_1$  and  $G_0$ . The purpose of each multiplexer is to produce an input to its corresponding flip-flop equal to the binary value of that bit of the next-state vector. The inputs of the multiplexers are determined from the decision boxes and state transitions given in the ASM chart. For example, state 00 stays at 00 or goes to 01, depending on the value of input  $w$ . Since the next state of  $G_1$  is 0 in either case, we place a signal equivalent to logic 0 in MUX1 input 0. The next state of  $G_0$  is 0 if  $w = 0$  and 1 if  $w = 1$ . Since the next state of  $G_0$  is equal to  $w$ , we apply control input  $w$  to MUX2 input 0. This means that when the select inputs of the multiplexers are equal to present state 00, the outputs of the multiplexers provide the binary value that is transferred to the register at the next clock pulse.

To facilitate the evaluation of the multiplexer inputs, we prepare a table showing the input conditions for each possible state transition in the ASM chart. Table 8.8 gives this information for the ASM chart of Fig. 8.20. There are two transitions from present state 00 or 01 and three from present state 10 or 11. The sets of transitions are separated by horizontal lines across the table. The input conditions listed in the table are obtained from the decision boxes in the ASM chart. For example, from Fig. 8.20, we note that present state 01 will go to next state 10 if  $x = 1$  or to next state 11 if  $x = 0$ . In the table, we mark these input conditions as  $x$  and  $x'$ , respectively. The two columns under “multiplexer inputs” in the table specify the input values that must be applied to MUX1 and MUX2. The multiplexer input for each present state is determined from



**FIGURE 8.20**  
Example of ASM chart with four control inputs

the input conditions when the next state of the flip-flop is equal to 1. Thus, after present state 01, the next state of  $G_1$  is always equal to 1 and the next state of  $G_0$  is equal to the complement of  $x$ . Therefore, the input of MUX1 is made equal to 1 and that of MUX2 to  $x'$  when the present state of the register is 01. As another example, after present state 10, the next state of  $G_1$  must be equal to 1 if the input conditions are  $yz'$  or  $yz$ . When these two Boolean terms are ORed together and then simplified, we obtain the single binary variable  $y$ , as indicated in the table. The next state of  $G_0$  is equal to 1 if the input



**FIGURE 8.21**  
Control implementation with multiplexers

**Table 8.8**  
Multiplexer Input Conditions

| Present State |       | Next State |       | Input Condition | Inputs              |                   |
|---------------|-------|------------|-------|-----------------|---------------------|-------------------|
| $G_1$         | $G_0$ | $G_1$      | $G_0$ |                 | MUX1                | MUX2              |
| 0             | 0     | 0          | 0     | $w'$            |                     |                   |
| 0             | 0     | 0          | 1     | $w$             | 0                   | $w$               |
| 0             | 1     | 1          | 0     | $x$             |                     |                   |
| 0             | 1     | 1          | 1     | $x'$            | 1                   | $x'$              |
| 1             | 0     | 0          | 0     | $y'$            |                     |                   |
| 1             | 0     | 1          | 0     | $yz'$           |                     |                   |
| 1             | 0     | 1          | 1     | $yz$            | $yz' + yz = y$      | $yz$              |
| 1             | 1     | 0          | 1     | $y'z$           |                     |                   |
| 1             | 1     | 1          | 0     | $y$             |                     |                   |
| 1             | 1     | 1          | 1     | $y'z'$          | $y + y'z' = y + z'$ | $y'z + y'z' = y'$ |

conditions are  $yz = 11$ . If the next state of  $G_1$  remains at 0 after a given present state, we place a 0 in the multiplexer input, as shown in present state 00 for MUX1. If the next state of  $G_1$  is always 1, we place a 1 in the multiplexer input, as shown in present state 01 for MUX1. The other entries for MUX1 and MUX2 are derived in a similar manner. The multiplexer inputs from the table are then used in the control implementation of Fig. 8.21. Note that if the next state of a flip-flop is a function of two or more control variables, the multiplexer may require one or more gates in its input. Otherwise, the multiplexer input is equal to the control variable, the complement of the control variable, 0, or 1.

### Design Example: Count the Number of Ones in a Register

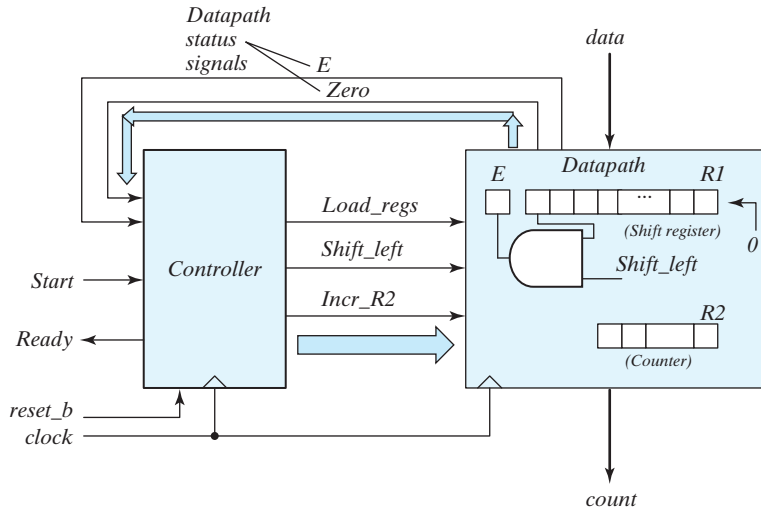
We will demonstrate the multiplexer implementation of the control unit for a system that is to count the number of 1's in a word of data. The example will also demonstrate the formulation of the ASMD chart and the implementation of the datapath subsystem.

From among various alternatives, we will consider a ones counter consisting of two registers  $R1$  and  $R2$ , and a flip-flop  $E$ . (A more efficient implementation is considered in the problems at the end of the chapter.) The system counts the number of 1's in the number loaded into register  $R1$  and sets register  $R2$  to that number. For example, if the binary number loaded into  $R1$  is 10111001, the circuit counts the five 1's in  $R1$  and sets register  $R2$  to the binary count 101. This is done by shifting each bit from register  $R1$  one at a time into flip-flop  $E$ . The value in  $E$  is checked by the control, and each time it is equal to 1, register  $R2$  is incremented by 1.

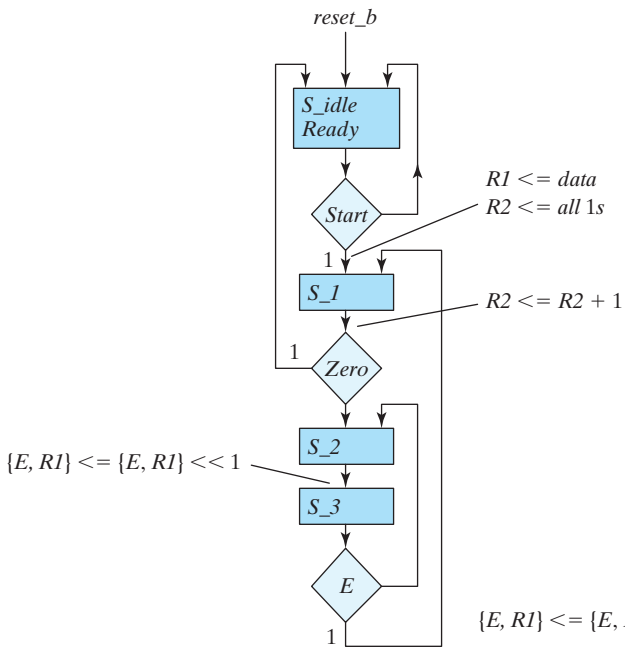
The block diagram of the datapath and controller are shown in Fig. 8.22(a). The datapath contains registers  $R1$ ,  $R2$ , and  $E$ , as well as logic to shift the leftmost bit of  $R1$  into  $E$ . The unit also contains logic (a NOR gate to detect whether  $R1$  is 0, but that detail is omitted in the figure). The external input signal *Start* launches the operation of the machine; *Ready* indicates the status of the machine to the external environment. The controller has status input signals  $E$  and  $Zero$  from the datapath. These signals indicate the contents of a register holding the MSB of the data word and the condition that the data word is 0, respectively.  $E$  is the output of the flip-flop.  $Zero$  is the output of a circuit that checks the contents of register  $R1$  for all 0's. The circuit produces an output  $Zero = 1$  when  $R1$  is equal to 0 (i.e., when  $R1$  is empty of 1's).

A preliminary ASMD chart showing the state sequence and the register operations is illustrated in Fig. 8.22(b), and the complete ASMD chart in Fig. 8.22(c). Asserting *Start* with the controller in  $S\_idle$  transfers the state to  $S\_1$ , concurrently loads register  $R1$  with the binary data word, and fills the cells of  $R2$  with 1's. Note that incrementing a number with all 1's in a counter register produces a number with all 0's. Thus, the first transition from  $S\_1$  to  $S\_2$  will clear  $R2$ . Subsequent transitions will have  $R2$  holding a count of the bits of data that have been processed. The content of  $R1$ , as indicated by  $Zero$ , will also be examined in  $S\_1$ . If  $R1$  is empty,  $Zero = 1$ , and the state returns to  $S\_idle$ , where it asserts *Ready*. In state  $S\_1$ ,  $Incr\_R2$  is asserted to cause the datapath unit to increment  $R2$  at each clock pulse. If  $R1$  is not empty of 1's, then  $Zero = 0$ , indicating that there are some 1's stored in the register. The number in  $R1$  is shifted and its leftmost

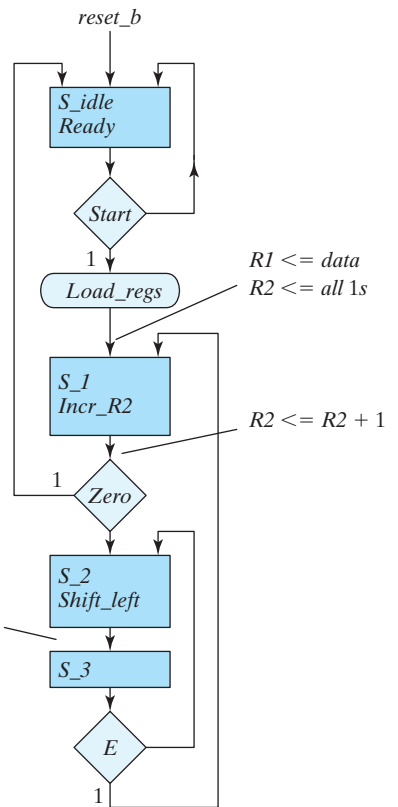




(a)



(b)



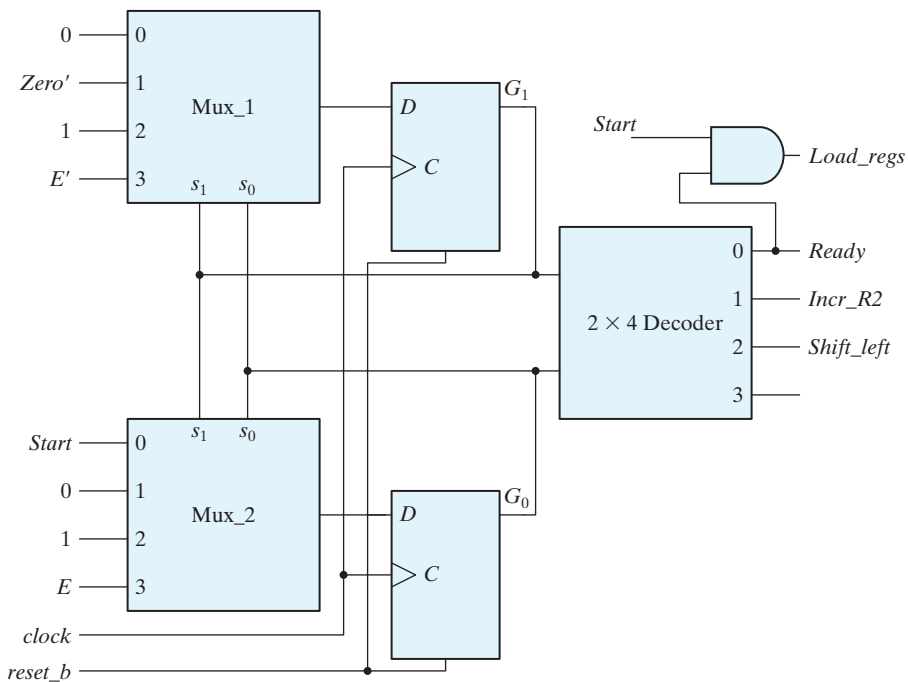
(c)

**FIGURE 8.22**  
Block diagram and ASMD chart for count-of-ones circuit

bit is transferred into  $E$ . This is done as many times as necessary, until a 1 is transferred into  $E$ . For every 1 detected in  $E$ , register  $R2$  is incremented and register  $R1$  is checked again for more 1's. The major loop is repeated until all the 1's in  $R1$  are counted. Note that the state box of  $S_3$  has no register operations, but the block associated with it contains the decision box for  $E$ . Note also that the serial input to shift register  $R1$  must be equal to 0 because we don't want to shift external 1's into  $R1$ . The register  $R1$  in Fig. 8.22(a) is a shift register. Register  $R2$  is a counter with parallel load.

The multiplexer input conditions for the controller are determined from Table 8.9. The input conditions are obtained from the ASMD chart for each possible binary state transition. The four states are assigned binary values 00 through 11. The transition from present state 00 depends on  $Start$ . The transition from present state 01 depends on  $Zero$ , and the transition from present state 11 depends on  $E$ . Present state 10 goes to next state 11 unconditionally. The values under MUX1 and MUX2 in the table are determined from the Boolean input conditions for the next state of  $G_1$  and  $G_0$ , respectively.

The implementation of the controller is shown in Fig. 8.23. This is a three-level implementation, with the multiplexers in the first level. The inputs to the multiplexers are obtained from Table 8.9.



**FIGURE 8.23**  
Control implementation for count-of-ones circuit

**Table 8.9**  
*Multiplexer Input Conditions for Design Example*

| Present State |       | Next State |       | Input Conditions | Multiplexer Inputs |         |
|---------------|-------|------------|-------|------------------|--------------------|---------|
| $G_1$         | $G_0$ | $G_1$      | $G_0$ |                  | MUX1               | MUX2    |
| 0             | 0     | 0          | 0     | $Start'$         |                    |         |
| 0             | 0     | 0          | 1     | $Start$          | 0                  | $Start$ |
| 0             | 1     | 0          | 0     | $Zero$           |                    |         |
| 0             | 1     | 1          | 0     | $Zero'$          | $Zero'$            | 0       |
| 1             | 0     | 1          | 1     | $None$           | 1                  | 1       |
| 1             | 1     | 1          | 0     | $E'$             |                    |         |
| 1             | 1     | 0          | 1     | $E$              | $E'$               | $E$     |

### HDL Example 8.8 (Ones Counter)

#### Verilog

The Verilog description instantiates structural models of the controller and the datapath. The listing of code includes the lower level modules implementing their structures. Note that the datapath unit does not have a reset signal to clear the registers, but the models for the flip-flop, shift register, and counter have an active-low reset. This illustrates the use of Verilog data type **supply1** to hardwire those ports to logic value 1 in their instantiation within *Datapath\_STR*. Note also that the testbench uses hierarchical de-referencing to access the state of the controller to make the debug and verification tasks easier, without having to alter the module ports to provide access to the internal signals. Another detail to observe is that the serial input to the shift register is hardwired to 0. The lower level models are described behaviorally for simplicity.

```

module Count_Ones_STR_STR (count, Ready, data, Start, clock, reset_b);
// Mux – decoder implementation of control logic
// controller is structural
// datapath is structural
parameter R1_size = 8, R2_size = 4;
output [R2_size - 1: 0] count;
output Ready;
input [R1_size - 1: 0] data;
input Start, clock, reset_b;
wire Load_regs, Shift_left, Incr_R2, Zero, E;

Controller_STR M0 (Ready, Load_regs, Shift_left, Incr_R2, Start, E, Zero, clock, reset_b);

Datapath_STR M1 (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock);
endmodule
```

```

module Controller_STR (Ready, Load_regs, Shift_left, Incr_R2, Start, E, Zero, clock, reset_b);
  output Ready;
  output Load_regs, Shift_left, Incr_R2;
  input    Start;
  input    E, Zero;
  input    clock, reset_b;
  supply0  GND;
  supply1  PWR;

```

```

  parameter S0 = 2'b00, S1 = 2'b01, S2 = 2'b10, S3 = 2'b11; // Binary code
  wire      Load_regs, Shift_left, Incr_R2;
  wire      G0, G0_b, D_in0, D_in1, G1, G1_b;
  wire      Zero_b = ~Zero;
  wire      E_b = ~E;
  wire [1: 0] select = {G1, G0};
  wire [0: 3] Decoder_out;
  assign    Ready = ~Decoder_out[0];
  assign    Incr_R2 = ~Decoder_out[1];
  assign    Shift_left = ~Decoder_out[2];
  and       (Load_regs, Ready, Start);
  mux_4x1_beh Mux_1 (D_in1, GND, Zero_b, PWR, E_b, select);
  mux_4x1_beh Mux_0 (D_in0, Start, GND, PWR, E, select);
  D_flip_flop_AR_b M1 (G1, G1_b, D_in1, clock, reset_b);
  D_flip_flop_AR_b M0 (G0, G0_b, D_in0, clock, reset_b);
  decoder_2x4_df M2 (Decoder_out, G1, G0, GND);
endmodule

```

```

module Datapath_STR (count, E, Zero, data, Load_regs, Shift_left, Incr_R2, clock);
  parameter    R1_size = 8, R2_size = 4;
  output       [R2_size - 1: 0] count;
  output       E, Zero;
  input        [R1_size - 1: 0] data;
  input        Load_regs, Shift_left, Incr_R2, clock;
  wire        [R1_size - 1: 0] R1;
  wire        Zero;
  supply0     Gnd;
  supply1     Pwr;
  assign Zero = (R1 == 0); // implicit combinational logic
  Shift_Reg M1 (R1, data, Gnd, Shift_left, Load_regs, clock, Pwr);
  Counter M2 (count, Load_regs, Incr_R2, clock, Pwr);
  D_flip_flop_AR M3 (E, w1, clock, Pwr);
  and         (w1, R1[R1_size - 1], Shift_left);
endmodule

```

```

module Shift_Reg (R1, data, SI_0, Shift_left, Load_regs, clock, reset_b);
  parameter      R1_size = 8;
  output          [R1_size - 1: 0]  R1;
  input           [R1_size - 1: 0]  data;
  input           SI_0, Shift_left, Load_regs;
  input           clock, reset_b;
  reg             [R1_size - 1: 0]  R1;
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) R1 <= 0;
    else begin
      if (Load_regs) R1 <= data; else
        if (Shift_left) R1 <= {R1[R1_size - 2: 0], SI_0}; end
  endmodule

```

```

module Counter (R2, Load_regs, Incr_R2, clock, reset_b);
  parameter      R2_size = 4;
  output          [R2_size - 1: 0]  R2;
  input           Load_regs, Incr_R2;
  input           clock, reset_b;
  reg             [R2_size - 1: 0]  R2;
  always @ (posedge clock, negedge reset_b)
    if (reset_b == 0) R2 <= 0;
    else if (Load_regs) R2 <= {R2_size {1'b1}}; // Fill with 1
    else if (Incr_R2 == 1) R2 <= R2 + 1;
  endmodule

```

```

module D_flip_flop_AR (Q, D, CLK, RST_b);
  output  Q;
  input   D, CLK, RST_b;
  reg     Q;
  always @ (posedge CLK, negedge RST_b)
    if (RST_b == 0) Q <= 1'b0;
    else Q <= D;
endmodule

module D_flip_flop_AR_b (Q, Q_b, D, CLK, RST_b);
  output  Q, Q_b;
  input   D, CLK, RST_b;
  reg     Q;
  assign  Q_b = ~Q;
  always @ (posedge CLK, negedge RST_b)
    if (RST_b == 0) Q <= 1'b0;
    else Q <= D;
endmodule

```

// Behavioral description of four-to-one line multiplexer

```

// Verilog 2005 port syntax
module mux_4x1_beh
  (output reg m_out,
   input in_0, in_1, in_2, in_3,
   input [1: 0] select
  );
  always @ (in_0, in_1, in_2, in_3, select) // Verilog 2005 syntax
    case (select)
      2'b00: m_out = in_0;
      2'b01: m_out = in_1;
      2'b10: m_out = in_2;
      2'b11: m_out = in_3;
    endcase
endmodule

// Dataflow description of two-to-four-line decoder
// See Fig. 4.19. Note: The figure uses symbol E, but the
// Verilog model uses enable to indicate functionality clearly.
module decoder_2x4_df (D, A, B, enable);
  output [0: 3] D;
  input A, B;
  input enable;
  assign D[0] = !(A && !B && !enable),
          D[1] = !(A && B && !enable),
          D[2] = !(A && !B && !enable),
          D[3] = !(A && B && !enable);

endmodule

module t_Count_Ones;
  parameter R1_size = 8, R2_size = 4;
  wire [R2_size - 1: 0] R2;
  wire [R2_size - 1: 0] count;
  wire Ready;
  reg [R1_size - 1: 0] data;
  reg Start, clock, reset_b;
  wire [1: 0] state; // Use only for debug
  assign state = {M0.M0.G1, M0.M0.G0};
  Count_Ones_STR_STR M0 (count, Ready, data, Start, clock, reset_b);
  initial #650 $finish;
  initial begin clock = 0; #5 forever #5 clock = ~clock; end
  initial fork
    #1 reset_b = 1;
    #3 reset_b = 0;
    #4 reset_b = 1;
    #27 reset_b = 0;
    #29 reset_b = 1;
    #355 reset_b = 0;
  join

```

```

#365 reset_b = 1;
#4 data = 8'Hff;
#145 data = 8'haa;
#25 Start = 1;
#35 Start = 0;
#55 Start = 1;
#65 Start = 0;
#395 Start = 1;
#405 Start = 0;
join
endmodule

```

---

## VHDL

The VHDL model of the ones counter instantiates a control unit and a datapath unit in the top level architecture. Structural models of those architectures are defined separately. The control unit is composed of an instantiated and connected AND gate, 4-channel multiplexer, D-type flip-flop with complemented output, and a  $2 \times 4$  decoder. The outputs of the decoder provide controlling inputs to the data path unit. The datapath unit is a structural model composed of a shift register, a counter, a D-type flip-flop, and a two-input AND gate. Those elements of the datapath unit are described by behavioral models. The testbench has a process defining the *Start*, *data*, and *reset\_b* signals. The **wait** statement at the end of the process terminates its activity.

```

entity Count_Ones_STR_vhdl is
generic integer R1_size := 8, R2_size := 4;
port (count; out Std_Logic_Vector (R2_size-1 downto 0);
      Ready: out Std_Logic; data: Std_Logic_Vector (R1_size-1 downto 0);
      Start, clock, reset_b: in Std_Logic);
end Count_Ones_STR_vhdl;

architecture Structural of Count_Ones_STR_vhdl is
  component Controller_STR_vhdl port (ready, Load_regs, Shift_left,
    Incr_R2: out Std_Logic;
  component Datapath_STR_vhdl port (count, E, Zero, data,
    Load_regs, Shift_left, Incr_R2, clock: in Std_Logic);
begin
  -- Instantiate Components
  M0: Controller_STR_vhdl port map (Ready => Ready, Load_regs => Load_regs,
    Shift_left => Shift_left, Incr_R2 => Incr_R2);
  M1: Datapath_STR_vhdl port map (count => count, E => E, Zero => Zero,
    data => data, Load_regs => Load_regs, Shift_left => Shift_left, Incr_R2 => Incr_R2,
    clock => clock);
end Structural;

```

```

entity controller_STR_vhdl is
  port (Ready, Load_regs, Shift_left, Incr_R2: out Std_Logic;
        Start, E, Zero, clock, reset_b: in Std_logic);
end Controller_STR_vhdl

architecture Structural of Controller_STR_vhdl is
  constant: Std_Logic GND := '0';
  constant: Std_Logic PWR  := '1';
  constant: Std_Logic: S0  := '00'; -- Binary state codes
  constant: Std_Logic: S1  := '01';
  constant: Std_Logic: S2  := '10';
  constant: Std_Logic: S3  := '11';
  signal    Zero_b: Std_Logic;
  signal    E_b: Std_Logic;
  signal Decoder_out: Std_Logic_Vector range (0 to 3);
  component and2_gate port (Load_regs : out Std_Logic; Ready, Start : in Std_Logic);
    end component;
  component mux_4x1_beh
    port (m_out : out Std_Logic; in_0, in_1, in_2, in_3 : in Std_Logic;
        select : in Std_Logic_Vector (1 downto 0));
    end component;
  component D_flip_flip_AR_b
    port (Q, Q_b : out Std_Logic; D : in Std_Logic; CLK, RST_b : out Std_Logic);
    end component;
  component decoder_2x4_df
    port ( D : out Std_Logic; A, B : in Std_Logic; enable : in Std_Logic);
    end component;

begin
  -- Declare concurrent signal assignments
  Zero_b <= not Zero;
  E_b <= not E;
  select <= G1 & G0;
  Ready <= not Decoder_out(0);
  incr_R2 <= not Decoder_out(1);
  Shift_left <= not Decoder_out(2);

  -- Instantiate components
  MUX_1: mux_4x1_beh port map (m_out => D_in1, in_0 => GND,
    in_1 => Zero_b, in_2 => PWR, in_3 => E_b, select => select);

  MUX_2: mux_4x1_beh port map (m_out => D_in0, in_0 => Start,
    in_1 => GND, in_2 => PWR, in_3 => E, select => select);

  M1: D_flip_flip_AR_b port map (Q => G1, Q_b => G1_b, D => D_in1,
    CLK => clock, RST_b => reset_b);

```



```
M0: D_flip_flop_AR_b port map (Q => G0, Q_b => G0_b, D => D_in0,
    CLK => clock, RST_b => reset_b);
```

```
M2: decoder_2x4_df port map (D => Decoder_out, G1, G0, GND);
```

```
G0: and2_gate port map (sig_out => Load_regs, Ready => Sig1, Start => Sig2);
end Structural;
```

```
entity Datapath_STR_vhdl is
```

```
    generic integer R1_size := 8, R2_size := 4;
```

```
    port (count: out Std_Logic_Vector (R2_size-1 downto 0);
```

```
        E, Zero: in Std_Logic; data: in Std_Logic_Vector (R1_size-1 downto 0);
```

```
        Load_regs, Shift_left, Incr_R2, clock: in Std_Logic);
```

```
end Datapath_STR_vhdl;
```

```
architecture Structural of Datapath_STR_vhdl is
```

```
    signal R1: Std_Logic_Vector (R1_size-1 downto 0);
```

```
    signal R2: Std_Logic_Vector (R2_size-1 downto 0);
```

```
    constant Zero := '0';
```

```
    constant PWR := '1';
```

```
    constant GND := '0';
```

```
    component Shift_Reg port (R1 out Std_Logic_Vector (R1_size-1 downto 0);
```

```
        data : in Std_Logic_Vector (R1_size-1 downto 0);
```

```
        SI_0, Shift_left, Load_regs : in Std_Logic;
```

```
        clock, reset_b : in Std_Logic); end component;
```

```
    component Counter port (R2: out Std_Logic_Vector (R2_size-1 downto 0);
```

```
        Load_regs, Incr_R2 : in Std_Logic;
```

```
        clock, reset_b : in Std_Logic); end component;
```

```
    component D_flip_flop_AR port (Q: out Std_Logic; D : in Std_Logic;
```

```
        CLK, RST, Reset_b : in Std_Logic); end component;
```

```
    component and2_gate port (sig_out: out Std_Logic; sig1, sig2 : in Std_Logic);
```

```
        end component;
```

```
begin
```

```
-- Concurrent signal assignments
```

```
    Zero <= (1 = '0');
```

```
-- Instantiate components
```

```
M1: Shift_Reg port map (R1 => R1, data => data, SI_0 => GND, Shift_left => Shift_left,
    Load_regs => Load_regs, clock => clock, reset_b => PWR);
```

```
M2: Counter port map (R2 => count, Load_regs => Load_regs, Incr_R2 => Incr_R2,
    clock => clock, reset_b => PWR);
```

```

M3: D_flip_flop_AR port map (Q => E, D => w1, CLK => clock, RST => PWR);
G0: and2_gate port map (sig_out => w1, sig1 => R1(R1_size-1), sig2 => Shift_left);
end Structural;

```

```

entity Shift_Reg is
  generic integer R1_size := 8;
  port (R1: out Std_Logic_Vector (R1_size-1 downto 0);
        data: in Std_Logic_Vector (R1_size-1 downto 0);
        SI_0, Shift_left, Load_regs: in Std_Logic;
        clock, reset_b: in Std_Logic);
end Shift_Reg;

architecture Behavioral of Shift_Reg is
  constant Std_Logic PWR := '1';
  constant Std_Logic GND := '0';
begin
  process (clock, reset_b)
  begin
    if (reset_b = '0' then R1 <= '0';
    elsif clock'event and clock = '1' then
      if Load_regs = '1' then R1 <= data;
      elsif Shift_left = '1' then R1 <= R1(R1_size-2 downto 0) & SI0;
      end if;
    end if;
  end process;
end Behavioral;

```

```

entity Counter is
  generic integer R2_size := 4;
  port (R2: out Std_Logic_Vector (R2_size-1 downto 0);
        Load_regs, Incr_R2: in Std_Logic; clock, reset_b: in Std_Logic);
end Counter;

architecture Behavioral of Counter'
  variable k: integer;
begin
  process (clock, reset_b) begin
    if (reset_b = '0' then R2 <= '0';
    elsif clock'event and clock = '1' then -- Fill with 1 s
      if Load_regs = '1' then for k in range (0 to R2_size-1) R2(k) <= '1';
      elsif Incr_R2 = '1' then R2 = R2 + 1;
      end if;
    end if;
  end process;
end Behavioral;

```

```

entity D_flip_flop_AR is
  port D_flip_flop_AR_b (Q, D, CLK, RST_b);
end D_flip_flop_AR;

```

```

architecture Behavioral of D_flip_flop_AR is
begin
process
  if RST_b = '0' then Q <= '0';
  elsif clock'event and clock = '1' then Q <= D;
  end if;
end process;
end Behavioral;

```

```

entity D_flip_flop_AR_b is
  port (Q, Q_b, D, CLK, RST_b);
end D_flip_flop_AR_b;

```

```

architecture Behavioral of D_flip_flop_AR_b is
begin Q_b <= not Q;
process begin
  if RST_b = '0' then Q <= '0';
  elsif clock'event and clock = '1' then Q <= D;
  end if;
end process;
end Behavioral;

```

```

entity t_Counter_Ones_STR_vhdl is
  generic R1: Std_Logic_Vector (R1_size-1 downto 0);
  generic R2: Std_Logic_Vector (R2_size-1 downto 0);
  port ();
end t_Countr_Ones_STR_vhdl;

```

```

architecture Behavioral of t_Countr_Ones_STR_vhdl is
  signal t_count : Std_Logic_Vector (R2_size-1 downto 0);
  signal t_Ready: Std_Logic; t_data: Std_Logic_Vector (R1_size-1 downto 0);
  signal t_Start, t_clock, t_reset_b: Std_Logic;
  component Count_Ones_STR_vhdl port (count; out Std_Logic_Vector (R2_size-1
downto 0); Ready: out Std_Logic;
    data: Std_Logic_Vector (R1_size-1 downto 0);
    Start, clock, reset_b: in Std_Logic);

```

```

begin
-- Instantiate UUT

```

```

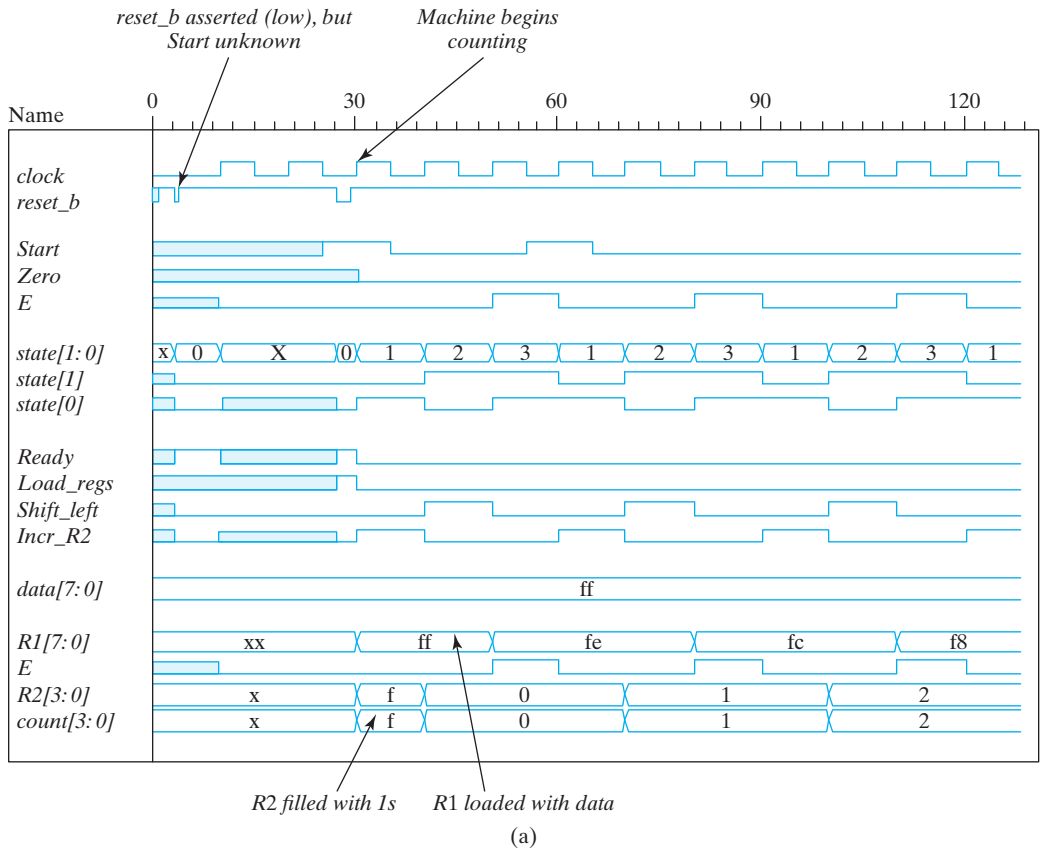
M0: Count_Ones_STR_vhdl
    port map (count => t_count, Ready => t_Ready,
              Start => t_Start, clock => t_clock, reset_b => t_reset_b);
-- Generate stimulus waveforms
process begin
    t_clock <= '0';
    loop t_clock <= not t_clock after 5 ns; end loop;
end process;
process begin
    reset_b <= '1' after 1 ns;
    reset_b <= '1' after 3 ns;
    reset_b <= '1' after 4 ns;
    reset_b <= '1' after 27 ns;
    reset_b <= '1' after 29 ns;
    reset_b <= '0' after 355 ns;
    reset_b <= '1' after 365 ns;
    data <= '11111111' after 4 ns;
    data <= '10101010' after 145 ns;
    Start <= '1' after 25 ns;
    Start <= '0' after 35 ns;
    Start <= '1' after 55 ns;
    Start <= '0' after 65 ns;
    Start <= '1' after 395 ns;
    Start <= '0' after 405 ns;
wait;           // Indefinite suspension
end process;
end Behavioral;

```

## Testing the Ones Counter

The testbench in HDL Example 8.8 produces the simulation results in Fig. 8.24. Annotations have been added for clarification and to encourage examination of the behavior of the machine. In Fig. 8.24(a), *reset\_b* is toggled low at  $t = 3$  to drive the controller into *S\_idle*, but with *Start* not yet having an assigned value. (The default is x.) Consequently, the controller enters an unknown state (the shaded waveform) at the next clock, and its outputs are unknown.<sup>12</sup> When *reset\_b* is asserted (low) again at  $t = 27$ , the state enters *S\_idle*. Then, with *Start* = 1 at the first clock after *reset\_b* is de-asserted, (1) the controller enters *S\_I*, (2) *Load\_regs* causes *R1* to be set to the value of *data*, namely, 8'Hff, and (3) *R2* is filled with 1's. At the next clock, *R2* starts counting from 0. *Shift\_left* is asserted while the controller is in state *S\_2*, and *incr\_R2* is asserted while the controller is in state *S\_I*. Notice that *R2* is incremented in the next cycle after *incr\_R2* is asserted. No output

<sup>12</sup> In actual hardware, the values will be 0 or 1. Without a known applied value for the inputs, the next state and outputs will be undetermined, even after the reset signal has been applied.

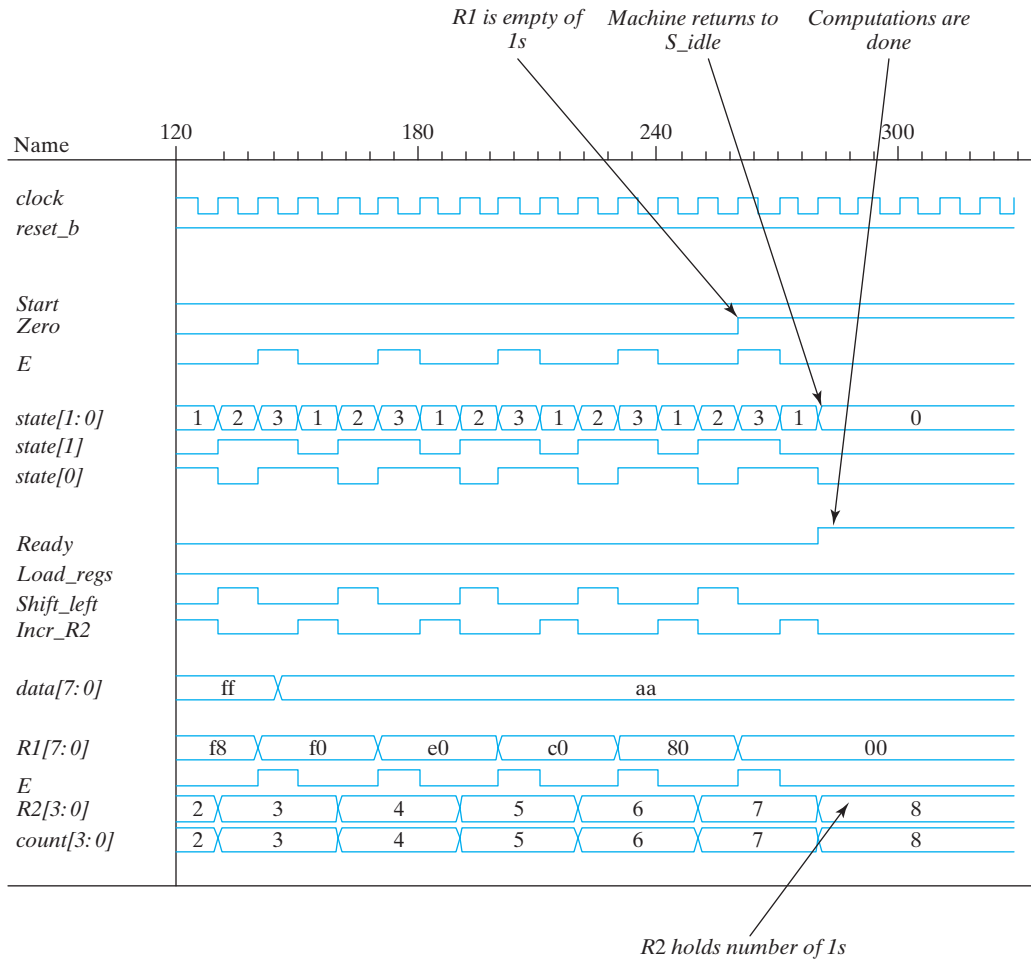
**FIGURE 8.24**

Simulation waveforms for count-of-ones circuit

is asserted in state  $S_3$ . The counting sequence continues in Fig. 8.24(b) until *Zero* is asserted, with *E* holding the last 1 of the data word. The next clock produces *count* = 8, and *state* returns to  $S_{idle}$ . (Additional testing is addressed in the problems at the end of the chapter.)

## 8.11 RACE-FREE DESIGN (SOFTWARE RACE CONDITIONS)

Once a circuit has been synthesized, either manually or with tools, it is necessary to verify that the simulation results produced by the HDL behavioral model match those of the netlist of the gates of the physical circuit. It is important to resolve any mismatch, because the behavioral model was presumed to be correct.



(b)

**FIGURE 8.24 (Continued)**

There are various potential sources of mismatch between the results of a simulation, but we will consider one that typically happens in HDL-based design methodology.

**Verilog** Three realities contribute to the potential problem: (1) a physical feedback path exists between a datapath unit and a control unit whose inputs include status signals fed back from the datapath unit; (2) blocking procedural assignments execute immediately, and behavioral models simulate with 0 propagation delays, effectively creating immediate changes in the outputs of combinational logic when its inputs change (i.e., changes in the inputs and the outputs are scheduled in the same time step of the simulation); and (3) the order in which a simulator executes multiple blocking assignments to the same variable at a given time step of the simulation is indeterminate (i.e., unpredictable).

Now consider a sequential machine with an HDL model in which all assignments are made with the blocking assignment operator. At a clock pulse, the register operations in the datapath, the state transitions in the controller, the updates of the next state and output logic of the controller, and the updates to the status signals in the datapath are all scheduled to occur at the same time step of the simulation. Which executes first? Suppose that when a clock pulse occurs, the state of the controller changes before the register operations execute. The change in the state could change the outputs of the control unit. The new values of the outputs would be used by the datapath when it finally executes its assignments at that same clock pulse. The result might not be the same as it would have been if the datapath had executed its assignments before the control unit updated its state and outputs. Conversely, suppose that when the clock pulse occurs, the datapath unit executes its operations and updates its status signals first. The updated status signals could cause a change in the value of the next state of the controller, which would be used to update the state. The result could differ from that which would result if the state had been updated before the edge-sensitive operations in the datapath executed. In either case, the timing of register transfer operations and state transitions in the different representations of the system might not match. *Failing to detect a mismatch can have disastrous consequences for the user of the design. Finding the source of the mismatch can be very time-consuming and costly. It is better to avoid the mismatch by following a strict discipline in your design.* Fortunately, there is a solution to this dilemma.

A designer can eliminate the *software race conditions* just described by observing the rule of modeling combinational logic with blocking assignments and modeling state transitions and edge-sensitive register operations with nonblocking assignments. A software race cannot happen if nonblocking operators are used as shown in all of the examples in this text, because the sampling mechanism of the nonblocking operator breaks the feedback path between a state transition or edge-sensitive datapath operation and the combinational logic that forms the next state or inputs to the registers in the datapath unit. The mechanism does this because simulators evaluate the expressions on the right-hand side of their nonblocking assignment statements before any blocking assignments are made. Thus, the nonblocking assignments cannot be affected by the results of the blocking assignments. This matches the hardware reality. Always use the blocking operator to model combinational logic, and use the nonblocking operator to model edge-sensitive register operations and state transitions.

It also might appear that the physical structure of a datapath and the controller together create a physical (i.e., hardware), race condition, because the status signals are fed back to the controller and the outputs of the controller are fed forward to the datapath. However, timing analysis can verify that a change in the output of the controller will not propagate through the datapath logic and then through the input logic of the controller in time to have an effect on the output of the controller until the next clock pulse. The state cannot update until the next edge of the clock, even though the status signals update the value of the next state. The flip-flops of the state register cut the feedback path between clock cycles. In practice, timing analysis verifies that the circuit will operate at the specified clock frequency, or it identifies signal paths whose propagation delays are problematic. Remember, the design must implement the correct logic and operate at the speed prescribed by the clock.

**VHDL** Race conditions are a concern in HDL design because they can create a mismatch between the behavior that is evident in a simulation and the behavior that is produced by the physical hardware. The mismatch can suggest that the results of simulation can be misleading. The mismatch might go undetected too, depending on the completeness of the testbench. The structure of a controller-datapath system admits the possibility of a mismatch because there is a feedback path between the units. The datapath and the controller are synchronized by a common clock. At the stroke of the clock, the datapath reads the inputs from the controller and takes action to execute register operations. At the same time, the outputs of the controller are changing to the value prescribed by the computed value of the next state. If the new values arrive at the datapath too soon they might corrupt the signals read by the datapath. Likewise, if changes in the datapath cause status signals to change before the control signals have been read, the datapath unit might be misdirected. Either way, the HDL model looks like it is behaving correctly, but the hardware behaves differently. The designer must abide by a coding discipline that precludes race conditions. That discipline requires that (1) assignments to all registers in the control unit, that is, the state register, and those in the datapath unit be made using the signal assignment operator, and (2) combinational logic functions for the next state in the control unit and for the status signals in a datapath be formed using a level-sensitive process and variable assignments. These rules provide assurance that the sampling mechanism of signal assignments in a process eliminates the possibility of signals being affected by assignments to variables, and prevents register operations in the control unit from affecting register operations in the datapath unit, and vice versa.

## 8.12 LATCH-FREE DESIGN (WHY WASTE SILICON?)

---

**Verilog** Continuous assignments model combinational logic implicitly. A feedback-free continuous assignment will synthesize to combinational logic, and the input–output relationship of the logic is automatically sensitive to all of the signals referenced by the assignment statement. In simulation, the simulator monitors the right-hand sides of all continuous assignments, detects a change in any of the referenced variables, and updates the left-hand side of an affected assignment statement. Unlike a continuous assignment, a cyclic behavior is sensitive to only those signals that are in its sensitivity list. If a level-sensitive cyclic behavior is used to describe combinational logic, it is essential that the sensitivity list include every variable that is referenced on the right-hand side of an assignment statement in the behavior. If the list is incomplete, the logic described by the behavior will be synthesized with latches at the outputs of the logic. This implementation wastes silicon area and may have a mismatch between the simulation of the behavioral model and the synthesized circuit. These difficulties can be avoided by ensuring that the sensitivity list is complete, but, in large circuits, it is easy to fail to include every referenced variable in the sensitivity list of a level-sensitive cyclic behavior. Consequently, Verilog 2001 included a new operator to reduce the risk of accidentally synthesizing latches.

In Verilog 2001, the tokens @ and \* can be combined as @\* or @(\*) and are used without a sensitivity list to indicate that execution of the associated statement is sensitive



to every variable that is referenced on the right-hand side of an assignment statement in the logic. In effect, the operator `@*` indicates that the logic is to be interpreted and synthesized as level-sensitive combinational logic; the logic has an implicit sensitivity list composed of all of the variables that are referenced by the procedural assignments. Using the `@*` operator will prevent accidental synthesis of latches.

### HDL Example 8.9

---

#### Verilog

The following level-sensitive cyclic behavior will synthesize a two-channel multiplexer:

```
module mux_2_V2001 (output reg [31: 0] y, input [31: 0] a, b, input sel);
  always @*
    y = sel ? a: b;
endmodule
```

The cyclic behavior has an implicit sensitivity list consisting of *a*, *b*, and *sel*.

---

**VHDL** Concurrent signal assignments model combinational logic implicitly. A feedback-free assignment will synthesize to combinational logic, and the input–output relationship of the logic is automatically sensitive to all of the signals referenced by the assignment statement. In simulation, the simulator monitors the right-hand sides of all signal assignments, detects a change in any of the referenced signal, and updates the left-hand side of an affected assignment statement.

Unlike a concurrent signal assignment, a process is sensitive to only those signals that are in its sensitivity list. If a level-sensitive process is used to describe combinational logic, it is essential that the sensitivity list include every signal that is referenced on the right-hand side of a statement in the process. If the list is incomplete, the logic described by the process will be synthesized with latches at the outputs of the logic. This implementation wastes silicon area and may have a mismatch between the simulation of the behavioral model and the synthesized circuit. These difficulties can be avoided by ensuring that the sensitivity list is complete.

## 8.13 SYSTEMVERILOG—AN INTRODUCTION

---

Verilog is more robust than is apparent from the examples we have presented. Multidimensional arrays, variable part selects, array, bit, and part selects, signed reg, net, and port declarations, and local parameters are some of the Verilog constructs that we have not discussed. Our presentation has selectively introduced mainline features of the language—enough to support meaningful examples and problems, and introduce modeling with HDLs. Verilog has not been a static language. In fact, the advance of design methodology and the needs of industry have led to the development and standardization of

the third language that we will consider: SystemVerilog.<sup>13</sup> Its features address recognized shortfalls of Verilog-2005 and extend its use beyond hardware description, addressing the need to more robustly and efficiently describe, verify, and synthesize hardware systems. However, all the features of Verilog-2005 are incorporated in SystemVerilog, so it actually subsumes Verilog-2005. Models that compile under Verilog-2005 rules will compile with SystemVerilog compilers. So our treatment of SystemVerilog builds on our treatment of Verilog.

We will introduce a limited, bare-bones, set of SystemVerilog features, focusing on those which have direct application to the examples and end-of-chapter problems at the level of an *introduction* to digital design. The remaining constructs are left to the reader's pursuit of continuing education [15].

## New Data types

The discussion in Chapter 4 noted that here are two predefined groups of data types in Verilog-2005 and earlier versions of the language: nets and variables. Objects of both groups take values in a predefined logic system whose value set consists of four values: {0, 1, x, z}. Nets express structural connectivity. They may be the input or the output of a module, and may be assigned value by a primitive, and a continuous assignment statement, thereby implementing implicit combinational logic. Variables are storage containers whose value is assigned by a procedural statement. They may be the output of a module, but may not be an input. Identifiers that are not explicitly typed have a default type, usually a **wire**.

A variable having type **reg** has no automatic association with a hardware register, and does not necessarily synthesize to such, but because its name suggests that it does, the need to clarify the typing of variables became apparent to the user community. In fact, a synthesis tool may implement a **reg** variable with combinational logic or with sequential logic, depending on its context in the source code. This ambiguity led to SystemVerilog providing a new type and keyword: **logic**. It differs from a **reg** mainly in name only, and its value is expressed in the same 4-valued data type logic as other Verilog variables. It may be a single bit or a vector, and each element of a vector has a value taken from the 4-value set of allowed values. A difference between a **reg** variable and a **logic** variable is that the latter may be assigned value by a continuous statement; a **reg** may not. If a signal is assigned value by multiple drivers it must be declared as a **wire** or a **tri**.<sup>14</sup> In fact, **logic** and **reg** are interchangeable, but in a SystemVerilog model the former name doesn't suggest how a synthesis tool will implement a variable in hardware. A **reg** variable may be assigned value only by a procedural statement; a **logic** variable is not so restricted. It can be the output of a gate (primitive) or be assigned value by a single continuous assignment, as well as a procedural statement. Consequently, the type **logic** can replace the type **wire** in a SystemVerilog model. This might not be attractive in structural modeling because it does not convey the semantics of connectivity, but it

<sup>13</sup> SystemVerilog 1800–2012 IEEE Standard for SystemVerilog—Unified hardware Design, Specification, and Verification Language

circumvents the restriction that a **wire** may not be assigned value by a procedural statement. Type **logic** does have one restriction: in a circuit having multiple drivers attached to the same net, the type cannot be **logic**—it must be **tri**. A **tri** net accommodates multiple drivers and has built-in resolution of multiple drivers. An identifier that is declared to have data type **logic** or **bit** but is not explicitly declared to be a net or a variable, is implicitly a variable, and its use must conform to the rules for nets. For example, the declaration **logic** [15:0] *Ibus* infers a 4-state data type variable, implying that *Ibus* may be assigned value by a procedural statement.

SystemVerilog also defines a two-state data type called **bit**.<sup>15</sup> Its set of possible values is limited to only 0 and 1. It is used where software tools have no need for additional logic values (i.e., x and z), such as formal verification tools. Scalar and vector variables having type **bit** are declared in the same way as variables having type **reg** or **logic**. *Be aware that variables of type **bit** are initialized to a value of 0 in simulation.* Synthesis tools do not abide by that default assignment. Variables of type **bit** or **logic** differ only in their having different sets of logic values. Whether a variable of either type is synthesized as the output of combinational or sequential logic is inferred by a synthesis tool and depends on the context of the assignment.

SystemVerilog relaxes restrictions on where a variable may be assigned value. Any variable may be assigned value (1) by a single continuous assignment statement, (2) as the output of a primitive, an output port of a module, an inout port of a module, (3) any number of **initial** or **always**<sup>16</sup> procedural blocks, and (4) by a single **always\_comb**, **always\_ff**, or **always\_latch** procedural block.<sup>17</sup> These contexts are sufficient for a synthesis tool to infer whether the logic is combinational or sequential. For example, a variable having type **logic** will be inferred to be a net if the variable is a module **input** or **inout** port.

A common mistake made by novices in Verilog is to declare a **reg** typed variable to be an input port. That mistake is precluded if type **logic** is used at input and output ports. The restricted possibilities for assigning value to variables in Verilog require the designer to know in advance the context in which a variable will be assigned value. The relaxed rules in SystemVerilog don't require such a priori decision, and provide more flexibility.<sup>18</sup> In general, it is advisable to avoid using type **bit**, except, for example, as a counter in a **for** loop, or a signal generated in a testbench and to use logic almost everywhere [18]. This guidance avoids mismatches between simulation and synthesis results having their origin in the fact that variables of type **bit** are initialized to 0 in simulation, but not necessarily so in synthesis.

<sup>14</sup> A synthesis tool will flag an error when it detects that a **logic** variable has multiple drivers.

<sup>15</sup> SystemVerilog also adds **byte**, **shortint**, **int**, and **longint** two-bit data types for use in abstract models, and special types: **void** and **shortreal**.

<sup>16</sup> Multiple assignments to the same variable are ill-advised, because the order in which simultaneous assignments are made is not deterministic, with consequences for the outcome of simulation.

<sup>17</sup> **always\_comb**, **always\_ff**, and **always\_latch** are specific to SystemVerilog, have no counterpart in Verilog, SystemVerilog tool environment.

<sup>18</sup> There are some restrictions, (e.g., regarding resolution of multiple drivers of a variable) but they are not within the scope of this text.

**HDL Example 8.10**


---

The 64-bit comparator described below uses SystemVerilog's new data types.

```

module Comparator_64_bit (
    output logic a_lt_b, a_eq_b, a_gt_b,
    input logic [63:0] a, b);

    always @ (a, b) begin
        a_lt_b = (a < b);
        a_eq_b = (a == b);
        a_gt_b = (a > b);
    end
endmodule

```

---

SystemVerilog correctly infers whether a signal is a net (i.e., establishes connectivity) or a variable (i.e., retains an assigned value). A signal declared to have type **logic** will be inferred to be a net or a variable automatically, depending on whether it is a module input or output, and whether it is assigned value by a primitive, a continuous assignment, or a procedural statement. An error will be detected, for example, if a signal has type **logic**, is assigned value by a procedural statement, and is connected to an input port of a module. A signal which has type **logic** and which is assigned value by a continuous assignment will be inferred to be a net, and can be connected to an output port of a module, the output of a primitive, or it can appear in an expression.

**User-Defined Data Types**

Verilog does not have user-defined data types. SystemVerilog addresses that limitation by defining (1) a new keyword, **typedef**, to declare user-defined data types, and (2) a new keyword, **enum**, to define enumerated data types. The **typedef** keyword may be used locally, within a module, or in a *compilation unit* (see below).

**HDL Example 8.11 (User-Defined Data Type)**


---

The code fragment below creates a type “double byte” having 16 bits. Variables having that type are then declared.

```

typedef [15:0] logic double_byte_t;           // User-defined 16-bit data type
double_byte_t  dbyte_A, dbyte_B, dbyte_C;    // Variables of type double_byte_t

```

---

**Practice Exercise 8.12**

Write a SystemVerilog statement declaring variables *A*, *B*, and *C* to be of type *Num\_type\_t*, whose values are three-bit vectors having type **logic**.

**Answer:** Num\_type\_t **logic** [2:0] *A*, *B*, *C*;

---

It is advisable to use user-defined data types together with packages to ensure that declarations are consistent throughout a project [18]. Packages gather in a common, shared declaration space definitions of parameters, constants and user-defined types, among other items. The contents of a package can be referenced from any module in the design. This eliminates the need to have duplicate declarations.

#### HDL Example 8.14 (Packages)

---

```
package Processor_types;
    typedef logic [63:0] data_bus_t;
    typedef logic [15:0] instr_bus_t;
endpackage;
```

An explicit reference to an item in a package is made by citing the package name followed by ::.

```
module simple_Machine (
    input Processor_types :: data_bus_t code_word,
    output Processor_types :: instr_bus_t fpoint_instr);
    . . .
endmodule
```

---

## Naming Convention

A SystemVerilog naming convention improves the readability and maintainability of the source code by adding the characters `_t` to complete the name of a user-defined data type.

## Enumerated Types

A SystemVerilog enumerated data type associates a set of unique, named values with an abstract variable.<sup>19</sup> This improves the readability of code by assigning meaningful names to what would otherwise be identified as numerical quantities.

#### HDL Example 8.15 (Enumerated Data Type)

---

The following SystemVerilog enumerated type represents speeds that might be states of a simple control system.

```
enum {slow, medium, fast, stopped} speeds_t;
```

---

<sup>19</sup> Verilog achieves this effect by defining **parameter** constants and using **define** macros substitutions. While effective, the resulting code is less maintainable.

The names of an enumerated variable are represented, by default,<sup>20</sup> with a 32-bit, 2-state value having type **int**. The assigned values ascend from 0, which is assigned to the first (left-most) name in the list, and increment by 1 toward the right-most name in the set. Alternatively, the user can assign values to the names to implement specific requirements, for example, a one-hot code for the states of a finite state machine.

### HDL Example 8.16 (Enumerated Data Type)

The declaration below creates an enumerated type, *state*, which has three names and implements a one-hot code. Each value is a vector of three bits. If values are explicitly assigned to the valid names of an enumerated variable, they must match the size of the data type, which has a default size of 32 bits.

```
enum logic [2:0] {S_idle = 3'b001, S_1 = 3'b010, S_2 = 3'b100} state;
```

Some cautionary observations are important to note: Verilog allows a variable of any type to be assigned to a variable, and automatically converts the value to the data type of the variable to which it is being assigned. Enumerated types are restricted in their assignments. A variable of an enumerated type can be assigned to (1) a value from the list of variables with which it was declared (enumerated type list), (2) a variable from the list of variables with which it was declared, and (3) a value that has been cast to its type. An attempt to do otherwise will produce a SystemVerilog compiler syntax error [18], but would go undetected in a Verilog environment, leading to difficulty in debugging in a gate-level implementation.

### Practice Exercise 8.13 (Enumerated type)

Write a SystemVerilog statement declaring an enumerated type named *state\_type\_t* and having one-hot state values for *s0*, *s1*, *s2*, and *s3*.

**Answer:**

```
enum logic [3:0] {s0 = 4'b0001, s1 = 4'b0010, s2 = 4'b0100, s3 = 4'b1000} state_type_t;
```

## Compilation Unit

An identifier in Verilog has a limited scope, that is, the module or named block in which it is declared. SystemVerilog aggregates into a *compilation unit* all source files that are compiled at the same time. This allows declarations to be made outside of a module, but makes the compiled objects visible to all modules in the compilation unit. The scope of a compilation unit may contain declarations of nets, variables, and constants, user-defined data types, tasks, and function, and declarations of time units and precision. Such declarations are referred to as *external declarations*. The scope of a compilation

<sup>20</sup> If a data type for the enumerated type names is not explicitly declared, the name values will default to type **int** [15].

unit is not global; it extends to only those source files that have been compiled together at the same time.

## Explicit Behavioral Intent

The Verilog keyword **always** declares a procedural block and may be associated with level or edge-sensitive behavior. Synthesis tools parse the code that follows the keyword to infer whether the outcome of synthesis is combinational logic, a latch, or a register. The keyword, **always**, does not convey the intent of the designer, and the code itself can inadvertently lead to undesired results. Merely omitting an identifier from the sensitivity list of a level-sensitive behavior will produce a latched circuit instead of a strictly combinational circuit. The result is undesirable because the behaviors of the physical circuit and the HDL model may not match. Verilog-2005 allows multiple **always** blocks to assign value to the same variable, introducing an element of randomness,<sup>21</sup> and confusion, in determining the outcome of simulation. The new variants of **always** blocks (see below) included in SystemVerilog permit only one **always** block to assign value to a variable.<sup>22</sup>

SystemVerilog provides three new keywords that explicitly convey the intent of the designer and ensure that the result of synthesis matches that intent. The **always\_comb** keyword is not combined with a sensitivity list, but declares that the intended behavior is that of synthesizable combinational logic. A SystemVerilog simulator and a compatible synthesis tool will automatically form an implicit sensitivity list consisting of all of the identifiers that are referenced within the statement or block that follows the keyword **always\_comb**, and all the identifiers that are referenced within any functions that are called by the procedural block. With **always\_comb** there is no need to fret about accidentally omitting an identifier and thereby synthesizing unwanted latches, and hardware that fails to operate correctly. The new procedural block **always\_comb** clearly indicates the designer's intent, making maintenance of the code easier, whether by other designers or by software tools. An **always\_comb** procedural block is automatically triggered at simulation time zero, after **initial** and **always** blocks have been launched. Consequently the combinational logic produced by **always\_comb** produces outputs that are consistent with the values of the inputs to the logic when simulation begins. This matters especially with variables having type **bit**, because they are automatically initialized to logic 0.

The **always\_ff** SystemVerilog keyword expresses the intent that the following procedural statement describe synthesizable sequential logic. The keyword must be accompanied by a sensitivity list, and every identifier in the list must have an edge qualifier (**posedge** or **negedge**). An identifier corresponding to synchronous control would not be included in the sensitivity list, but would be tested by the procedural statement. It is left to a synthesis tool to determine whether the procedural statement(s) infer sequential (i.e., registered) logic.

<sup>21</sup> Verilog-2005 permits statements in multiple behaviors assigning to the same variable, but does not specify the order in which such assignments are to be made if they occur simultaneously. The results are implementation-dependent on the simulator. Therefore, avoid writing code in which a variable is assigned value in multiple behavioral statements.

<sup>22</sup> This restriction applies to assignments by **initial** procedural blocks too.

The **always\_latch** keyword expresses the intent that the accompanying procedural statement(s) actually models latched behavior. The code structure that corresponds to latched behavior must have at least one path through the logic such that at least one variable is *not* assigned value. The **always\_latch** procedural block would have the same sensitivity list as the **always\_comb** block, but would not have the same internal assignments to variables. For example, the code in an **always\_latch** block might have an **if** statement without a matching **else** statement. A synthesis tool would determine whether the procedural statements actually infer a latch. An **always\_latch** procedural block executes automatically at simulation time = 0, thereby ensuring that the outputs of latched logic are consistent with the input values when simulation begins [15].

Important guidance: The benefit of using SystemVerilog's new procedural blocks is so great that it is recommended by industry experts that the new blocks be used exclusively in RTL code.

---

### Practice Exercise 8.14

Write an **always\_comb** statement to implement combinational logic described by the Boolean equation  $y = (A \& B) | (C \& D)$ ;

**Answer:** **always\_comb**  $y = (A \& B) | (C \& D)$ ;

---

## Bottom-Testing Loop

The **while** conditional loop in Verilog is known as a top-testing loop. The statements in the loop do not execute if the condition governing execution is false. SystemVerilog has a **do . . . while** which executes the loop statements *before* testing whether to reexecute the loop, that is, the test is made at the bottom of the list of statements within the loop. This guarantees that the loop statements execute at least once. The code is considered to be more efficient and intuitive.

### HDL Example 8.17 (do . . . while loop)

---

A procedural block is to monitor eight-bit addresses and assert a flag if an address is invalid [15]. The **do . . . while** loop is used because it allows the flags to be initialized before checking the status of an address. The code checks whether an address is within a specified range for validity, and, if so, reads memory at the indicated address. Reading is bypassed if the address is invalid, that is, out of range, because the code checks whether the address is valid first. The loop is executed as long as the address is valid. At each iterant, the address is decremented.

```
always_comb begin
  do begin
    done = 0;
    Error_Out_of_Range = 0;      // Initialize flag
    mem_out = mem[address];
    if (address < 128 | ADDRESS > 255) begin // Invalid address
```



```

        Error_Out_of_Range = 1;
        mem_out = mem[128]
    end
    else if (address == 128) done = 1;
    address = address - 1;
end
while (address >= 128 && address <= 255) ;
end

```

---

## Operators

C-like operators that combine increment/decrement operations with assignment are new in SystemVerilog, and were presented in Table 4.11b in Chapter 4.

### (case . . . inside)

Section 4.13.4 noted that **casex** and **casez** should not be used in RTL code that is intended to be synthesized, the root problem being that these constructs treat don't cares in both the case expression and the case items, which leads to possible mismatches between results in simulation and in synthesis. SystemVerilog provides a new construct, **case . . . inside**, which eliminates the possibility of mismatch. This new construct treats don't cares in only the case items. All bits in the case expression are considered, and only those in the case items are masked, according to the presences of x, z, or ?.

#### HDL Example 8.18 (case . . . inside)

---

In the code fragment below bit 4 is masked.

```

case (instruction) inside
    8'b0000_?000: opc = instruction {4: 0};

```

---

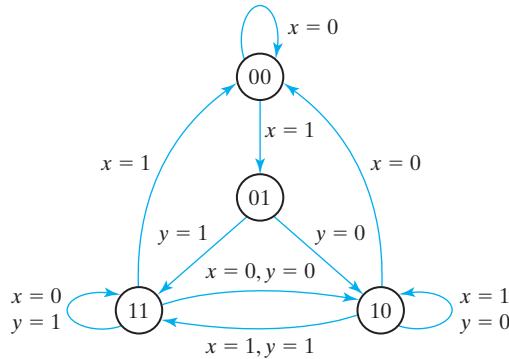
## PROBLEMS

---

(Answers to problems marked with \* appear at the end of the book.)

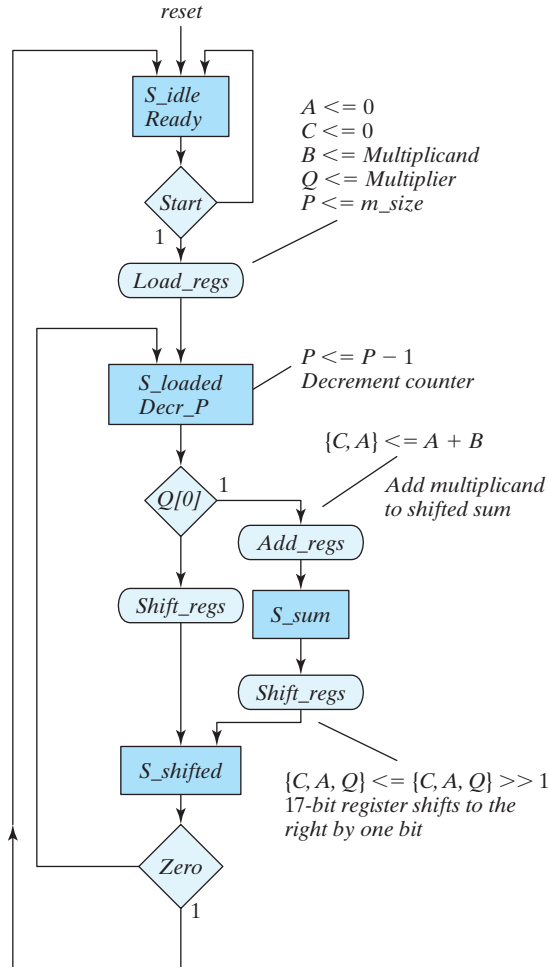
- 8.1** Write statements in RTL (register transfer level) notation that would perform the following operations:
- (a)\* Transfer the contents of register *R1* to *R2* and clear *R1* concurrently.
  - (b) Increase contents of register *R4* and then transfer the incremented value to *R0*.
  - (c) Right-shift register *R1* and left-shift *R2* concurrently if control *T1* is 1.
- 8.2** A logic circuit with active-low synchronous reset has two control input flags, *Flag1* and *Flag2*. When both the flags are 0, register *R* is cleared to 0 and control remains in initial state. When both are 1, *G* is loaded to *R* and control goes to second state. When *Flag1* is 0 and *Flag2* is 1, *R* is right-shifted, while when *Flag1* is 1 and *Flag2* is 0, *R* is left-shifted. In the last two cases, control goes to a third state. Draw an ASM showing the controller, datapath unit (with internal registers), and signals.

- 8.3** Draw the ASMD charts for the following state transitions:
- (a) Start from state  $S_1$ ; if  $x = 0$ , generate a conditional operation  $R = 1$ ; if  $x = 1$  generate conditional operation  $R = R + 1$  and go from  $S_1$  to  $S_2$ .
  - (b) Start from state  $S_1$ ; control goes to  $S_2$  if  $x$  and  $y$  are same; otherwise control goes to  $S_3$ .
  - (c) Start from state  $S_1$  and monitor input  $x$ ; if two consecutive 1s are observed, go to state  $S_2$ . Remain in  $S_2$  until two consecutive 1s are observed.
- 8.4** Draw the ASMD chart and state transition chart for a logic circuit that goes from state  $S_1$  to state  $S_2$  if input is 00 and state  $S_3$  if input is 10; otherwise it remains in  $S_1$ .
- 8.5** Explain how the ASM and ASMD charts differ from a conventional flowchart. Using Fig. 8.5 as an illustration, show the difference in interpretation. Explain the difference between an ASM chart and an ASMD chart. In your own words, discuss the use and merit of using an ASMD chart.
- 8.6** Construct a block diagram and an ASMD chart for a digital system that counts the number of people in a room. The one door through which people enter the room has a photocell that changes a signal  $x$  from 1 to 0 while the light is interrupted. They leave the room from a second door with a similar photocell that changes a signal  $y$  from 1 to 0 while the light is interrupted. The datapath circuit consists of an up-down counter with a display that shows how many people are in the room.
- 8.7\*** Draw a block diagram and an ASMD chart for a circuit with two eight-bit registers  $RA$  and  $RB$  that receive two unsigned binary numbers. The circuit performs the subtraction operation
- $$RA \leftarrow RA - RB$$
- Use the method for subtraction described in Section 1.5, and set a borrow flip-flop to 1 if the answer is negative. Write and verify an HDL model of the circuit.
- 8.8\*** Design a digital circuit with three 16-bit registers  $AR$ ,  $BR$ , and  $CR$  that perform the following operations:
- (a) Transfer two 16-bit signed numbers (in 2's-complement representation) to  $AR$  and  $BR$ .
  - (b) If the number in  $AR$  is negative, divide the number in  $AR$  by 2 and transfer the result to register  $CR$ .
  - (c) If the number in  $AR$  is positive but nonzero, multiply the number in  $BR$  by 2 and transfer the result to register  $CR$ .
  - (d) If the number in  $AR$  is zero, clear register  $CR$  to 0.
  - (e) Write and verify a behavioral model of the circuit.
- 8.9\*** Design the controller whose state diagram is given by Fig. 8.11(a). Use one flip-flop per state (a one-hot assignment). Write, simulate, verify, and compare RTL and structural models of the controller.
- 8.10** The state diagram of a control unit is shown in Fig. P8.10. It has four states and two inputs  $x$  and  $y$ . Draw the equivalent ASM chart. Write and verify an HDL model of the controller.
- 8.11\*** Design the controller whose state diagram is shown in Fig. P8.10. Use  $D$  flip-flops.
- 8.12** Design the four-bit counter with synchronous clear specified in Fig. 8.10. Repeat for asynchronous clear.
- 8.13** Simulate *Design\_Example\_STR* (see HDL Example 8.4), and verify that its behavior matches that of the RTL description. Obtain state information by displaying  $G0$  and  $G1$  as a concatenated vector for the state.
- 8.14** What, if any, are the consequences of the machine in *Design\_Example\_RTL* (see HDL Example 8.2) entering an unused state?



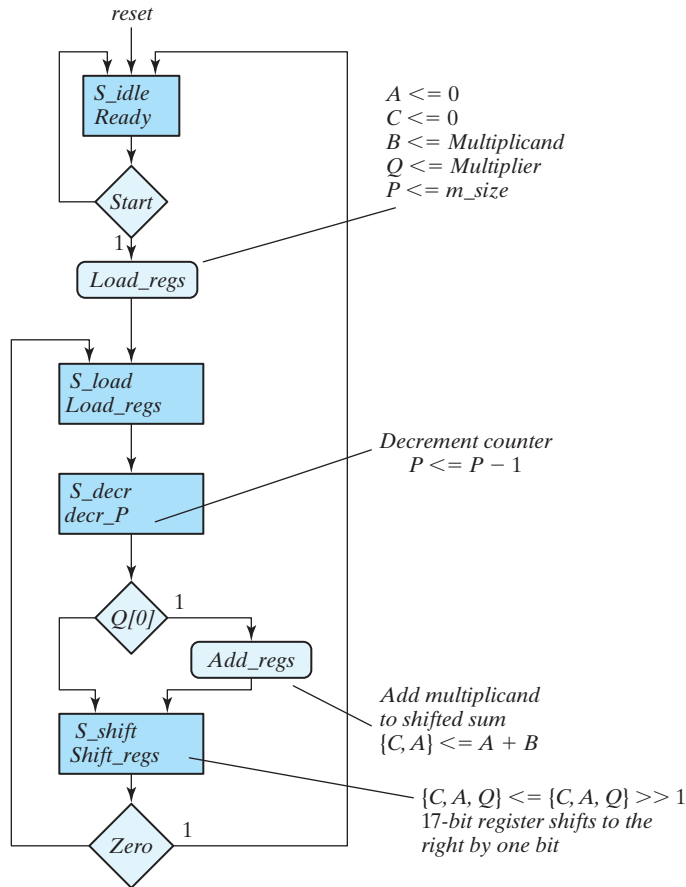
**FIGURE P8.10**  
Control state diagram for Problems 8.10 and 8.11

- 8.15** Simulate *Design\_Example\_RTL* in HDL Example 8.2, and verify that it recovers from an unexpected reset condition during its operation, that is, a “running reset” or a “reset on-the-fly.”
- 8.16\*** Develop a block diagram and an ASMD chart for a digital circuit that multiplies two binary numbers by the repeated-addition method. For example, to multiply  $5 \times 4$ , the digital system evaluates the product by adding the multiplicand four times:  $5 + 5 + 5 + 5 = 20$ . Design the circuit. Let the multiplicand be in register *BR*, the multiplier in register *AR*, and the product in register *PR*. An adder circuit adds the contents of *BR* to *PR*. A zero-detection signal indicates whether *AR* is 0. Write and verify a HDL behavioral model of the circuit.
- 8.17\*** Prove that the multiplication of two  $n$ -bit numbers gives a product of length less than or equal to  $2n$  bits.
- 8.18\*** In Fig. 8.14, the *Q* register holds the multiplier and the *B* register holds the multiplicand. Assume that each number consists of 32 bits.
- What is the maximum number of bits that can be expected in the product?
  - How many bits are in the *P* counter, and what is the binary number loaded into it initially?
  - What circuit will you include to check for zero in the *P* counter?
- 8.19** List the contents of registers *C*, *A*, *Q*, and *P* in a manner similar to Table 8.5 during the process of multiplying the two numbers 10101 (multiplicand) and 11001 (multiplier).
- 8.20\*** Determine the time it takes to process the multiplication operation in the binary multiplier described in Section 8.8. Assume that the *Q* register has  $n$  bits and the clock cycle is  $t$  ns.
- 8.21\*** Design the control circuit of the binary multiplier specified by the state diagram of Fig. 8.16, using multiplexers, a decoder, and a register.
- 8.22** Figure P8.22 shows an alternative ASMD chart for a sequential binary multiplier. Write and verify an RTL model of the system. Compare this design with that described by the ASMD chart in Fig. 8.15(b).
- 8.23** Figure P8.23 shows an alternative ASMD chart for a sequential binary multiplier. Write and verify an RTL model of the system. Compare this design with that described by the ASMD chart in Fig. 8.15(b).



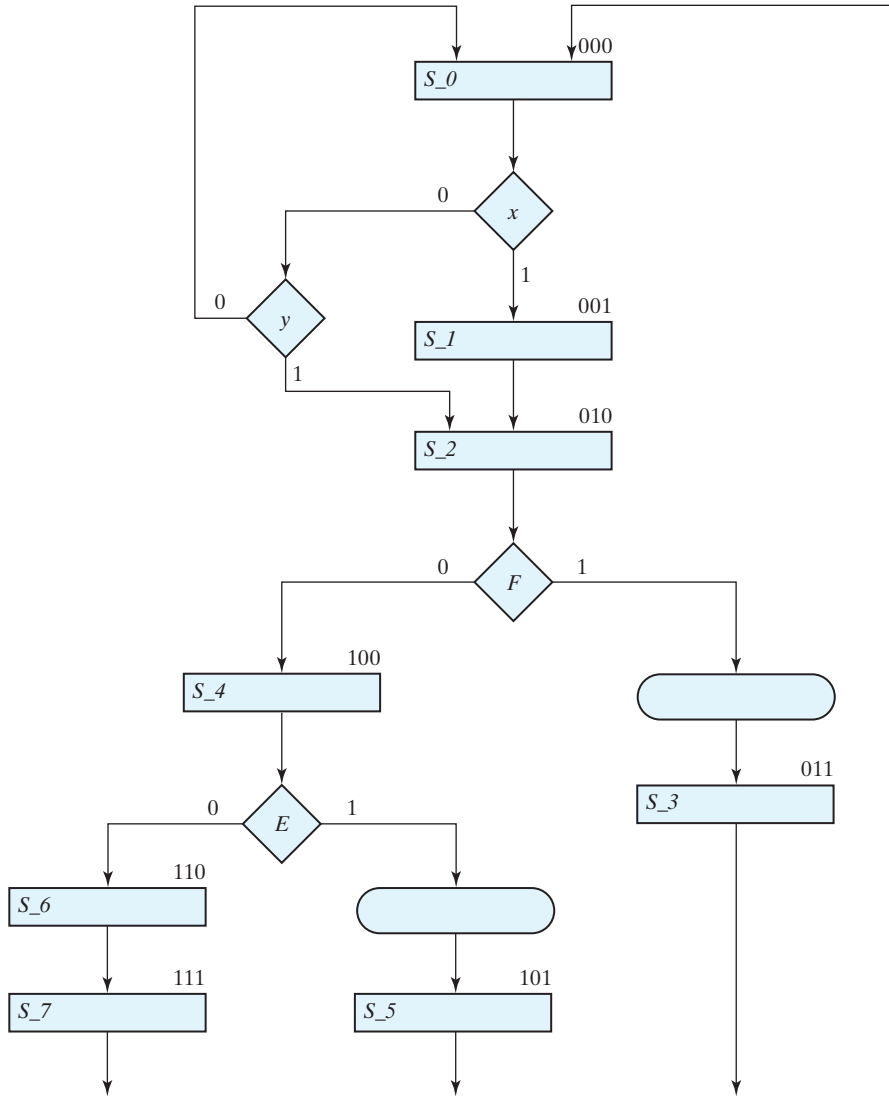
**FIGURE P8.22**  
ASMD chart for Problem 8.22

- 8.24** The HDL description of a sequential binary multiplier given in HDL Example 8.5 encapsulates the descriptions of the controller and the datapath in a single HDL module. Write and verify a model that encapsulates the controller and datapath in separate modules.
- 8.25** The sequential binary multiplier described by the ASMD chart in Fig. 8.15 does not consider whether the multiplicand or the shifted multiplier is 0. Therefore, it executes for a fixed number of clock cycles, independently of the data.
- Develop an ASMD chart for a more efficient multiplier that will terminate execution as soon as either word is found to be zero.
  - Write an HDL description of the circuit. The controller and datapath are to be encapsulated in separate design units.
  - Write a test plan and a testbench, and verify the circuit.



**FIGURE P8.23**  
ASMD chart for Problem 8.23

- 8.26** Modify the ASMD chart of the sequential binary multiplier shown in Fig. 8.15 to add and shift in the same clock cycle. Write and verify an RTL description of the system.
- 8.27** The second testbench given in HDL Example 8.6 generates a product for all possible values of the multiplicand and multiplier. Verifying that each result is correct would not be practical, so modify the testbench to include a statement that forms the expected product. Write additional statements to compare the result produced by the RTL description with the expected result. Your simulation is to produce an error signal indicating the result of the comparison. Repeat for the structural model of the multiplier.
- 8.28** Write the HDL structural description of the multiplier designed in Section 8.8. Use the block diagram of Fig. 8.14(a) and the control circuit of Fig. 8.18. Simulate the design and verify its functionality by using the testbench of HDL Example 8.6.
- 8.29** An incomplete ASMD chart for a finite state machine is shown in Fig. P8.29. The register operations are not specified, because we are interested only in designing the control logic.
- Draw the equivalent state diagram.
  - Design the control unit with one flip-flop per state.



**FIGURE P8.29**  
ASMD chart for Problem 8.29

- List the state table for the control unit.
- Design the control unit with three *D* flip-flops, a decoder, and gates.
- Derive a table showing the multiplexer input conditions for the control unit.
- Design the control unit with three multiplexers, a register with three flip-flops, and a  $3 \times 8$  decoder.
- Using the results of (f), write and verify a structural model of the controller.
- Write and verify an RTL description of the controller.

**8.30\*** If  $RA = 0011$  and  $RB = 1001$ , find their values after execution of each HDL block.

**Verilog** (block and nonblocking operators  $RA$  and  $RB$  are registers)

- |                                        |                                         |
|----------------------------------------|-----------------------------------------|
| (a) $RA = RA + RB;$                    | (b) $RA <= RA + RB;$                    |
| <b>if</b> ( $RA > RB$ ) $RB = RB - 1;$ | <b>if</b> ( $RA > RB$ ) $RB <= RB - 1;$ |
| <b>else</b> $RB = RB + 1;$             | <b>else</b> $RB <= RB + 1;$             |

**VHDL** (Variable and signal assignments,  $RA$  and  $RB$  are variables/signals)

- |                                                     |                                                     |
|-----------------------------------------------------|-----------------------------------------------------|
| (a) $RA := RA + RB;$                                | (b) $RA <= RA + RB;$                                |
| <b>if</b> ( $RA > RB$ ) <b>then</b> $RB := RB - 1;$ | <b>if</b> ( $RA > RB$ ) <b>then</b> $RB <= RB - 1;$ |
| <b>else</b> $RB := RB + 1;$                         | <b>else</b> $RB <= RB + 1;$                         |

**8.31\*** Using Verilog operators listed in Table 8.2, assume that  $A = 4'b1001$ ,  $B = 4'b0110$ , and  $C = 4'b0001$  and evaluate the result of the following operations:

$A + B$ ;  $A - B$ ;  $A * C$ ;  $\sim B$ ;  $!A$ ;  $A >> 1$ ;  $A >>> 1$ ;  $B \& C$ ;  $B \&\& C$ ;  $B | C$ ;  
 $B | C$ ;  $A \wedge B$ ;  $\sim(A | B)$ ;  $\sim(A \& C)$ ;  $A < B$

**8.32** Consider the following always block:

```
always @ (posedge CLK)
if (S1) then  $R1 <= R1 + R2;$ 
else if (S2)  $R1 <= R1 + 1;$ 
else  $R1 <= R1;$ 
```

Using a four-bit counter with parallel load for  $R1$  (as in Fig. 6.15) and a four-bit adder, draw a block diagram showing the connections of components and control signals for a possible synthesis of the block.

**8.33** The multilevel **case** statement is often translated by a logic synthesizer into hardware multiplexers. How would you translate the following **case** block into hardware (assume registers of eight bits each)?

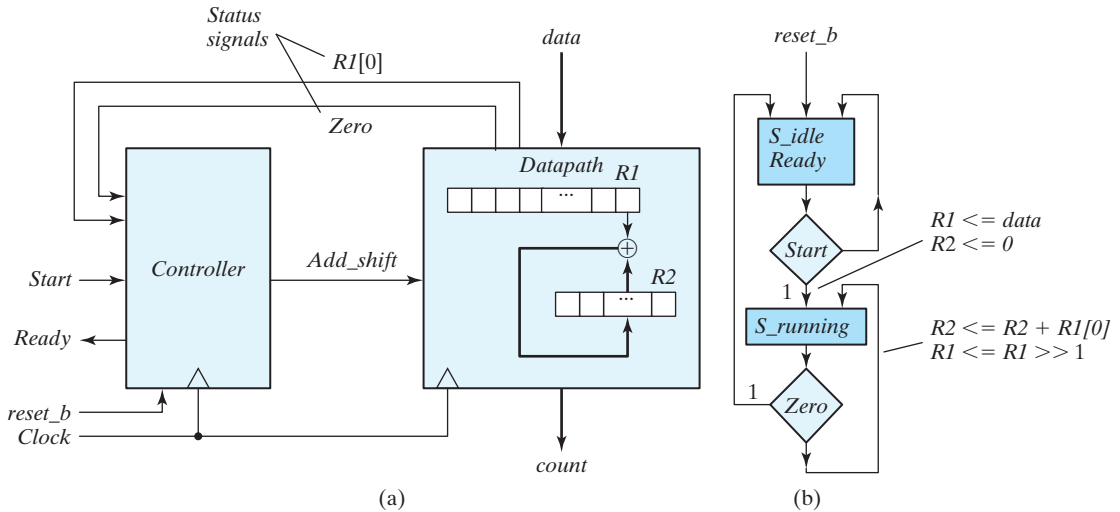
```
case (state)
  S0:  $R4 = R0;$ 
  S1:  $R4 = R1;$ 
  S2:  $R4 = R2;$ 
  S3:  $R4 = R3;$ 
endcase
```

**8.34** The design of a circuit that counts the number of ones in a register is carried out in Section 8.10. The block diagram for the circuit is shown in Fig. 8.22(a), a complete ASMD chart for the circuit appears in Fig. 8.22(c), and structural HDL models of the datapath and controller are given in HDL Example 8.8. Using the operations and signal names indicated on the ASMD chart,

- Write *Datapath\_BEH*, an RTL description of the datapath unit of the ones counter. Write a test plan specifying the functionality that will be tested, and write a test-bench to implement the plan. Execute the test plan to verify the functionality of the datapath unit, and produce annotated simulation results relating the test plan to the waveforms produced in a simulation.
- Write *Controller\_BEH*, an RTL description of the control unit of the ones counter. Write a test plan specifying the functionality that will be tested, and write a test-bench to implement the plan. Execute the test plan to verify the functionality of the control unit, and produce annotated simulation results relating the test plan to the waveforms produced in a simulation.

- (c) Write *Count\_Ones\_BEH\_BEH*, a top-level module encapsulating and integrating *Controller\_BEH* and *Datapath\_BEH*. Write a test plan and a testbench, and verify the description. Produce annotated simulation results relating the test plan to the waveforms produced in a simulation.
  - (d) Write *Controller\_BEH\_1Hot*, an RTL description of a one-hot controller implementing the ASMD chart of Fig. 8.22(c). Write a test plan specifying the functionality that will be tested, and write a testbench to implement the plan. Execute the test plan and produce annotated simulation results relating the test plan to the waveforms produced in a simulation.
  - (e) Write *Count\_Ones\_BEH\_1\_Hot*, a top-level module encapsulating the module *Controller\_BEH\_1\_Hot* and *Datapath\_BEH*. Write a test plan and a testbench, and verify the description. Produce annotated simulation results relating the test plan to the waveforms produced in a simulation.
- 8.35** The HDL description and testbench for a circuit that counts the number of ones in a register are given in HDL Example 8.8. Modify the testbench and simulate the circuit to verify that the system operates correctly for the following patterns of data: 8'hff, 8'h0f, 8'hf0, 8'h00, 8'haa, 8'h0a, 8'ha0, 8'h55, 8'h05, 8'h50, 8'ha5, and 8'h5a.
- 8.36** The design of a circuit that counts the number of ones in a register is carried out in Section 8.10. The block diagram for the circuit is shown in Fig. 8.22(a), a complete ASMD chart for this circuit appears in Fig. 8.22(c), and structural HDL models of the datapath and controller are given in HDL Example 8.8. Using the operations and signal names indicated on the ASMD chart,
- (a) Design the control logic, employing one flip-flop per state (a one-hot assignment). List the input equations for the four flip-flops.
  - (b) Write *Controller\_Gates\_1\_Hot*, a gate-level HDL structural description of the circuit, using the control designed in part (a) and the signals shown in the block diagram of Fig. 8.22(a).
  - (c) Write a test plan and a testbench, and then verify the controller.
  - (d) Write *Count\_Ones\_Gates\_1\_Hot\_STR*, a top-level module encapsulating and integrating instantiations of *Controller\_Gates\_1\_Hot* and *Datapath\_STR*. Write a test plan and a testbench to verify the description. Produce annotated simulation results relating the test plan to the waveforms produced in a simulation.
- 8.37** Compared with the circuit presented in HDL Example 8.8, a more efficient circuit that counts the number of ones in a data word is described by the block diagram and the partially completed ASMD chart in Fig. P8.37. This circuit accomplishes addition and shifting in the same clock cycle and adds the LSB of the data register to the counter register at every clock cycle.
- (a) Complete the ASMD chart.
  - (b) Using the ASMD chart, write an RTL description of the circuit. A top-level design unit, *Count\_of\_ones\_2\_Beh* is to instantiate separate modules for the datapath and control units.
  - (c) Design the control logic, using one flip-flop per state (a one-hot assignment). List the input equations for the flip-flops.
  - (d) Write the HDL structural description of the circuit, using the controller designed in part (c) and the block diagram of Fig. P8.37(a).
  - (e) Write a testbench to test the circuit. Simulate the circuit to verify the operation described in both the RTL and the structural programs.
- 8.38** The addition of two signed binary numbers in the signed-magnitude representation follows the rules of ordinary arithmetic: If the two numbers have the same sign (both positive or



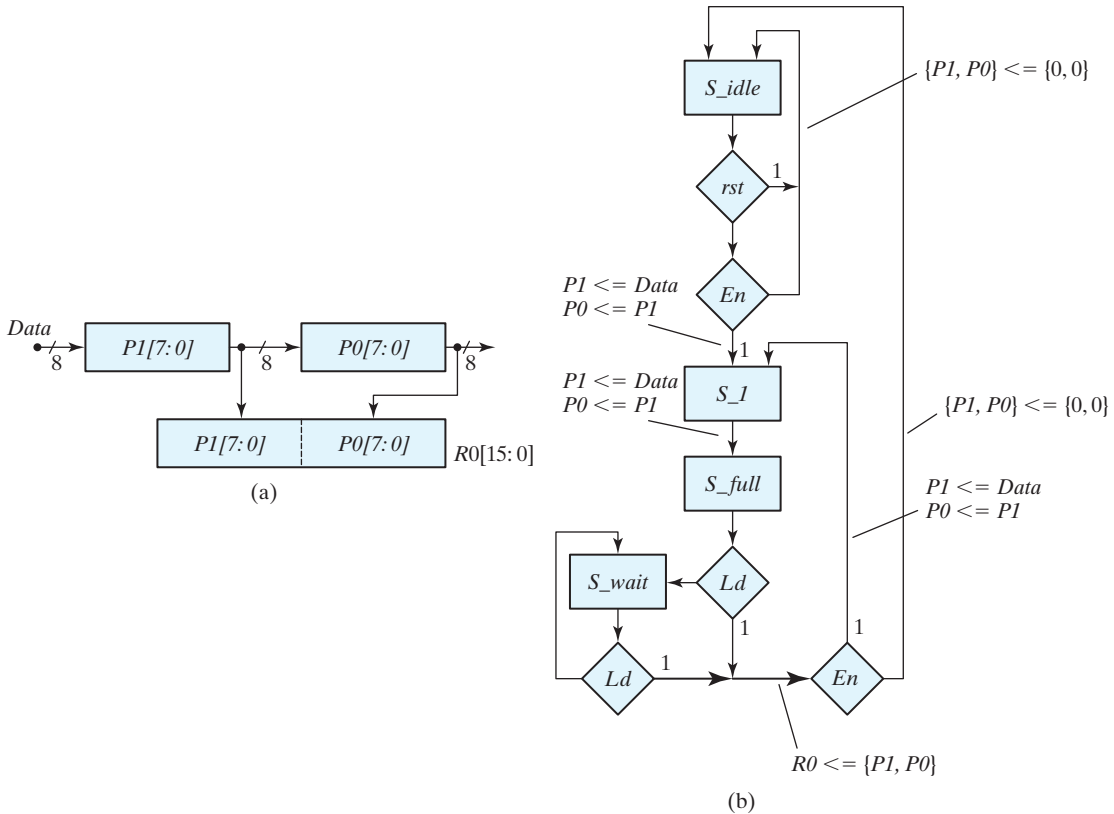
**FIGURE P8.37**

(a) Alternative circuit for a ones counter

(b) ASMD Chart for Problem 8.37

both negative), the two magnitudes are added and the sum has the common sign; if the two numbers have opposite signs, the smaller magnitude is subtracted from the larger and the result has the sign of the larger magnitude. Write an HDL behavioral description for adding two 8-bit signed numbers in signed-magnitude representation and verify. The leftmost bit of the number holds the sign and the other seven bits hold the magnitude.

- 8.39\*** For the circuit designed in Problem 8.16,
- Write and verify a structural HDL description of the circuit. The datapath and controller are to be described in separate units.
  - Write and verify an RTL description of the circuit. The datapath and controller are to be described in separate units.
- 8.40** Modify the block diagram of the sequential multiplier given in Fig. 8.14(a) and the ASMD chart in Fig. 8.15(b) to describe a system that multiplies 32-bit words, but with 8-bit (byte-wide) external datapaths. The machine is to assert *Ready* in the (initial) reset state. When *Start* is asserted, the machine is to fetch the data bytes from a single 8-bit data bus in consecutive clock cycles (multiplicand bytes first, followed by multiplier bytes, least significant byte first) and store the data in datapath registers. *Got\_Data* is to be asserted for one cycle of the clock when the transfer is complete. When *Run* is asserted, the product is to be formed sequentially. *Done\_Product* is to be asserted for one clock cycle when the multiplication is complete. When a signal *Send\_Data* is asserted, each byte of the product is to be placed on an 8-bit output bus for one clock cycle, in sequence, beginning with the least significant byte. The machine is to return to the initial state after the product has been transmitted. Consider safeguards, such as not attempting to send or receive data while the product is being formed. Consider also other features that might eliminate needless multiplication by 0. For example, do not continue to multiply if the shifted multiplier is empty of 1's.
- 8.41** The block diagram and partially completed ASMD chart in Fig. P8.41 describe the behavior of a two-stage pipeline that acts as a 2:1 decimator with a parallel input and output. Decimators

**FIGURE P8.41****Two-stage pipeline register: Datapath unit and ASMD chart**

are used in digital signal processors to move data from a datapath with a high clock rate to a datapath with a lower clock rate, converting data from a parallel format to a serial format in the process. In the datapath shown, entire words of data can be transferred into the pipeline at twice the rate at which the contents of the pipeline must be dumped into a holding register or consumed by some processor. The contents of the holding register *R0* can be shifted out serially, to accomplish an overall parallel-to-serial conversion of the data stream. The ASMD chart indicates that the machine has synchronous reset to *S\_idle*, where it waits until *rst* is de-asserted and *En* is asserted. Note that synchronous transitions which would occur from the other states to *S\_idle* under the action of *rst* are not shown. With *En* asserted, the machine transitions from *S\_idle* to *S\_I*, accompanied by concurrent register operations that load the MSByte of the pipe with *Data* and move the content of *P1* to the LSByte (*P0*). At the next clock, the state goes to *S\_full*, and now the pipe is full. If *Ld* is asserted at the next clock, the machine moves to *S\_I* while dumping the pipe into a holding register *R0*. If *Ld* is not asserted, the machine enters *S\_wait* and remains there until *Ld* is asserted, at which time it dumps the pipe and returns to *S\_I* or to *S\_idle*, depending on whether *En* is asserted, too. The data rate at *R0* is one-half the rate at which data are supplied to the unit from an external datapath.

(a) Develop the complete ASMD chart.

(b) Using the ASMD chart developed in (a), write and verify an HDL model of the datapath.

- (c) Write and verify a HDL behavioral model of the control unit.
- (d) Encapsulate the datapath and controller in a top-level design unit, and verify the integrated system.

- 8.42** The count-of-ones circuit described in Fig. 8.22 has a latency that is to be eliminated. It arises because the status signal  $E$  is formed as the output of a flip-flop into which the MSB of  $R1$  is shifted. Develop a design that eliminates the latency.
- 8.43** Write a HDL model of a finite state machine that functions as a divide-by-four counter by asserting output  $y\_out$  every fourth pulse of the clock signal ( $clk$ ). The machine is to have active-low asynchronous reset signal  $rst\_b$ .
- 8.44** Draw the logic diagram implied by the following SystemVerilog model:

```

module Prob_xyz_sv
  input in_1, in_2, in3,
  output y_out
);
logic sig_1;

  always_ff (negedge clk) begin
    sig_1 <= in_1 & in_2;
    y_out <= sig_1 | in_3;
  end
endmodule

```

- 8.45** Do the following (Verilog) statements produce different results?

```

always_ff @ (negedge clk) begin
  y1 <= x1 & x2;
  y2 <= x4 | x3;
end

always_ff @ (negedge clk) begin
  y2 <= x4 | x3;
  y1 <= x1 & x2;
end

```

- 8.46** Do the following (Verilog) statements produce different results?

```

always_ff @ (negedge clk) begin
  y1 = x1 & x2;
  y2 = x4 | x3;
end

always_ff @ (negedge clk) begin
  y2 = x4 | x3;
  y1 = x1 & x2;
end

y2 := x4 | x3;

```

```

    y1 := x1 & x2;
end if;
end

```

**8.47** Find the error in the code for the following finite state machine:

```

module Clock_Divider (input logic clk, rst, output logic y_out);

    logic [1:0] state, next_state;
    parameter      s0 = 2'b00,
                  s1 = 2'b01,
                  s2 = 2'b10;

    // State transitions
    always_ff @ (posedge clk, negedge rst)
        if (rst == 0) state <= s0; else state <= next_state;

    // Next state logic

    always_comb (state)
        case (state)
            s0: next_state = s1;
            s1: next_state = s2;
            s2: next_state = s0;
        endcase

    // Output logic

    assign y_out = (state == s2);
endmodule

```

**8.48** Find the error in the following model of a pseudo flip-flop:

```

module pseudo_flop (
    input logic clk, rst, set, data,
    output logic q
);

    always_ff (posedge clk, negedge rst)
        if (!rst) q <= 0;
        else q <= data;

    always @ (set)
        if (set) q <= 1;
endmodule

```

**8.49** Explain why the following code does not describe a transparent latch, and explain how to repair the model:

```

module pseudo_latch (
    input logic enable,

```

```

    input logic data,
    output logic q
);
    always_latch @ (enable)
        if (enable) q <= data;
endmodule

```

**8.50** Draw the logic diagram of the circuit described by the following VHDL process:

```

process (clock) begin
    if clock'event and clock = '0' then begin
        VRA := VRA + VRB;    -- Variable assignment
        VRD := VRA;
        RA <= VRA + VRB;    -- Signal assignment
        RD <= VRA;
    end
end process;

```

**8.51** Using SystemVerilog constructs, write a statement declaring *state\_type*, an enumerated type having values s0, s1, s2, s, s4. Declare state and next-state to have type state\_type.

**8.52** Write a SystemVerilog procedural statement to describe the following combinational logic:

```

y1 = A + B;
y2 = A|B;

```

**8.53** Write a SystemVerilog description of a 8-bit data register having synchronous load and asynchronous reset.

**8.54** Write a SystemVerilog description of an 8-bit data latch having active-high enable.

**8.55** Write a SystemVerilog description of a  $3 \times 8$  decoder.

**8.56** Write a SystemVerilog description of a 4-bit priority decoder.

**8.57** Write a SystemVerilog description of a divide-by-five FSM having active-low asynchronous reset.

**8.58** Write a SystemVerilog declaration of a one-hot encoding of enumerated states s0, s1, s2, s3.

## REFERENCES

- 1.** ARNOLD, M. G. 1999. *Verilog Digital Computer Design*. Upper Saddle River, NJ: Prentice Hall.
- 2.** BHASKER, J. 1997. *A Verilog HDL Primer*. Allentown, PA: Star Galaxy Press.
- 3.** BHASKER, J. 1998. *Verilog HDL Synthesis*. Allentown, PA: Star Galaxy Press.
- 4.** CILETTI, M. D. 2003. *Modeling, Synthesis, and Rapid Prototyping with Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.
- 5.** CILETTI, M. D. 2010. *Advanced Digital Design with the Verilog HDL*. Upper Saddle River, NJ: Prentice Hall.

6. CLARE, C. R. 1971. *Designing Logic Systems Using State Machines*. New York: McGraw-Hill.
7. HAYES, J. P. 1993. *Introduction to Digital Logic Design*. Reading, MA: Addison-Wesley.
8. *IEEE Standard Hardware Description Language Based on the Verilog Hardware Description Language* (IEEE Std 1364-2005). 2005. New York: Institute of Electrical and Electronics Engineers.
9. MANO, M. M. 1993. *Computer System Architecture*, 3rd ed., Upper Saddle River, NJ: Prentice Hall.
10. MANO, M. M., and C. R. KIME. 2005. *Logic and Computer Design Fundamentals*, 3rd ed., Upper Saddle River, NJ: Prentice Hall.
11. PALNITKAR, S. 2003. *Verilog HDL: A Guide to Digital Design and Synthesis*. Mountain View, CA: SunSoft Press (a Prentice Hall Title).
12. SMITH, D. J. 1996. *HDL Chip Design*. Madison, AL: Doone Publications.
13. THOMAS, D. E., and P. R. MOORBY. 2002. *The Verilog Hardware Description Language*, 5th ed., Boston: Kluwer Academic Publishers.
14. WINKLER, D., and F. PROSSER. 1987. *The Art of Digital Design*, 2nd ed., Englewood Cliffs, NJ: Prentice-Hall.
15. SUTHERLAND, S., S. DAVIDMANN, and P. FLAKE, 2004. *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design*, Boston: Kluwer Academic Publishers.
16. GAJSKI, D. et al. "Essential Issues in Design," in: Staunstrup, J. Wolf W. Eds., *Hardware Software Co-Design: Principles and Practices*. Boston, MA: Kluwer, 1997.
17. 1800-2012 IEEE Standard for SystemVerilog: Unified Hardware Design, Specification, and Verification Language, IEEE, Piscataway, New Jersey. Copyright 2013. ISBN 978—0-7381-8110-390 (PDF), 978—0-7381-8111-390 (print).
18. SUTHERLAND, S., and D. MILLS, "Synthesizing SystemVerilog—Busting the Myth that SystemVerilog is only for Verification," (SNUG)<sup>23</sup> Conference, San Jose, CA, 2013.

## WEB SEARCH TOPICS

---

Algorithmic state machine  
 Algorithmic state machine chart  
 Asynchronous circuit  
 Decimator  
 Digital control unit  
 Digital datapath unit  
 Mealy machine  
 Moore machine  
 Race condition

<sup>23</sup>Synopsys Users Group.